

STUDY GUIDE · DOCKER · FROM BEGINNER TO ADVANCED

Docker Practical Course

Docker From Beginner to Advanced

Every one of the 116 lessons on a page or less: what it teaches, its diagram, its commands and its recap. Then the 25 troubleshooting problems by symptom, the projects, the capstone, the command reference, a glossary and 28 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-docker-practical-course>

Contents

[How to use this study guide](#)

[The method and the big picture](#)

[Module 01 · Container fundamentals](#)

[Module 02 · Containers](#)

[Module 03 · Images](#)

[Module 04 · Dockerfiles](#)

[Module 05 · Docker builds](#)

[Module 06 · Application containerization](#)

[Module 07 · Networking](#)

[Module 08 · Storage](#)

[Module 09 · Environment configuration](#)

[Module 10 · Docker Compose](#)

[Module 11 · Registries](#)

[Module 12 · Container security](#)

[Module 13 · Multi-stage builds](#)

[Module 14 · Observability](#)

[Module 15 · Resource management](#)

[Module 16 · Advanced Docker](#)

[Module 18 · Production](#)

[Module 19 · Docker in CI/CD](#)

[Module 20 · Docker and Kubernetes](#)

[Module 17 · Troubleshooting](#)

[Module 21 · Projects](#)

[Capstone · a production-style container stack](#)

[Command reference](#)

[Glossary](#)

[Docker interview questions](#)

The full lessons, with every command and its real, tested output, the exercises, challenges, assessments, troubleshooting labs and projects, are in the repository.

How to use this study guide

This guide is the revision companion to the course. Every lesson is condensed to what it teaches, its diagram, its command table and its recap. The full lessons, with every command, its real output, the labs, the challenges and their solutions, live in the repository: each summary links to its lesson.

How to study

1. **Do the lesson first, read the summary afterwards.** Docker is learned with your hands: run every command of a lesson, break it the way the *Break it* section does, and read the real error before you read the explanation.
2. **Read the error's last line.** Almost every Docker error says what went wrong: `port is already allocated`, `executable file not found`, `Read-only file system`, `401 Unauthorized`. The course teaches you to trust it.
3. **Look before you fix.** `docker ps -a`, `docker logs`, `docker inspect`, `docker exec`: the four commands that answer most questions. Troubleshooting problems 01–25 practise them in the order you would use them.
4. **End every module with its assessment:** a knowledge check, a practical task, a troubleshooting task and a real-world scenario. Redo the tasks you could not do without the answer.
5. **Finish with the capstone and the final exam.** The exam grades 13 tasks against the real state of your engine.

A study plan

Week	Modules	Goal
1	01–04	run, inspect and remove containers; read images; write Dockerfiles
2	05–09	fast builds; containerize five stacks; networks, storage and configuration
3	10–13	multi-container applications with Compose; registries; security; multi-stage builds
4	14–17	healthchecks, logs, limits, Buildx; the 25 troubleshooting problems
5	18–22	production practice, CI/CD, Kubernetes; projects, capstone and final exam

One hour a day is enough: a lesson takes 15–30 minutes, plus its challenge.

The method and the big picture

How a lesson works

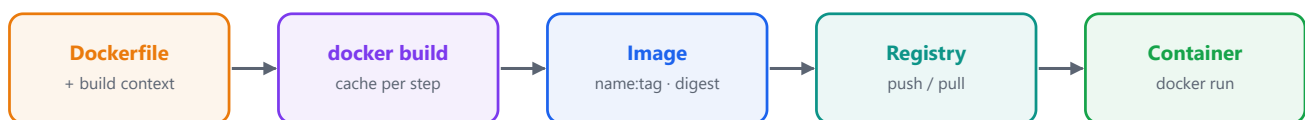
Every lesson folder has the full lesson in `README.md`, plus the same content split for practice:

File	Contents
<code>README.md</code>	what, visual, lab setup, demonstration, command breakdown, lab, break it, troubleshoot it, fix it, challenge, real-world example, recap, cleanup
<code>lab.md</code>	the hands-on lab: instructions, expected result, verification
<code>challenge.md</code> / <code>solution.md</code>	the practice challenge, and its solution in a separate file
<code>troubleshooting.md</code>	the intentional failure, its investigation and the fix
<code>commands.md</code>	every command of the lesson, in order
<code>examples/</code>	the files the lesson uses: <code>before/</code> , <code>broken/</code> , <code>fixed/</code> , <code>final/</code> where useful

Every module ends with an **assessment**: a knowledge check, a practical task, a troubleshooting task and a real-world scenario.

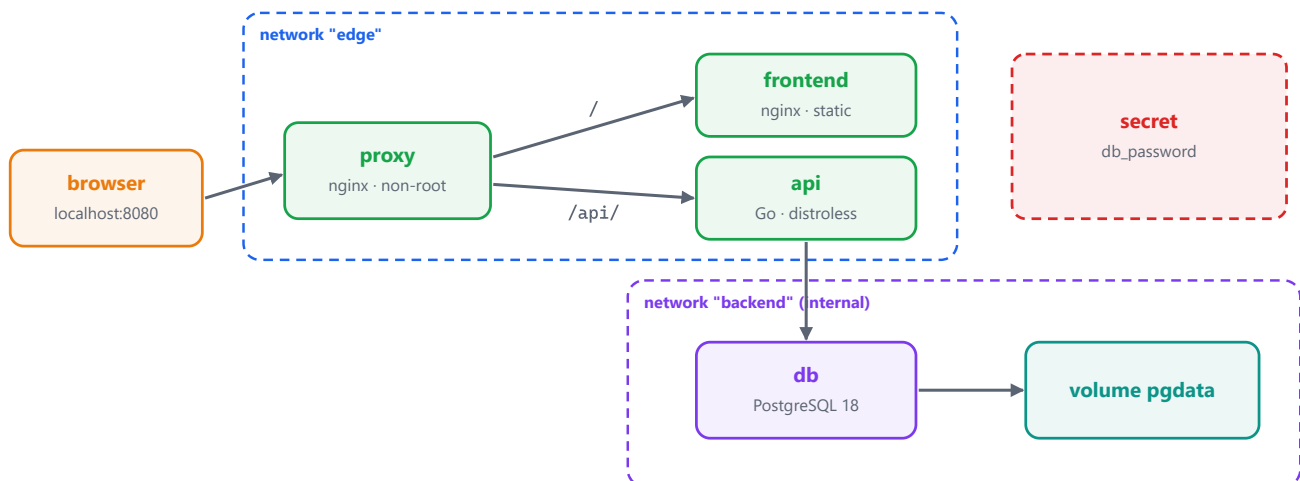
Diagrams

From Dockerfile to running container

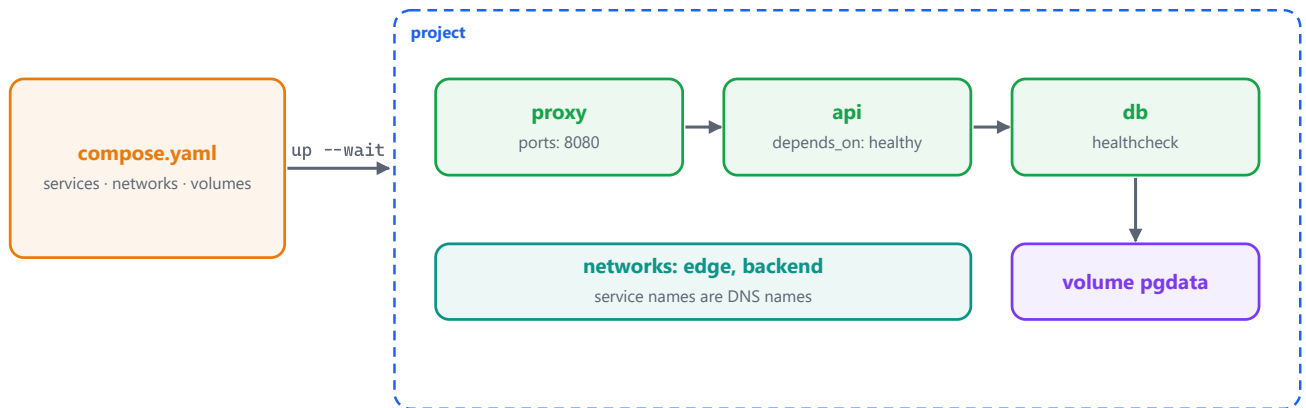


.dockerignore keeps secrets and junk out of the context · tags move, digests do not

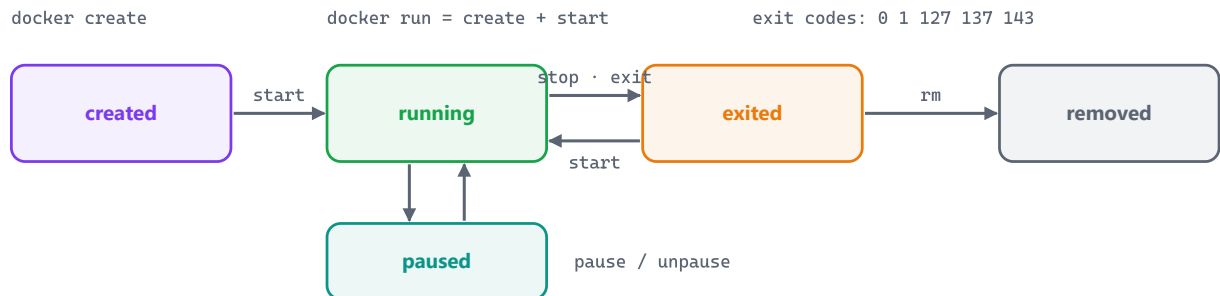
Capstone: a production-style stack



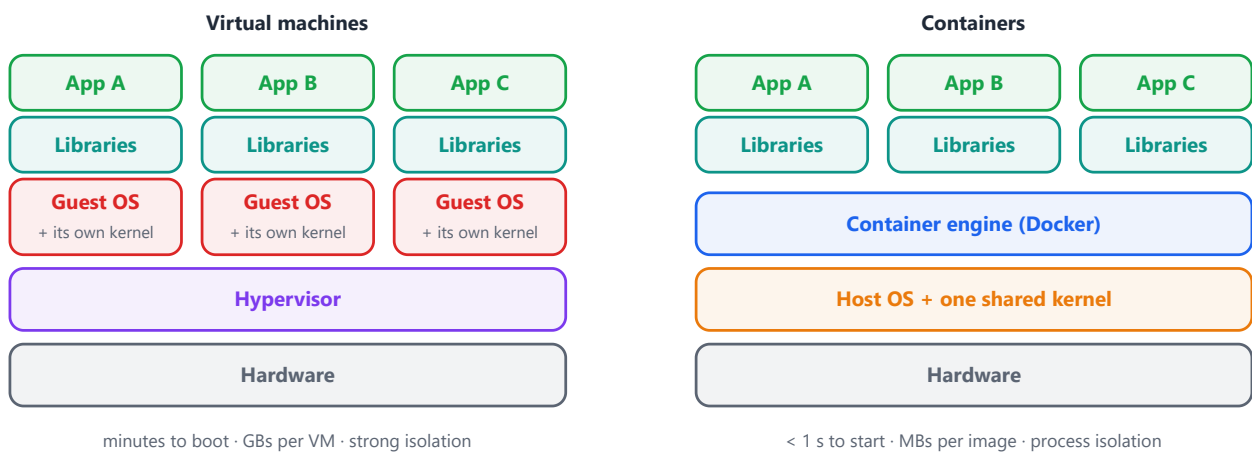
Docker Compose: the whole application in one file



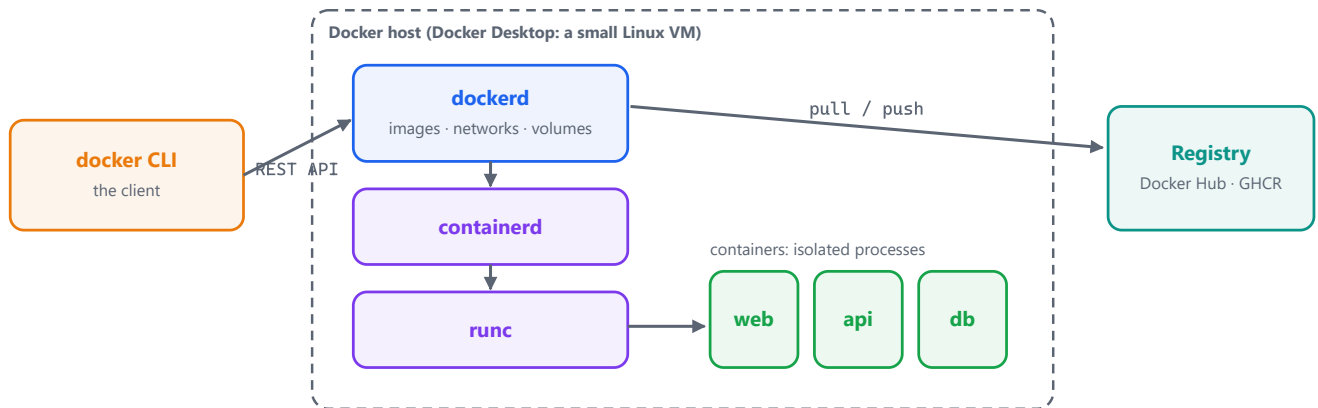
Container lifecycle



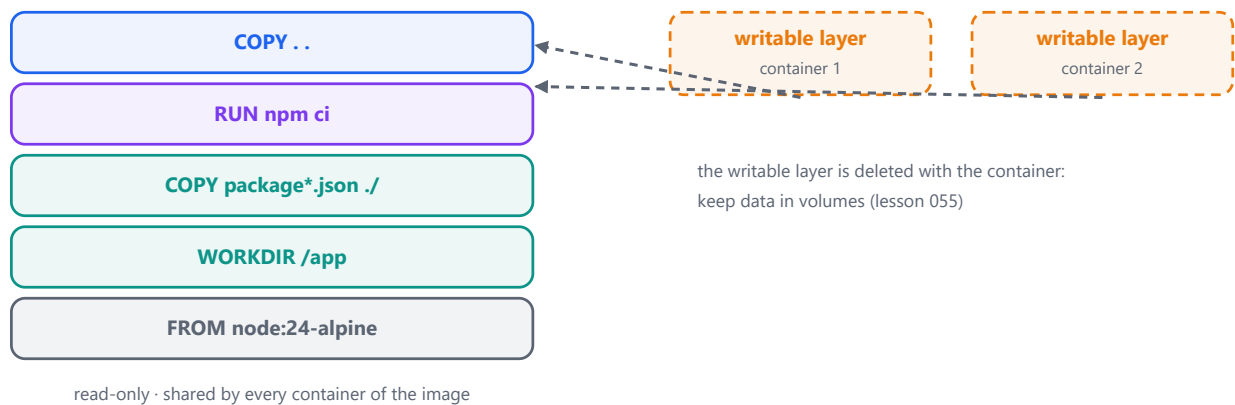
Virtual machines vs containers



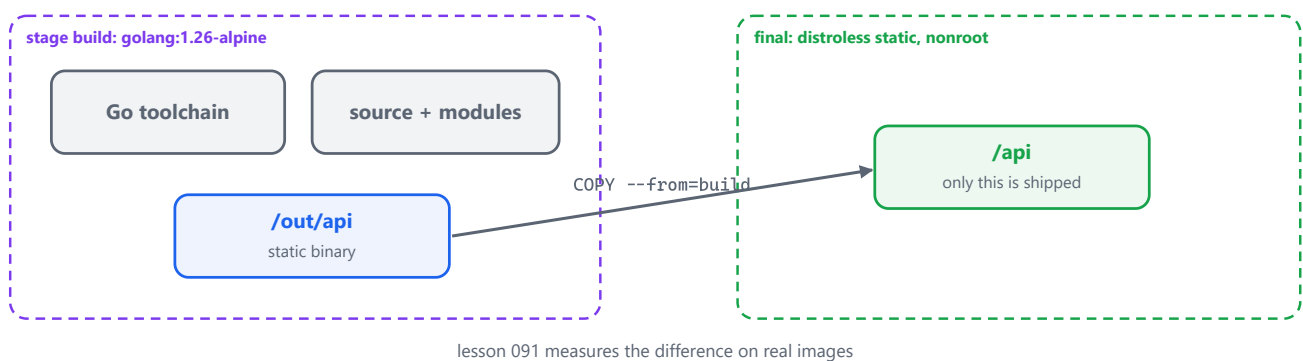
Docker architecture: the client asks, the engine works



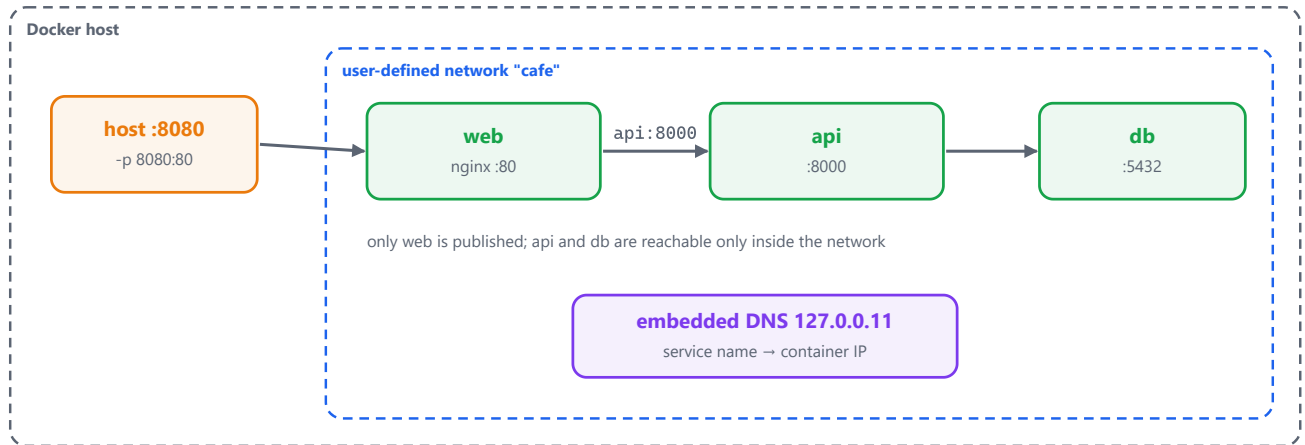
Images are layers; a container adds one writable layer



Multi-stage build: compile big, ship small



Networking: publish to the host, talk by name inside



Container hardening, layer by layer

minimal, pinned base
fewer packages, fewer CVEs

non-root USER
not uid 0

read-only root file system
+ tmpfs where needed

cap_drop: ALL
add back only what is needed

no-new-privileges
no setuid escalation

memory · CPU · pids limits
one container cannot starve the host

secrets as files at run time, never in ENV, ARG or a layer · never mount docker.sock

Where container data lives

writable layer
dies with the container

(nothing to add)
temporary files only

named volume
managed by Docker · survives

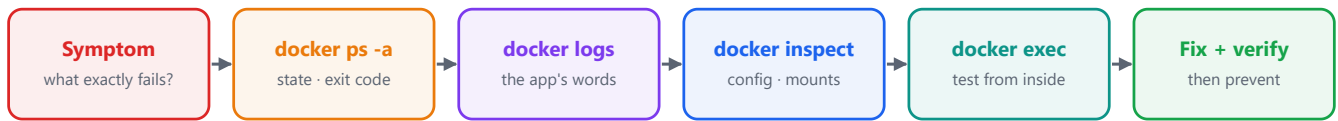
-v pgdata:/var/lib/postgresql
databases, uploads

bind mount
a host folder · you manage it

-v "\${PWD}":/app
source code in development

tmpfs
in memory · for --read-only

The troubleshooting method



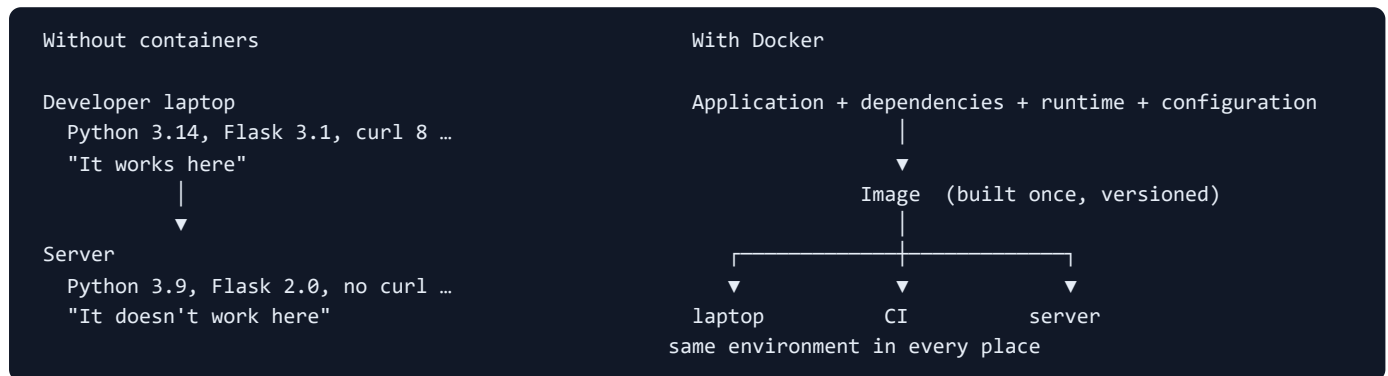
the 25 problems of module 17 follow these steps, from symptom to prevention

Module 01 · Container fundamentals

Lesson 001 · What is Docker?

Docker packages an application **together with everything it needs to run**: the runtime, the libraries, the dependencies and the configuration. That package (an **image**) runs as a **container**: the same environment on your laptop, on a colleague's laptop, in CI and on a production server. That is the end of "it works on my machine".

VISUAL



COMMANDS

Command / part	Meaning
<code>docker run IMAGE COMMAND</code>	create a container from IMAGE and run COMMAND in it
<code>--rm</code>	remove the container when it stops (lessons 008, 009)
<code>python:3.14-slim</code>	an image: repository <code>python</code> , tag <code>3.14-slim</code> (lesson 021)
<code>docker version</code>	the Docker client and engine versions (lesson 005)

RECAP

- Docker packages an application with its runtime, dependencies and configuration into an image.
- A container is a running instance of an image: an isolated process with its own file system.
- The same image behaves the same on every machine: no more "it works on my machine".

Full lesson with every command and its real output: [001-what-is-docker](#)

Lesson 002 · Why containers?

An application is never just its code. It needs a runtime, libraries, system packages and configuration, all in the right versions. The traditional way installs them on each server by hand, and every server slowly becomes different. A container image ships the application **and** that whole stack, so what you tested is exactly what runs.

VISUAL

What an application really needs	Traditional deployment	Container deployment
Application code	server 1: Python 3.11, lib 1.2	image cafe-api:1.4.0
+ Runtime (Python 3.14)	server 2: Python 3.12, lib 1.4	= code + runtime +
libraries		
+ Libraries (Flask 3.1)	server 3: Python 3.11, lib 1.3	+ dependencies + config
defaults		
+ System dependencies (libpq, CA certs)	→ "works on server 2 only"	→ identical everywhere it
runs		
+ Configuration (ports, URLs)	manual installs, slow drift	built once, run anywhere

COMMANDS

Part	What it does
<code>-v "\$(pwd):/app"</code>	make the current folder visible inside the container (a bind mount, lesson 054)
<code>-w /app</code>	run the command in that folder
<code>docker build -t NAME:TAG .</code>	package the folder's Dockerfile into an image (lesson 035)
<code>docker run IMAGE COMMAND</code>	run a command in a new container of that image

RECAP

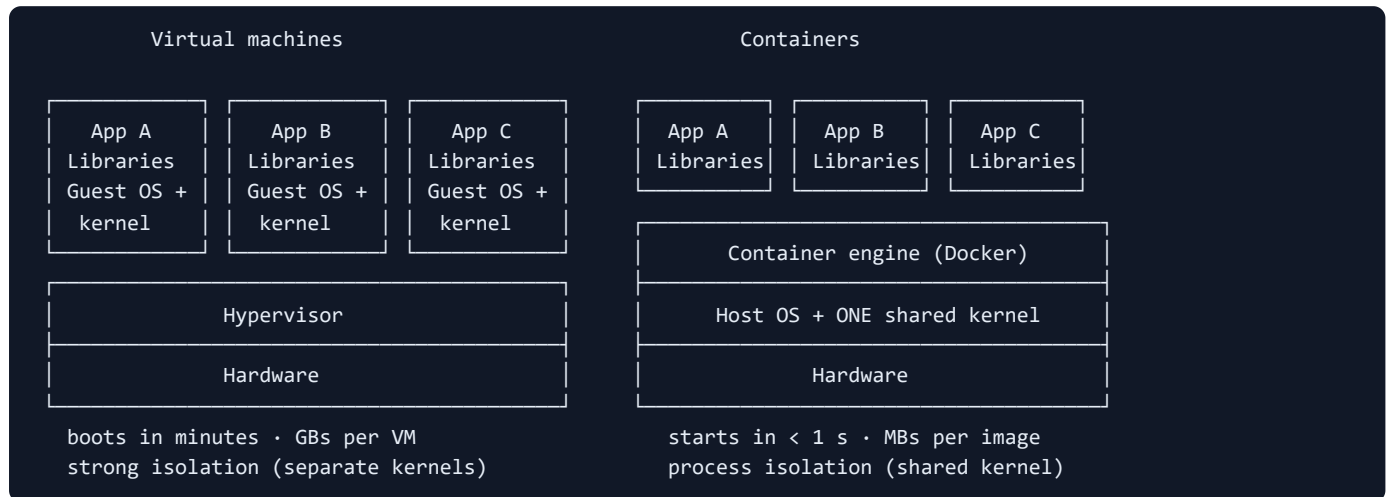
- An application needs code + runtime + libraries + system dependencies + configuration.
- Installing them by hand on servers causes drift and "works on my machine" bugs.
- A container image packages the whole stack once; every environment runs the same image.

Full lesson with every command and its real output: [002-why-containers](#)

Lesson 003 · Docker vs virtual machines

A **virtual machine** emulates a whole computer: a hypervisor runs a complete guest operating system, with its own kernel, for each VM. A **container** is an isolated process on the host's kernel: it brings its own files (libraries, runtime, application) but shares the kernel with every other container. That one difference explains everything else: containers start in under a second, weigh megabytes instead of gigabytes, and are isolated less strongly than VMs.

VISUAL



	Virtual machine	Container
Isolates	a whole computer	a process (namespaces + cgroups)
Kernel	its own, per VM	the host's, shared
Size	gigabytes (full OS)	megabytes (only the files the app needs)
Start time	tens of seconds to minutes	well under a second
Typical use	different OSes, strong tenant isolation	packaging and running applications

COMMANDS

Command	What it does
<code>docker info --format '{{.KernelVersion}}'</code>	print one field of the engine's information (lesson 005)
<code>uname -r</code>	print the release of the running kernel
<code>ps</code>	list the processes this container can see
<code>time COMMAND</code>	measure how long a command takes (a shell feature)

RECAP

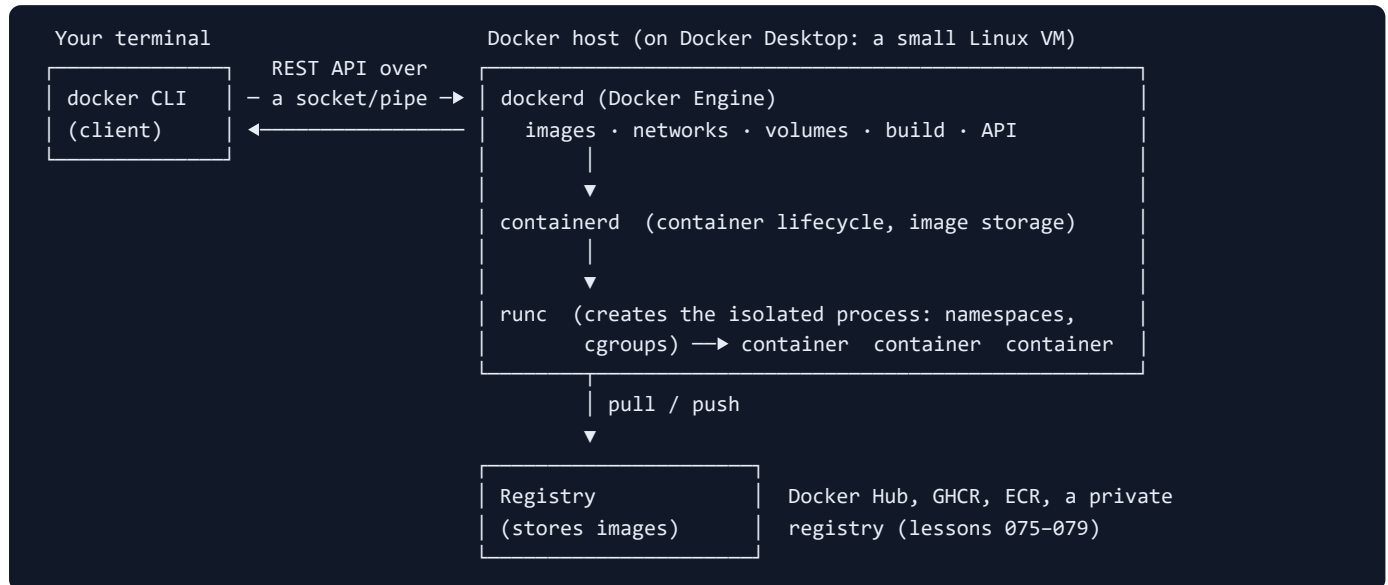
- A VM virtualises hardware and runs a full guest OS with its own kernel.
- A container is an isolated process sharing the host's kernel: small, fast to start, weaker isolation.
- Kernel settings are protected inside containers; only namespaced ones can be set, with `--sysctl`.
- VMs and containers are complementary: containers usually run inside VMs in the cloud.

Full lesson with every command and its real output: [003-docker-vs-vm](#)s

Lesson 004 · Docker architecture

`docker` is only a **client**. Every command you type becomes an HTTP request to the **Docker daemon** (`dockerd`, the engine), which does the real work: it pulls **images** from a **registry**, stores them, and asks **containerd** and **runc** to create and run **containers**. Knowing which part does what tells you where to look when something fails.

VISUAL



Component	Role
Docker client (<code>docker</code>)	turns commands into API requests; holds no containers itself
Docker daemon (<code>dockerd</code>)	the engine: manages images, containers, networks, volumes, builds
containerd	runs and supervises containers for the daemon (also used directly by Kubernetes)
runc	the low-level runtime that creates the isolated process, then exits
Image	a read-only template: files + metadata (lesson 007)
Container	a running (or stopped) instance of an image
Registry	a server that stores and distributes images

COMMANDS

Command	What it does
<code>docker version</code>	versions of the client and of the server components
<code>--format '{{...}}'</code>	print only chosen fields (a Go template; used throughout the course)
<code>docker container create</code>	the daemon creates a container from an image, without starting it
<code>docker container start -a</code>	start it and attach to its output
<code>docker context ls</code>	the engines this client knows about, and which one is active

RECAP

- The `docker` CLI is a client; `dockerd` is the engine that does the work, through containerd and runc.
- Client and daemon talk over a REST API on a socket or named pipe; a context says which daemon to use.
- "Cannot connect to the Docker daemon": daemon not running, wrong `DOCKER_HOST` /context, or no permission.

- Registries store images; the daemon pulls from and pushes to them.

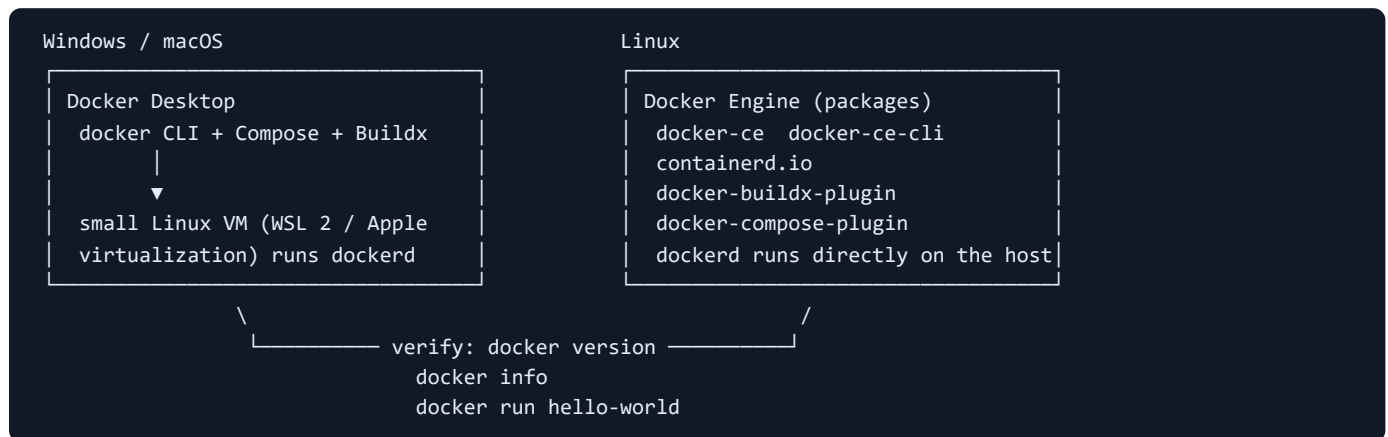
Full lesson with every command and its real output: [004-docker-architecture](#)

Lesson 005 · Installing Docker

How to install Docker on each operating system, and how to **verify** an installation instead of assuming it works:

`docker version` (are the client and the engine both there?), `docker info` (how is the engine configured?) and `docker run hello-world` (can it pull and run a container end to end?).

VISUAL



COMMANDS

Command	What it checks
<code>docker version</code>	client and server versions; a missing <code>Server</code> section means no engine
<code>docker info</code>	engine configuration: storage driver, CPUs, memory, counts, warnings
<code>docker info --format '{{.Field}}'</code>	one field of that report (names are case-sensitive)
<code>docker info --format '{{json .}}'</code>	the whole report as JSON, to discover field names
<code>docker run hello-world</code>	the full path: client → daemon → image → container

RECAP

- Windows and macOS: Docker Desktop (the engine runs in a small Linux VM). Linux: Docker Engine from Docker's repository.
- Verify with `docker version` (client and server), `docker info` (configuration) and `docker run hello-world`.
- `--format` with Go templates extracts single fields; field names are case-sensitive.
- The `docker` group grants root-equivalent access on Linux.

Full lesson with every command and its real output: [005-installing-docker](#)

Module 02 · Containers

Lesson 006 · Your first container

`docker run hello-world` is the smallest complete container run. It looks like one command, but Docker does four things: it looks for the image locally, downloads it from a registry if it is missing, creates a container from it, and starts it. The container prints a message and exits. Understanding these steps is what lets you read every later error.

VISUAL

```
docker run hello-world
|
▼
1. Is the image hello-world:latest on this machine? — no → 2. pull it from the registry (Docker Hub)
| yes
|
▼
3. create a container from the image (new ID, a random name, a writable layer)
|
▼
4. start it: run the image's command (/hello) → prints the message → the process ends
|
▼
the container is now "Exited (0)": stopped, but still there until you remove it
```

COMMANDS

Command / part	Meaning
<code>docker run IMAGE</code>	pull (if missing) + create + start a container from IMAGE
<code>hello-world</code>	short for <code>docker.io/library/hello-world:latest</code> (lesson 077)
<code>--rm</code>	remove the container when it exits
<code>docker ps</code>	running containers only
<code>docker ps -a</code>	all containers, including stopped ones (lesson 008)
<code>--filter ancestor=IMAGE</code>	only containers created from IMAGE

RECAP

- `docker run` = find or pull the image, create a container, start it.
- A container whose process ends is **exited**, not deleted: `docker ps -a` still lists it.
- `--rm` removes the container automatically when it exits.
- `pull access denied ... repository does not exist` usually means a typo in the image name.

Full lesson with every command and its real output: [006-first-container](#)

Lesson 007 · Images vs containers

An **image** is a read-only template: a file system plus metadata (which command to run, which user, which ports). A **container** is an instance of an image: the image's layers, plus a thin **writable layer** of its own, plus a running (or stopped) process. One image can be the template for any number of containers, and changes inside a container never change the image. The analogy: an image is a class, a container is an object.

VISUAL

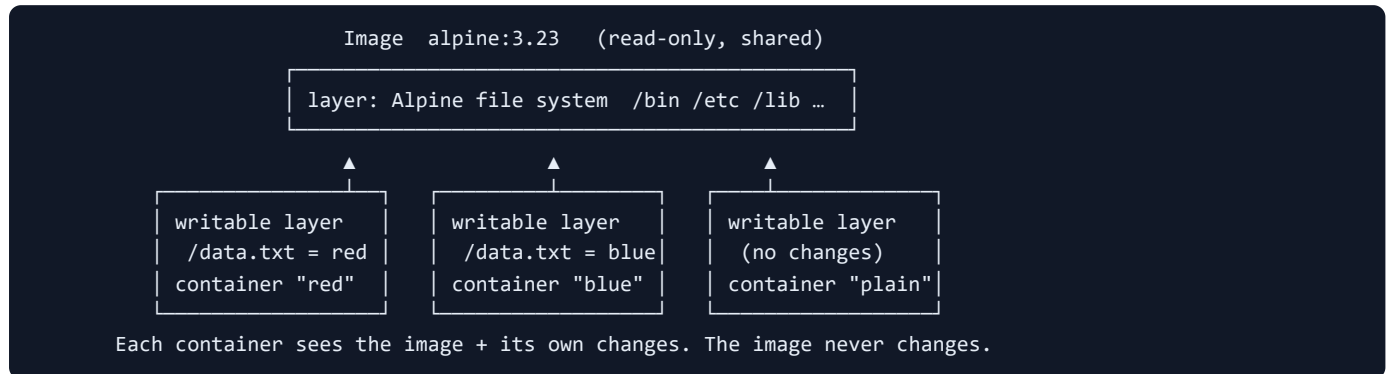


	Image	Container
What it is	template: files + metadata	instance: image + writable layer + process
Changes?	never (read-only)	yes, in its writable layer
Lifetime	until you remove it	created → running → exited → removed
Listed by	<code>docker image ls</code>	<code>docker ps -a</code>

COMMANDS

Command	What it does
<code>docker run --name NAME IMAGE CMD</code>	create and start a named container (lesson 010)
<code>docker container diff NAME</code>	files the container added (A), changed (C) or deleted (D)
<code>docker cp NAME:PATH DEST</code>	copy a file out of a container (running or stopped)
<code>docker image ls</code>	the images on this machine
<code>docker ps -a --filter ancestor=IMAGE</code>	the containers created from IMAGE

RECAP

- Image: read-only template. Container: image + its own writable layer + a process.
- Many containers can run from one image; their changes stay in their own writable layers.
- `docker container diff` shows a container's changes; `docker cp` copies files out.
- An image cannot be removed while containers created from it exist: remove the containers first.

Full lesson with every command and its real output: [007-images-vs-containers](#)

Lesson 008 · Listing containers

`docker ps` is the command you will type most often. By default it lists **running** containers only; `-a` adds the stopped ones. Filters (`--filter`) narrow the list down, `--format` chooses the columns, and `-q` prints only IDs for use in other commands. On a busy machine, these options are the difference between finding your container in a second and scrolling through a wall of text.

VISUAL

`docker ps`

`running containers``web Up 2 minutes`

`docker ps -a`

`running``+ created (never started)``+ exited (stopped)``+ paused, restarting, dead`

`docker ps -q`

`3f2a9c1e0b77``8d1e44b2c9a0`

only IDs: input for other commands

Columns: CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

Narrow: --filter status=exited --filter name=web --filter ancestor=nginx:1.30-alpine

Shape: --format 'table {{.Names}}\t{{.Status}}' -n 3 (last 3) -1 (latest)

COMMANDS

Option	Meaning
<code>-a</code> , <code>--all</code>	include stopped containers
<code>-q</code> , <code>--quiet</code>	print only container IDs
<code>-l</code> , <code>--latest</code> / <code>-n N</code>	the most recently created container / the last N
<code>--filter status=...</code>	<code>created</code> , <code>running</code> , <code>paused</code> , <code>restarting</code> , <code>removing</code> , <code>exited</code> , <code>dead</code>
<code>--filter name=TEXT</code>	names containing TEXT
<code>--filter ancestor=IMAGE</code>	created from IMAGE
<code>--filter exited=CODE</code>	stopped with that exit code
<code>--format 'table {{.Names}}\t{{.Status}}'</code>	choose columns (<code>table</code> keeps the header)
<code>--no-trunc</code>	do not shorten IDs and commands
<code>-s</code> , <code>--size</code>	the size of each container's writable layer

RECAP

- `docker ps` : running containers; `docker ps -a` : all of them.
- `--filter status=exited` (not "stopped"), `name=`, `ancestor=`, `exited=CODE`.
- `--format` picks columns; `-q` gives IDs for scripts.
- An error that lists valid values tells you the fix: read it fully.

Full lesson with every command and its real output: [008-listing-containers](#)

Lesson 009 · Container lifecycle

A container moves through states: **created**, **running**, **paused**, **exited**, and finally **removed**. Each `docker` command moves it from one state to another. The most important transition is **stop**: Docker first asks the main process to shut down cleanly (a `SIGTERM` signal), waits for a grace period, and only then kills it (`SIGKILL`). Applications that handle `SIGTERM` finish their work; those that do not are killed and lose it.

VISUAL



COMMANDS

Command	Transition
<code>docker create IMAGE</code>	→ created (not running)
<code>docker start NAME</code>	created/exited → running (<code>-a</code> to see its output)
<code>docker run -d IMAGE</code>	create + start, in the background; prints the container ID
<code>docker pause / unpause</code>	running ⇌ paused (processes frozen, not stopped)
<code>docker stop [-t SECONDS]</code>	running → exited: <code>SIGTERM</code> , wait (<code>-t</code>), then <code>SIGKILL</code>
<code>docker kill</code>	running → exited: <code>SIGKILL</code> at once (exit code 137)
<code>docker restart</code>	stop + start
<code>docker rm / rm -f</code>	exited → removed / any state → removed (kills it first)

RECAP

- States: created → running ⇌ paused → exited → removed; `run` = create + start.
- `stop` sends `SIGTERM`, waits, then `SIGKILL`; `kill` sends `SIGKILL` at once.
- Exit code 137 means the process was killed, not that it finished.
- A running container cannot be removed without `-f`; restart policies restart failed containers.

Full lesson with every command and its real output: [009-container-lifecycle](#)

Lesson 010 • Container names

Every container has a unique **ID** (64 hexadecimal characters, usually shown shortened to 12) and a unique **name**. Without `--name`, Docker invents one such as `gallant_ritchie`. Choosing names makes commands readable (`docker logs api` instead of `docker logs 3f2a9c1e0b77`), and on a user-defined network other containers can reach a container by its name (lesson 050). Because names are unique, a name can only be used again once the old container is gone.

VISUAL

```
docker run nginx:1.30-alpine
```

ID	3f2a9c1e0b77...	(unique)
name	gallant_ritchie	(random)

```
docker run --name web nginx:1.30-alpine
```

ID	8d1e44b2c9a0...	(unique)
name	web	(yours)

Every command accepts the full ID, any unique prefix of it, or the name:
`docker stop 8d1e44b2c9a0` `docker stop 8d1` `docker stop web`

Names are unique per engine, including stopped containers:
`web (exited) + docker run --name web ...` → Conflict. The container name `"/web"` is already in use

COMMANDS

Command / option	Meaning
<code>--name NAME</code>	give the container a name (letters, digits, <code>_</code> , <code>.</code> , <code>-</code> ; must start with a letter or digit)
<code>docker rename OLD NEW</code>	change a container's name
<code>docker inspect --format '{{.Id}}'</code> <code>NAME</code>	the full ID of a container
<code>docker ps --filter name=TEXT</code>	containers whose name contains TEXT (<code>name=^web\$</code> for an exact match)

RECAP

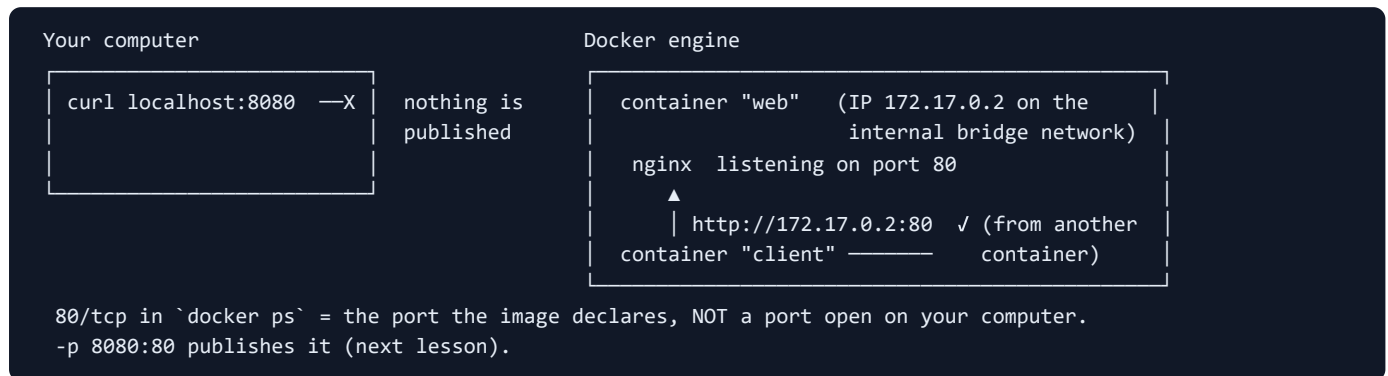
- Every container has a unique ID and a unique name; commands accept either, or an ID prefix.
- `--name` chooses the name; `docker rename` changes it.
- Names stay taken by exited containers: remove the old container or pick another name.

Full lesson with every command and its real output: [010-container-names](#)

Lesson 011 · Running Nginx

So far our containers ran a command and exited. A **server** container keeps running: its main process (here the Nginx web server) waits for requests until it is stopped. We run one in the background with `-d`, check that it serves pages, and discover the first rule of container networking: a server inside a container is **not** reachable from your computer until you publish its port.

VISUAL



COMMANDS

Command / part	Meaning
<code>docker run -d</code>	run in the background and print the container ID
<code>nginx:1.30-alpine</code>	the official Nginx image, version 1.30, on Alpine Linux
<code>80/tcp</code> in <code>PORTS</code>	a port the image declares (<code>EXPOSE</code> , lesson 032), not published
<code>--format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'</code>	the container's IP address on its networks
<code>wget -qO- URL</code>	download URL and print it (BusyBox <code>wget</code> in Alpine)
<code>docker cp FILE NAME:PATH</code>	copy a file into a container

RECAP

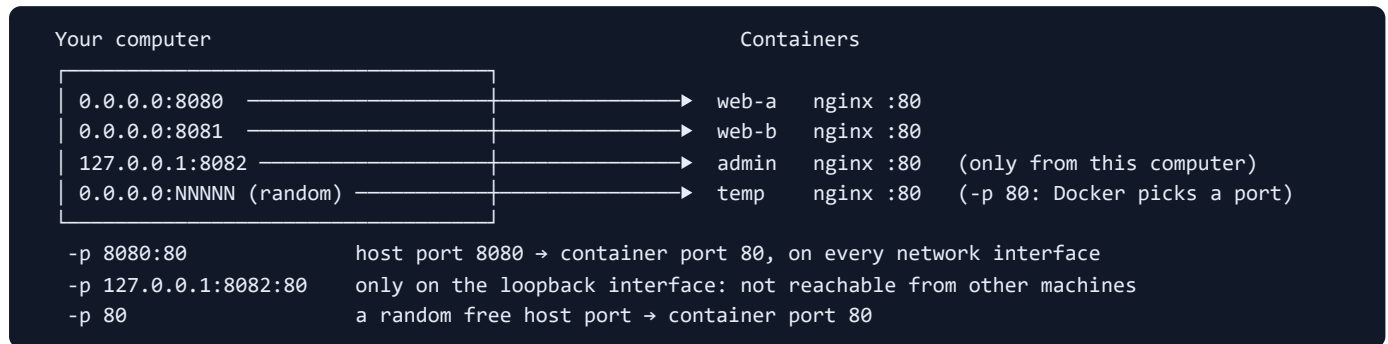
- `docker run -d` runs a server container in the background.
- `80/tcp` in `docker ps` is a declared port, not a published one: the host cannot reach it.
- Other containers can reach it by IP on Docker's network.
- Ports are set at creation: publish with `-p HOST:CONTAINER` (lesson 012).

Full lesson with every command and its real output: [011-running-nginx](#)

Lesson 012 · Port mapping

`-p HOST_PORT:CONTAINER_PORT` publishes a container's port: Docker listens on `HOST_PORT` of your computer and forwards every connection to `CONTAINER_PORT` inside the container. The two numbers are independent, so ten Nginx containers can all listen on port 80 inside while each one gets a different port outside. A host port, however, can belong to only one container at a time.

VISUAL



COMMANDS

Option	Meaning
<code>-p 8080:80</code>	host port 8080 (all interfaces) → container port 80
<code>-p 127.0.0.1:8082:80</code>	only on 127.0.0.1 of the host
<code>-p 80</code>	a random free host port → container port 80
<code>-p 5353:53/udp</code>	UDP instead of TCP
<code>-P</code> (capital)	publish every port the image declares (<code>EXPOSE</code>) on random host ports
<code>docker port NAME [PORT]</code>	show the published ports

Ports are fixed when the container is created: to change them, create a new container.

RECAP

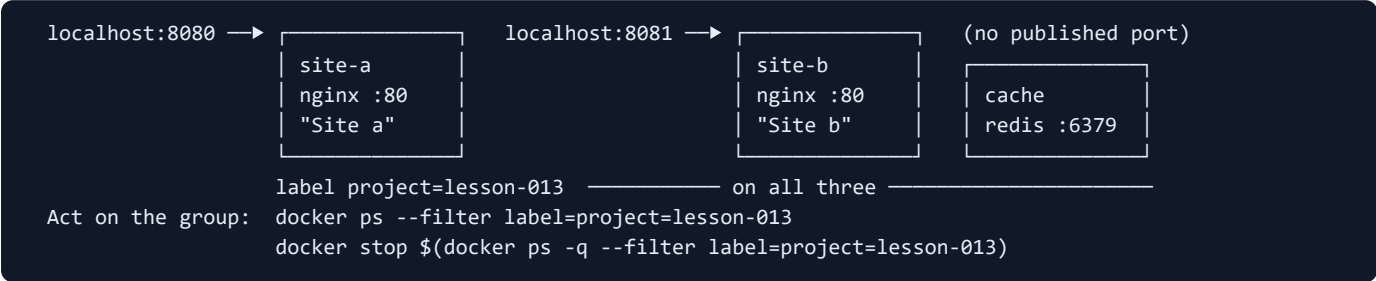
- `-p HOST:CONTAINER` forwards a host port to a container port; the numbers are independent.
- One host port belongs to one container (or program) at a time: `port is already allocated`.
- `-p 127.0.0.1:...` restricts access to the local machine; `-p PORT` picks a random host port.
- Ports are fixed at creation; `docker port` shows them.

Full lesson with every command and its real output: [012-port-mapping](#)

Lesson 013 · Running multiple containers

Real systems are several containers: a website, an API, a cache, a database. Each container is independent: its own process, its own file system, its own published ports. This lesson runs several at once and shows how to manage them as a **group**, with labels and filters, instead of one name at a time. Compose (module 10) automates this; here you learn what it does under the hood.

VISUAL



COMMANDS

Command / part	Meaning
<code>--label KEY=VALUE</code>	attach metadata to a container; filter on it later
<code>--filter label=KEY=VALUE</code>	only containers with that label
<code>\$(docker ps -q --filter ...)</code>	the IDs of the matching containers, as arguments for another command
<code>docker stop A B C</code> / <code>docker start A B C</code>	commands accept several containers
<code>exec nginx -g 'daemon off;'</code>	replace the shell with Nginx running in the foreground

RECAP

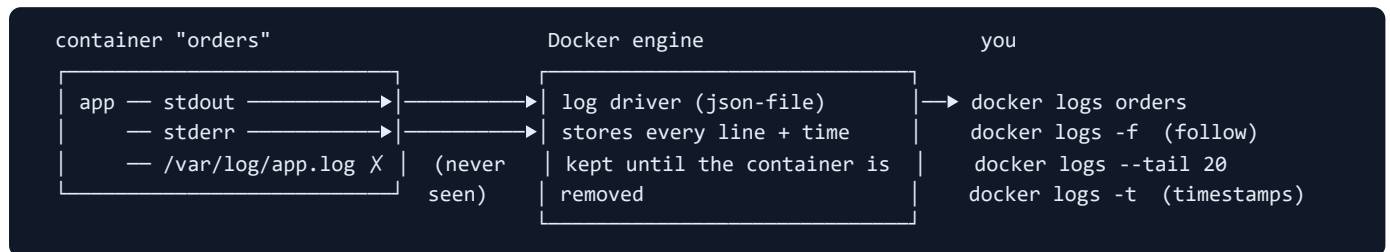
- Each container is independent: its own process, files and published ports.
- Labels group containers; `--filter label=...` with `-q` acts on the whole group.
- `$(docker ps -q ...)` can be empty: guard commands that need at least one ID.
- Running many containers by hand is repetitive; Compose (module 10) describes them in one file.

Full lesson with every command and its real output: [013-multiple-containers](#)

Lesson 014 • Container logs

Docker captures everything a container's main process writes to **standard output** (stdout) and **standard error** (stderr) and keeps it as the container's log. `docker logs` reads it, even after the container has stopped. This only works if the application writes its logs to stdout/stderr: logs written to a file inside the container are invisible to Docker. That is why official images such as Nginx redirect their log files to stdout.

VISUAL



COMMANDS

Command / option	Meaning
<code>docker logs NAME</code>	the whole log (stdout and stderr)
<code>-f</code> , <code>--follow</code>	keep printing new lines (Ctrl+C to stop)
<code>--tail N</code>	only the last N lines
<code>-t</code> , <code>--timestamps</code>	prefix every line with its time
<code>--since 10m</code> / <code>--until ...</code>	only a time range (<code>10m</code> , <code>1h</code> , or a date)
<code>2>&1 > /dev/null</code>	keep only the container's stderr

RECAP

- `docker logs` shows what the main process wrote to stdout and stderr, even after it exited.
- `--tail`, `-t`, `--since` and `-f` narrow and follow the log.
- Logs written to files inside the container are invisible to Docker: log to stdout, or link the file to `/dev/stdout`.
- Logs are deleted with the container.

Full lesson with every command and its real output: [014-container-logs](#)

Lesson 015 · Running commands in containers (exec)

`docker exec` starts an **additional** process inside a **running** container: it shares the container's file system, network and environment. It is the tool for looking inside: read a configuration file, check what the application sees, test a connection from the container's point of view, or open an interactive shell. It does not restart or change the main process, and whatever you change by hand is lost when the container is replaced.

VISUAL

container "web" (running)

PID 1	nginx: master process	← the main process
PID 30	nginx: worker process	
PID 45	cat /etc/nginx/nginx.conf	← docker exec web cat ... (ends, container keeps running)
PID 52	sh	← docker exec -it web sh (interactive, until you type exit)

same files · same network · same environment variables · needs the container to be running

COMMANDS

Command / option	Meaning
<code>docker exec NAME CMD</code>	run CMD in the running container NAME
<code>-it</code>	interactive with a terminal: for shells
<code>-u USER</code>	run as another user (<code>-u root</code> , <code>-u nginx</code>)
<code>-w DIR</code>	working directory for the command
<code>-e KEY=VALUE</code>	an extra environment variable for this command only
<code>sh -c '...'</code>	needed for pipes, <code>></code> , <code>&&</code> inside the container

RECAP

- `docker exec` runs an extra process in a running container: same files, network and environment.
- `-it` for interactive shells; `sh -c '...'` for pipes and redirections inside the container.
- `executable file not found`: the program is not in the image; small images often have only `sh`.
- Manual changes made with `exec` are lost when the container is replaced: use it to inspect, not to fix.

Full lesson with every command and its real output: [015-exec-into-containers](#)

Lesson 016 · Inspecting containers

`docker inspect` prints everything Docker knows about a container as JSON: its configuration (image, command, environment, ports, mounts), its state (running or not, exit code, start and finish times, whether it was killed for memory) and its network settings. It is the most precise answer to "what is this container, and what happened to it?". `--format` extracts exactly the field you need.

VISUAL

```
docker inspect web
[
  {
    "Id": "8d1e44b2c9a0...",
    "Name": "/web",
    "State": { "Status": "running", "ExitCode": 0, "OOMKilled": false, "StartedAt": "...", ... },
    "Config": { "Image": "nginx:1.30-alpine", "Cmd": [ "nginx", "-g", "daemon off;" ], "Env": [ ... ] },
    "HostConfig": { "PortBindings": { "80/tcp": [ { "HostPort": "8080" } ] }, "RestartPolicy": { ... } },
    "Mounts": [ ... ],
    "NetworkSettings": { "Networks": { "bridge": { "IPAddress": "172.17.0.2", ... } } }
  }
]
--format '{{.State.Status}}' → running          (a path into the JSON, case-sensitive)
--format '{{json .Config.Env}}' → [ "PATH=...", "NGINX_VERSION=1.30..." ]
```

COMMANDS

Command / template	Meaning
<code>docker inspect NAME</code>	everything about a container (also works for images, networks, volumes)
<code>--format '{{.A.B}}'</code>	one field; names are case-sensitive, as in the JSON
<code>{{json .X}}</code>	print a whole object or list as JSON
<code>{{range .List}}{{println .}}{{end}}</code>	one list entry per line
<code>{{range \$k, \$v := .Map}}...{{end}}</code>	loop over a map (networks, ports)
<code>.State.ExitCode</code> , <code>.State.Error</code> , <code>.State.OOMKilled</code>	why a container stopped

RECAP

- `docker inspect` is the full truth about a container: configuration, state, networking.
- `--format` with Go templates extracts fields; `json`, `range` and `index` handle objects, lists and maps.
- For a stopped container: exit code and OOM flag (`inspect`), then the message (`logs`), then the configuration.
- Exit code below 128: the application exited; 128 + n: killed by signal n.

Full lesson with every command and its real output: [016-inspecting-containers](#)

Module 03 · Images

Lesson 017 · Image layers

An image is not one big file. It is a stack of read-only **layers**: each step that changes files when the image is built (a `RUN`, `COPY` or `ADD` in a Dockerfile) adds one layer on top of the previous ones. A container adds one thin **writable layer** on top. Layers are stored once and shared between images and containers, and they explain why an image is as big as it is.

VISUAL

```
docker image history nginx:1.30-alpine      (newest layer on top)
```

container's writable layer (per container)	← changes made while it runs; removed with the container
RUN ... install nginx modules 51.9 MB	] read-only image layers, shared by every container of the image
COPY docker-entrypoint scripts ~60 kB	
RUN ... install nginx 5.6 MB	
ADD alpine-minirootfs ... 9.1 MB	
← the base image	

ENV, CMD, EXPOSE, LABEL ... only change metadata: 0 B, no new file layer

A file deleted in a later layer is only hidden: its bytes stay in the layer that created it.

COMMANDS

Command	What it does
<code>docker image history IMAGE</code>	the build steps of an image, newest first, with the size each one added
<code>--no-trunc</code>	show the full commands instead of cutting them at ...
<code>docker image inspect --format '{{.RootFS.Layers}}' IMAGE</code>	the digests of the file layers, base first
<code>{{len ...}}</code>	Go template: the number of entries in a list

RECAP

- An image is a stack of read-only layers; `RUN`, `COPY` and `ADD` create layers, other instructions only metadata.
- A container adds a thin writable layer on top.
- Layers are content-addressed (digests) and shared between images: stored, pushed and pulled once.
- Deleting a file in a later layer hides it but does not shrink the image: create and delete in the same step.

Full lesson with every command and its real output: [017-image-layers](#)

Lesson 018 · Pulling images

`docker image pull` downloads an image from a **registry**. Every image name is really a full address, `REGISTRY/NAMESPACE/REPOSITORY:TAG`, and Docker fills in what you leave out: `alpine:3.23` means `docker.io/library/alpine:3.23`. A pull downloads only the layers you do not have yet. A **tag** can move to a new image; a **digest** (`@sha256:...`) always means exactly the same bytes.

VISUAL

```
docker image pull alpine:3.23
      | expands to
      v
docker.io / library / alpine : 3.23      mirror.gcr.io / library / busybox : 1.37
-----
registry      namespace  repo      tag      another registry, same naming rules

client → dockerd → registry: "manifest for alpine:3.23?"
                        ← manifest: digest sha256:85fe..., layers L1 L2 ...
                        → download only the layers not already on disk
result:  tag  alpine:3.23 → digest sha256:85fe... (a tag can be moved to a newer image; a digest cannot)
```

COMMANDS

Command / part	Meaning
<code>docker image pull NAME:TAG</code>	download an image (<code>docker pull</code> is the short form)
<code>docker.io/library/</code>	the default registry and namespace of official images, added when you omit them
<code>-q</code>	quiet: print only the image name
<code>--platform linux/arm64</code>	pull the variant for another CPU architecture (lesson 104)
<code>docker image ls --digests</code>	show the digest each local tag refers to
<code>NAME@sha256:...</code>	an image by digest: immutable, always the same content

RECAP

- A full image name is `REGISTRY/NAMESPACE/REPOSITORY:TAG`; `docker.io/library/` is the default.
- A pull downloads only missing layers; a tag can move, a digest never does.
- `pull access denied ... does not exist or may require 'docker login'`: wrong name, private repository, or no login.
- `429 Too Many Requests`: the registry's rate limit, not a problem with the image.

Full lesson with every command and its real output: [018-pulling-images](#)

Lesson 019 · Listing and removing images

Images pile up: every pull and every build leaves one behind. This lesson lists them (`docker image ls` , with filters and formats), removes them safely (`docker image rm`), and cleans up **dangling** images: images that no longer have a name, typically left behind by builds.

VISUAL

```
docker image ls

name (tag)      image ID
cafe:1.1        ────┐
cafe:latest     ────┴───> 4e91ae7992f1
conflict
cafe:1.0        ────┐
"Deleted"       ────┴───> 60c13e724dd4
<none>:<none>    ────┐
                  ────┴───> 9707e1f15475 ← dangling:
                                no name left
```

what removing does

```
docker image rm cafe:1.1
├─ another tag points to the same ID? → "Untagged" only
├─ a container (even stopped) uses it? → refused:
└─ otherwise                               → "Untagged" +

docker image prune      removes dangling images only
docker image prune -a   removes EVERY image no container uses
```

COMMANDS

Command	What it does
<code>docker image ls</code> (<code>docker images</code>)	list images; <code>-a</code> also shows intermediate images
<code>--filter reference=PATTERN</code>	only names matching a pattern (<code>*</code> wildcard)
<code>--filter dangling=true</code>	only images without a name
<code>--filter before=IMAGE / since=IMAGE</code>	images created before / after another one
<code>--format 'table {{.Repository}}\t{{.Tag}}'</code>	choose the columns
<code>-q</code>	IDs only
<code>docker image rm NAME:TAG</code> (<code>docker rmi</code>)	remove a name, and the image when no other name refers to it
<code>docker image prune [-f]</code>	remove dangling images (<code>-f</code> : without asking)
<code>docker image prune -a</code>	remove every image not used by a container: all your cached images

RECAP

- `docker image ls` with `--filter` and `--format` finds exactly the images you need.
- `docker image rm` removes a name; the image is deleted when nothing else refers to it.
- A container, even a stopped one, blocks the removal of its image: remove the container first.
- Dangling images have no name; `docker image prune` removes them, `prune -a` removes everything unused.

Full lesson with every command and its real output: [019-listing-and-removing-images](#)

Lesson 020 · Image tags

A **tag** is a movable name that points to an image ID, like a Git branch points to a commit. One image can have many tags, and `docker image tag` adds one without copying anything. Teams use this for versioning: the exact version `1.4.2` never moves, while the shorter `1.4` and `1` move to every new patch or minor release.

VISUAL

after releasing 1.4.2

cafe-menu:1.4.2 → image A
cafe-menu:1.4 → image A
cafe-menu:1 → image A

after releasing 1.4.3

cafe-menu:1.4.2 → image A (exact version: never moves)
cafe-menu:1.4.3 → image B
cafe-menu:1.4 → image B (moved)
cafe-menu:1 → image B (moved)

tag format: [REGISTRY[:PORT]]/[NAMESPACE/]REPOSITORY[:TAG]
repository: lowercase letters, digits, . _ - and / tag: up to 128 of A-Z a-z 0-9 _ . -

COMMANDS

Command	What it does
<code>docker image tag SOURCE TARGET (docker tag)</code>	add the name TARGET to the image SOURCE; move it if it exists
<code>docker build -t NAME:TAG</code>	build and tag in one step; <code>-t</code> can be repeated
<code>docker image rm NAME:TAG</code>	remove one name (lesson 019)
no <code>:TAG</code>	means <code>:latest</code> (lesson 021)

RECAP

- A tag is a movable name for an image ID; `docker image tag` adds or moves one and copies nothing.
- Exact versions (`1.4.2`) stay put; short tags (`1.4` , `1`) move to new releases.
- Repository names are lowercase; tags allow `A-Z a-z 0-9 _ . -` , up to 128 characters.
- A registry address in the name (`localhost:5000/cafe/menu`) decides where `docker push` sends it.

Full lesson with every command and its real output: [020-image-tags](#)

Lesson 021 · The danger of the latest tag

`latest` is not magic and not "the newest version". It is an ordinary tag that Docker uses **when you do not write one**: `docker run nginx` means `nginx:latest`, `docker build -t cafe-menu .` creates `cafe-menu:latest`. Whatever was tagged `latest` most recently (or pushed most recently) wins, so the same command can run different software on different days and machines. Production uses explicit versions.

VISUAL

```
Monday    docker build -t cafe-menu .      cafe-menu:latest → v1      server A runs v1
Tuesday   docker build -t cafe-menu .      cafe-menu:latest → v2      server B pulls: v2
Wednesday hotfix built from an old branch  cafe-menu:latest → v1-fix  server C pulls: v1-fix

same command "docker run cafe-menu" → three different programs, and no record of which one

instead:  cafe-menu:1.4.3  (exact, never moves)    or    cafe-menu@sha256:... (content, can never change)
```

COMMANDS

Command	Means
<code>docker run nginx</code>	<code>docker run docker.io/library/nginx:latest</code>
<code>docker build -t cafe-menu .</code>	<code>docker build -t cafe-menu:latest .</code>
FROM <code>python</code> in a Dockerfile	FROM <code>python:latest</code> : a different base every few months
<code>docker container inspect --format '{{.Image}}' NAME</code>	the image ID a container really runs

RECAP

- `latest` is the default tag when none is given, not "the newest".
- It moves with every untagged build or push, so it does not identify what runs.
- Image IDs and digests identify content; exact version tags identify releases.
- Pin base images and deployments to explicit versions.

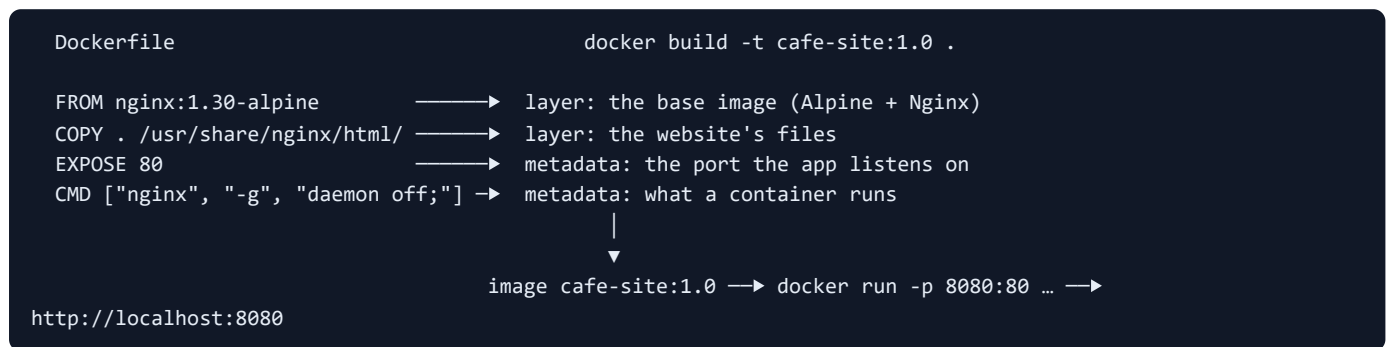
Full lesson with every command and its real output: [021-latest-tag-danger](#)

Module 04 · Dockerfiles

Lesson 022 · Your first Dockerfile

A **Dockerfile** is a text file with the recipe for an image: start from a base image, add files, run commands, and say what the container runs. `docker build` follows the recipe step by step and produces an image. Because the recipe is a file, it lives in Git next to the code: every image can be rebuilt, reviewed and changed like code.

VISUAL



Instruction	Purpose	In depth
FROM	the base image every Dockerfile starts from	lesson 023
WORKDIR	the working directory for the following instructions	lesson 024
COPY / ADD	add files from the build context	lesson 025
RUN	run a command at build time; its result becomes a layer	lesson 026
CMD / ENTRYPOINT	what the container runs	lessons 027–029
ENV / ARG	runtime / build-time variables	lessons 030, 031
EXPOSE / USER	document the port / choose the user	lessons 032, 033

COMMANDS

Command / part	Meaning
<code>docker build -t NAME:TAG .</code>	build the Dockerfile in <code>.</code> , using <code>.</code> as the build context
<code>-f path/Dockerfile</code>	use a Dockerfile with another name or location
<code># ...</code>	a comment line
<code>INSTRUCTION arguments</code>	one instruction per line; instructions are written in capitals by convention
<code>docker run -d -p 8080:80</code>	run in the background, port 8080 on your computer → port 80 in the container (lesson 012)

RECAP

- A Dockerfile is the recipe of an image: `FROM`, then instructions that add files, run commands and set metadata.
- `docker build -t NAME:TAG .` builds it; `.` is the build context.
- Parse errors name the line and the instruction; nothing is built until the file parses.
- An image is fixed once built: a change means a new build and a new tag.

Full lesson with every command and its real output: [022-first-dockerfile](#)

Lesson 023 · FROM: choosing a base image

Every Dockerfile starts with `FROM`: the **base image** whose files become the bottom layers of your image. The choice decides the image's size, which tools exist (a shell? `bash`? a package manager?), the C library (`glibc` or `musl`), and how many packages, and therefore vulnerabilities, you ship. Choose deliberately and always with an explicit tag.

VISUAL

Base image families (each with a tag, never just the name)

<code>ubuntu:24.04</code> / <code>debian:13-slim</code>	full or slim Linux distribution · <code>glibc</code> · <code>apt</code> · <code>bash</code>	tens of MB to ~100 MB
<code>alpine:3.23</code>	minimal distribution · <code>musl</code> · <code>apk</code> · <code>busybox sh</code> (no <code>bash</code>)	under 10 MB
<code>python:3.14-slim</code> , <code>node:24-alpine</code>	a language runtime on top of one of the above	what most apps use
<code>gcr.io/distroless/...</code>	only the runtime and its libraries: no shell, no package manager	
<code>scratch</code>	empty: for a single static binary (lesson 089)	

smaller base → fewer packages → smaller attack surface, faster pulls → but fewer tools for you

COMMANDS

Instruction	Meaning
<code>FROM IMAGE:TAG</code>	start from this image; always with a tag (lesson 021)
<code>FROM IMAGE@sha256:...</code>	start from an exact image by digest (lesson 018)
<code>FROM IMAGE AS NAME</code>	name the stage, for multi-stage builds (lesson 087)
<code>FROM scratch</code>	start from an empty file system
<code>ARG V=...</code> + <code>FROM image:\${V}</code>	choose the tag with a build argument (lesson 031)

RECAP

- `FROM` chooses the base image: size, tools, C library and vulnerabilities come with it.
- Alpine: tiny, `musl`, `apk`, BusyBox `sh`; Debian/Ubuntu: `glibc`, `apt-get`, `bash`; distroless: no shell.
- Exit code 127 / `not found`: the base image does not contain that program.
- Always pin the tag (or digest); never rely on `latest`.

Full lesson with every command and its real output: [023-from](#)

Lesson 024 · WORKDIR

`WORKDIR /app` sets the working directory for every instruction after it (`RUN` , `COPY` , `CMD` , `ENTRYPOINT`) and for the container that runs the image. It creates the directory if it does not exist. Without it, everything happens in `/` , and a `cd` in a `RUN` step is forgotten as soon as that step ends: each `RUN` starts a fresh shell.

VISUAL

```
FROM alpine:3.23
WORKDIR /app
COPY menu.csv .
RUN ls
WORKDIR data
CMD ["cat", "../menu.csv"]
```

working directory: `/`

—————→ `/app` (created if missing)

copies to `/app/menu.csv`

runs in `/app`

relative: now `/app/data`

the container starts in `/app/data`

`RUN cd /app` ❌ only changes the directory of that one `RUN` step's shell; the next step starts in `/` again

COMMANDS

Instruction / option	Meaning
<code>WORKDIR /path</code>	set the working directory; created if missing
<code>WORKDIR relative</code>	relative to the previous <code>WORKDIR</code>
<code>COPY file .</code>	<code>.</code> is the current <code>WORKDIR</code>
<code>docker run -w /path</code>	override the working directory for one container
<code>docker build --progress=plain</code>	show every step's output in full

RECAP

- `WORKDIR` sets the directory for later instructions and for the container, creating it if needed.
- A relative `WORKDIR` builds on the previous one.
- `RUN cd ...` only affects that one step; use `WORKDIR` instead.
- Check an image's working directory with `docker image inspect --format '{{.Config.WorkingDir}}'`.

Full lesson with every command and its real output: [024-workdir](#)

Lesson 025 · COPY and ADD

`COPY` puts files from the **build context** (the folder you pass to `docker build`) into the image. `ADD` does the same and two extra things: it unpacks local `.tar` archives, and it can download URLs. Because those extras can surprise you, the rule is simple: use `COPY`, and `ADD` only when you want one of its extras. Neither can reach files outside the build context.

VISUAL

```
docker build -t menu .           build context = this folder, sent to the builder

┌ menu.csv                      ─ COPY menu.csv /app/           ─> /app/menu.csv
├ public/index.html            ─ COPY public/ /app/public/      ─> /app/public/index.html (the folder's
└ public/styles.css            ─ ADD assets.tar /app/assets/    ─> /app/assets/... (unpacked!)
  assets.tar                   COPY assets.tar /app/           ─> /app/assets.tar (as is)

../shared/config.json X outside the context: COPY cannot see it
```

COMMANDS

Instruction	Meaning
<code>COPY SRC... DEST</code>	copy files or folders from the build context
<code>COPY --chown=USER:GROUP</code>	set the owner of the copied files
<code>COPY --chmod=0755</code>	set the permissions of the copied files
<code>ADD archive.tar DEST/</code>	copy and unpack a local tar archive (also <code>.tar.gz</code> , <code>.tar.bz2</code> , <code>.tar.xz</code>)
<code>ADD --checksum=sha256:... URL DEST</code>	download a file and verify its checksum
<code>COPY --from=STAGE</code>	copy from another build stage (lesson 087)

RECAP

- `COPY` copies from the build context; a folder source copies its contents.
- `--chown` and `--chmod` set ownership and permissions while copying.
- `ADD` also unpacks local archives and downloads URLs: use it only for that.
- Nothing outside the build context can be copied: choose the context (`.`) and the Dockerfile (`-f`) accordingly.

Full lesson with every command and its real output: [025-copy-and-add](#)

Lesson 026 • RUN

RUN executes a command **while the image is built**, in a temporary container of the image so far, and saves the changed files as a new layer: installing packages, creating users, compiling code. If the command fails (exits with a status other than 0), the build stops. How you write **RUN** steps decides how big, how reproducible and how debuggable the image is.

VISUAL

```
RUN apk add --no-cache curl
|
▼
temporary container from the image so far → /bin/sh -c "apk add --no-cache curl"
|
| exit status 0 → changed files saved as a new layer, build continues
| exit status ≠ 0 → build stops: "did not complete successfully: exit code: N"
```

shell form: `RUN apk add --no-cache curl` runs through `/bin/sh -c` (variables, `&&`, pipes work)
exec form: `RUN ["apk", "add", "--no-cache", "curl"]` runs the program directly, no shell

one step, cleaned up in the same step:
`RUN apk add --no-cache curl \`
`&& curl --version`

COMMANDS

Form / option	Meaning
<code>RUN command args</code>	shell form: <code>/bin/sh -c "command args"</code> , so <code>&&</code> , <code>\ </code> , <code>\$VAR</code> work
<code>RUN ["prog", "arg"]</code>	exec form: runs <code>prog</code> directly; no shell, no variable expansion
<code>&&</code>	run the next command only if the previous one succeeded
<code>\</code> at the end of a line	continue the instruction on the next line
<code>set -eux</code>	in a long script: stop on errors (<code>e</code>), on unset variables (<code>u</code>), print each command (<code>x</code>)
<code>docker build --progress=plain</code>	show the full output of each step

RECAP

- **RUN** executes at build time; each **RUN** adds a layer; a non-zero exit status stops the build.
- Shell form uses `/bin/sh -c`; exec form runs the program directly.
- Chain with `&&` and clean caches in the same step; use `--no-cache` with `apk`.
- The real error is in the step's own output (`--progress=plain`); use `set -o pipefail` with pipes.

Full lesson with every command and its real output: [026-run](#)

Lesson 027 · CMD

`CMD` sets the **default command** of a container: what runs when you `docker run IMAGE` without a command. It does nothing at build time. Anything you write after the image name in `docker run` replaces it. A Dockerfile has one effective `CMD` (the last one), and it should be written in **exec form**, a JSON array with double quotes.

VISUAL

```
Dockerfile: CMD ["cat", "/app/menu.csv"]           stored in the image as its default command

docker run menu           → cat /app/menu.csv           (the default)
docker run menu ls /app   → ls /app                     (replaced: everything after the image name)
docker run menu wc -l /app/menu.csv → wc -l /app/menu.csv

exec form  CMD ["echo", "hi"] → runs echo directly: it is the container's main process (PID 1)
shell form CMD echo hi       → runs /bin/sh -c "echo hi": $VARIABLES are expanded by the shell
```

COMMANDS

Form	Meaning
<code>CMD ["prog", "arg"]</code>	exec form (preferred): JSON array, double quotes
<code>CMD prog arg</code>	shell form: <code>/bin/sh -c "prog arg"</code>
<code>CMD ["arg1", "arg2"]</code> after an <code>ENTRYPOINT</code>	default arguments for the entrypoint (lesson 029)
<code>docker run IMAGE COMMAND...</code>	replace <code>CMD</code> for this container
<code>docker image inspect --format '{{json .Config.Cmd}}'</code>	show an image's default command

RECAP

- `CMD` is the default command of the container; it does nothing at build time.
- Arguments after the image name in `docker run` replace it entirely.
- Only the last `CMD` counts; it also replaces the base image's `CMD`.
- Use the exec form with double quotes; single quotes silently turn it into shell form.

Full lesson with every command and its real output: [027-cmd](#)

Lesson 028 · ENTRYPOINT

`ENTRYPOINT` sets the program a container **always** runs. Unlike `CMD`, it is not replaced by the arguments of `docker run`: they are **appended** to it. That turns an image into a command-line tool (`docker run price espresso` works like `price espresso`). To run something else, you must say so explicitly with `--entrypoint`.

VISUAL

Dockerfile: `ENTRYPOINT ["price"]`

`docker run price espresso`
`docker run price "flat white"`
`docker run price sh`
`docker run --entrypoint sh price`

→ `price espresso`
→ `price "flat white"`
→ `price sh`
→ `sh`

arguments are appended

X "sh" is just another argument
the entrypoint replaced, for this container only

	<code>CMD</code>	<code>ENTRYPOINT</code>
run args	replace it	are appended to it
override	<code>docker run IMAGE other-command</code>	<code>docker run --entrypoint other IMAGE</code>
use for	a default that is often changed	the image's fixed main program

COMMANDS

Instruction / option	Meaning
<code>ENTRYPOINT ["prog", "arg"]</code>	exec form (preferred): <code>docker run</code> arguments are appended
<code>ENTRYPOINT prog arg</code>	shell form: runs through <code>/bin/sh -c</code> , and ignores both <code>CMD</code> and run arguments
<code>docker run --entrypoint PROG IMAGE ARGS...</code>	replace the entrypoint for one container; ARGS go to PROG
<code>docker image inspect --format '{{json .Config.Entrypoint}}'</code>	show an image's entrypoint

RECAP

- `ENTRYPOINT` is the program a container always runs; `docker run` arguments are appended to it.
- `--entrypoint` replaces it for one container; arguments after the image name go to the new program.
- Use the exec form; the shell form ignores all arguments.
- The entrypoint's exit status is the container's exit status.

Full lesson with every command and its real output: [028-entrypoint](#)

Lesson 029 · CMD and ENTRYPOINT together

Used together, `ENTRYPOINT` is the fixed program and `CMD` its **default arguments**. `docker run IMAGE` runs `entrypoint + CMD`; `docker run IMAGE args` runs `entrypoint + your args`. This is the most common pattern for tool images and for service images with an entrypoint script. It only works when both are in exec form.

VISUAL

```
ENTRYPOINT ["price"]
CMD ["espresso"]

docker run IMAGE           → price espresso      ENTRYPOINT + CMD
docker run IMAGE cappuccino → price cappuccino  ENTRYPOINT + run args (CMD replaced)
docker run --entrypoint wc IMAGE -l /app/menu.csv → wc -l /app/menu.csv
```

	no ENTRYPOINT	ENTRYPOINT ["e"]	ENTRYPOINT e (shell form)
no CMD	(base image's CMD)	e	/bin/sh -c e
CMD ["c", "x"]	c x	e c x	/bin/sh -c e (CMD lost)
run args a b	a b	e a b	/bin/sh -c e (args lost)

COMMANDS

Combination	Result
ENTRYPOINT ["e"] + CMD ["c"]	e c ; run arguments replace c
ENTRYPOINT ["e"], no CMD	e ; run arguments are appended
ENTRYPOINT e (shell form)	/bin/sh -c e : CMD and run arguments are ignored
docker run --entrypoint X IMAGE	replaces the entrypoint and clears the image's CMD

RECAP

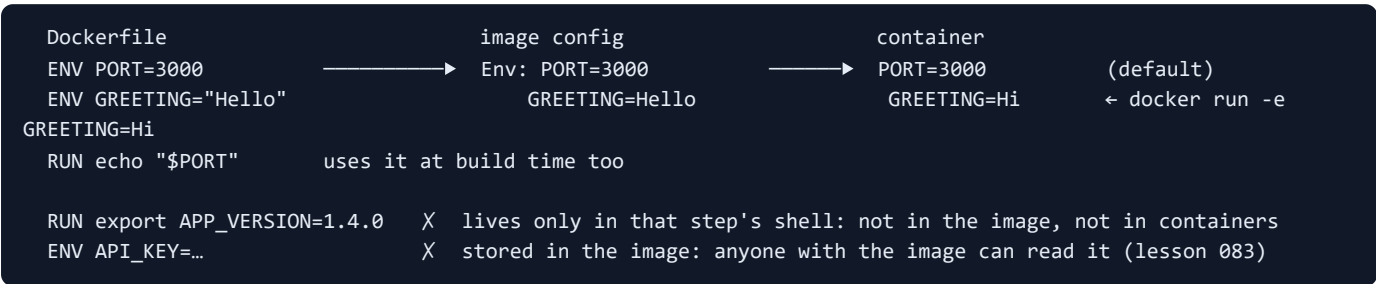
- With both, `ENTRYPOINT` is the program and `CMD` its default arguments.
- Run arguments replace `CMD`, never `ENTRYPOINT`; `--entrypoint` replaces the entrypoint and clears `CMD`.
- Both must be in exec form: the shell form of `ENTRYPOINT` drops every argument.
- Entrypoint scripts prepare the container and end with `exec "$@"`.

Full lesson with every command and its real output: [029-cmd-and-entrypoint](#)

Lesson 030 · ENV

`ENV NAME=value` sets an environment variable in the image: it is available to the following build steps **and** to every container of the image, as a default that `docker run -e NAME=other` can override. Applications read their configuration from environment variables (ports, URLs, feature flags), so `ENV` is how an image ships sensible defaults. Everything set with `ENV` is visible in the image: never put secrets there.

VISUAL



COMMANDS

Instruction / option	Meaning
<code>ENV NAME=value</code>	set a variable in the image; several in one instruction with <code>\</code> continuation
<code>ENV NAME="two words"</code>	quote values with spaces
<code>docker run -e NAME=value</code>	override (or add) a variable for one container
<code>docker image inspect --format '{{json .Config.Env}}'</code>	the variables an image sets
<code>docker exec CONTAINER env</code>	the variables a running container really has

RECAP

- `ENV` sets variables for later build steps and for every container, as defaults.
- `docker run -e` overrides them per container, without a rebuild.
- `RUN export` does not persist: use `ENV`.
- `ENV` values are stored in the image and visible to anyone: no secrets.

Full lesson with every command and its real output: [030-env](#)

Lesson 031 · ARG

`ARG NAME=default` declares a **build-time** variable: `docker build --build-arg NAME=value` sets it, the following build steps can use it, and it is gone when the container runs. Use it for things that change *how* an image is built: a version number, a base image tag, a feature switch. Its values are recorded in the image's history, so `ARG` is **never** for passwords or tokens.

VISUAL

```
docker build --build-arg VERSION=1.4.0 .
```

```
ARG ALPINE_VERSION=3.23
FROM alpine:${ALPINE_VERSION}
ARG VERSION=dev
RUN echo "$VERSION" > /version
CMD ["cat", "/version"]
```

← before FROM: only usable in FROM lines

← after FROM: usable by the steps below (default "dev")

build time: VERSION=1.4.0

run time: no VERSION variable (only the file written at build time)

	ARG	ENV
set by	--build-arg (build)	ENV in the Dockerfile, -e at run time
exists	during the build only	in the image and every container
history	values visible	values visible → neither is for secrets

COMMANDS

Instruction / option	Meaning
ARG NAME / ARG NAME=default	declare a build argument (with a default)
docker build --build-arg NAME=value	set it for this build
ARG before FROM	usable only in FROM lines, e.g. to choose the base image version
ARG NAME again after FROM	makes a pre- FROM argument usable in the build steps
ENV NAME=\$NAME	turn a build argument into a run-time variable (lesson 030)

RECAP

- `ARG` is a build-time variable set with `--build-arg` ; it does not exist in containers.
- An `ARG` before `FROM` is only for `FROM` ; declare it again after `FROM` to use it in the steps.
- To keep a value at run time, copy it into `ENV` (or a file, or a label).
- Values used by `RUN` steps are recorded in the history: use build secrets for anything sensitive.

Full lesson with every command and its real output: [031-arg](#)

Lesson 032 · EXPOSE

`EXPOSE 3000` **documents** the port the application in the image listens on. It does not open, publish or forward anything: a container is reachable from your computer only through `-p HOST:CONTAINER` (lesson 012). `EXPOSE` is used by people reading the image, by tools, and by `docker run -P`, which publishes every exposed port to a random free port. It must match the port the application really listens on.

VISUAL

Dockerfile: `EXPOSE 3000`

image config: `ExposedPorts {"3000/tcp"}` (metadata only)

`docker run IMAGE`

→ app listens on 3000 inside; nothing reachable from your computer

`docker run -p 8080:3000 IMAGE`

→ localhost:8080 → container:3000 (EXPOSE not needed for this)

`docker run -P IMAGE`

→ localhost:<random> → container:3000 (uses the EXPOSE list)

`EXPOSE 80, app listens on 3000`

→ -P publishes port 80, where nothing listens → no response

COMMANDS

Instruction / option	Meaning
<code>EXPOSE 3000</code>	document a TCP port (<code>EXPOSE 3000/tcp</code>); <code>EXPOSE 5353/udp</code> for UDP
<code>docker run -P</code>	publish every exposed port to a random free host port
<code>docker run -p 8080:3000</code>	publish container port 3000 on host port 8080; works with or without <code>EXPOSE</code>
<code>docker port CONTAINER [PORT]</code>	show which host ports a container's ports are published on

RECAP

- `EXPOSE` documents the application's port; it publishes nothing.
- `-p HOST:CONTAINER` publishes a port, with or without `EXPOSE` ; `-P` publishes all exposed ports to random ports.
- `docker port` shows the mapping; a wrong `EXPOSE` makes `-P` publish the wrong port.
- Keep `EXPOSE` consistent with the application's configuration.

Full lesson with every command and its real output: [032-expose](#)

Lesson 033 · USER

Unless the image says otherwise, a container's process runs as **root** (user ID 0). `USER appuser` switches the following `RUN` steps and the container to a normal user. If the application is compromised, the attacker then has the rights of a normal user inside the container, not root's (lesson 081 goes deeper). The price: that user can only write where you allow it to, so files and folders need the right owner.

VISUAL

```
FROM alpine:3.23
RUN adduser -S -u 10001 appuser      create the user (as root)
WORKDIR /app
COPY take-order.sh .               owned by root: appuser can read and run it, not change it
RUN mkdir data && chown appuser data the one folder appuser may write to
USER appuser                       — everything below, and the container, runs as uid 10001
ENTRYPOINT ["/take-order.sh"]

/app      root:root    r-x for appuser    code: read-only for the application
/app/data appuser      rwx for appuser    data: writable
```

COMMANDS

Instruction / option	Meaning
<code>adduser -S -G GROUP -u UID NAME</code>	Alpine: create a system user (<code>-S</code> : no password, no login shell)
<code>useradd --system --uid UID NAME</code>	Debian/Ubuntu equivalent
<code>USER NAME</code> or <code>USER UID[:GID]</code>	run the following steps and the container as that user
<code>COPY --chown=USER:GROUP</code>	copy files with an owner (lesson 025)
<code>docker run --user UID</code>	override the user for one container
<code>docker image inspect --format '{{.Config.User}}'</code>	the image's user; empty means root

RECAP

- Containers run as root unless the image or `docker run --user` says otherwise.
- Create a user (`adduser` / `useradd`) and switch with `USER` , preferably by numeric ID.
- Files created before `USER` belong to root: give the user only the folders it must write to.
- Never fix permission errors by switching back to root.

Full lesson with every command and its real output: [033-user](#)

Lesson 034 · Dockerfile best practices

A Dockerfile that "works" can still be slow to rebuild, larger than needed, run as root and stop badly. This lesson reviews a first version of the cafe API's Dockerfile against the practices of this module, measures the difference, and uses Docker's built-in checker (`docker build --check`) to catch mistakes before they ship.

VISUAL

Dockerfile.before		Dockerfile.after	
FROM node:24-alpine		FROM node:24-alpine	pinned, small base
(023)			
COPY . /app	← everything	ENV NODE_ENV=production	
WORKDIR /app		WORKDIR /app	WORKDIR before use
(024)			
RUN npm install	← after the code	COPY package.json package-lock.json ./	dependencies first
(025, 039)			
		RUN npm ci --omit=dev && npm cache clean --force	one clean step
(026)			
		COPY server.js ./	then only the code
(033)		USER node	not root
EXPOSE 3000		EXPOSE 3000	documented port
(032)			
CMD node server.js	← shell form	CMD ["node", "server.js"]	exec form
(027)			
		+ Dockerfile.after.dockerignore	a small build context
(037)			

COMMANDS

Practice	Why
pin the base tag (or digest)	reproducible builds (lessons 021, 023)
copy dependency files first, then the code	code changes do not re-install dependencies (lesson 039)
<code>npm ci</code> / pinned versions	installs exactly the lock file
clean caches in the same <code>RUN</code>	smaller layers (lesson 017)
<code>.dockerignore</code>	small context; no secrets or junk in the image (lesson 037)
<code>USER</code> a non-root user	less damage if the application is compromised (lesson 033)
exec form <code>CMD</code> / <code>ENTRYPOINT</code>	signals reach the application (lesson 027)
<code>docker build --check</code>	Docker's built-in Dockerfile checks, without building

RECAP

- Pinned small base, dependencies before code, one clean `RUN` per concern, `.dockerignore`, non-root `USER`, exec form.
- Dependency-first ordering keeps rebuilds fast: code changes reuse the dependency layer.
- `docker build --check` catches Dockerfile mistakes without building and fails CI on warnings.
- Tools catch some problems; a review against the checklist catches the rest.

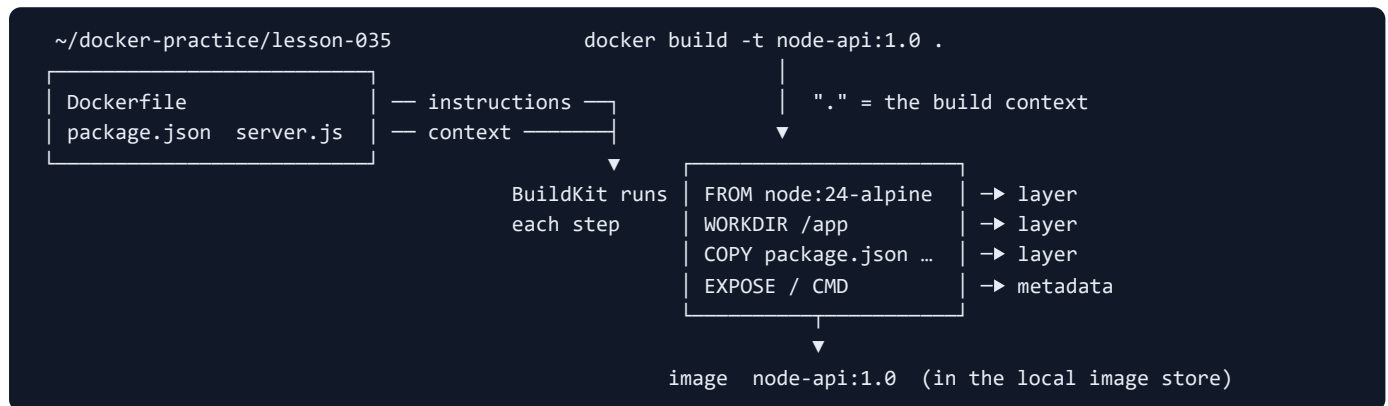
Full lesson with every command and its real output: [034-dockerfile-best-practices](#)

Module 05 · Docker builds

Lesson 035 · docker build

`docker build` turns a **Dockerfile** and a folder of files (the **build context**) into an **image**. The Docker CLI sends the context to the builder (BuildKit), the builder runs the Dockerfile's instructions one by one, and the result is stored as an image with the name and tag you give it with `-t`. This lesson covers the command itself: naming, tagging, choosing the Dockerfile, and reading the build output.

VISUAL



COMMANDS

Command / flag	Meaning
<code>docker build PATH</code>	build an image; PATH is the build context (lesson 036)
<code>-t NAME:TAG</code>	name and tag the result; repeat <code>-t</code> for several tags; no tag means <code>latest</code> (lesson 021)
<code>-f FILE</code>	use another Dockerfile (default: <code>Dockerfile</code> in the context)
<code>-q</code>	quiet: print only the image ID
<code>--progress=plain</code>	full, line-by-line build output (the default in a terminal is a compact live view)
<code>--no-cache</code>	run every step again, ignoring the cache (lesson 039)

RECAP

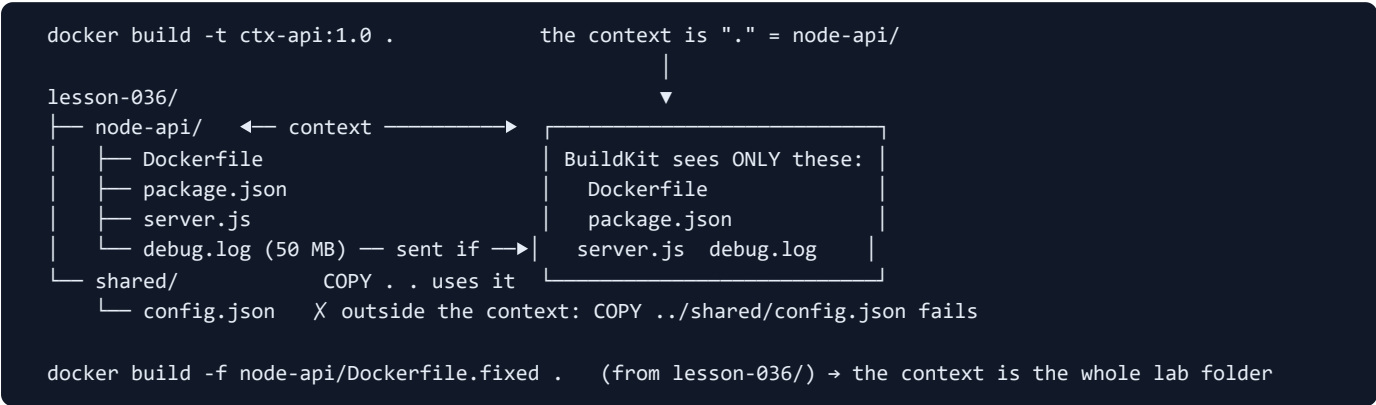
- `docker build -t NAME:TAG CONTEXT` builds an image from `CONTEXT/Dockerfile`.
- `-f` chooses another Dockerfile; several `-t` give one image several names.
- The build output lists one step per instruction and ends with `naming to ...`.
- `failed to read dockerfile` means the builder found no Dockerfile under the expected name.

Full lesson with every command and its real output: [035-docker-build](#)

Lesson 036 · The build context

The last argument of `docker build` is the **build context**: the folder whose files the builder may use. The CLI sends that folder to the builder (BuildKit) before any instruction runs, and `COPY / ADD` can only read files from it. Two consequences: everything in the context is transferred (big folders make builds slow), and a Dockerfile can never copy a file from outside the context, such as `../shared/config.json`.

VISUAL



COMMANDS

Part	Meaning
<code>docker build</code>	the context is the current folder
<code>docker build/../ / ... /path/to/dir</code>	any folder can be the context
<code>-f node-api/Dockerfile.fixed</code>	the Dockerfile may live anywhere; <code>COPY</code> paths stay relative to the context
<code>transferring context: N</code>	how much of the context was sent (only the paths <code>COPY / ADD</code> need)
<code>--no-cache</code>	run every step again (used here so each build reports its context)

RECAP

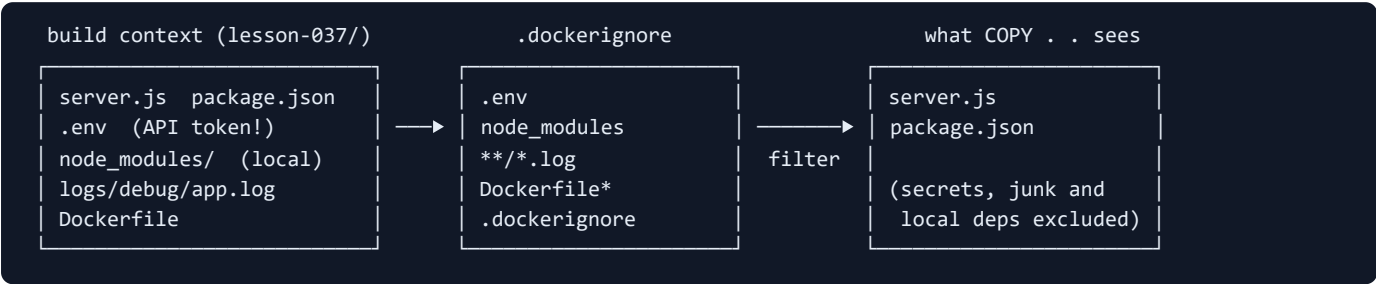
- The context is the folder given to `docker build`; it is sent to the builder before the build starts.
- `COPY` and `ADD` can only read files inside the context; `../` cannot leave it.
- BuildKit transfers what `COPY / ADD` name; `COPY ..` sends (and bakes in) everything: check `transferring context`.
- To use files from several folders, make their common parent the context and select the Dockerfile with `-f`.

Full lesson with every command and its real output: [036-build-context](#)

Lesson 037 · .dockerignore

A `.dockerignore` file at the root of the build context lists files the builder must **never** see. `COPY . .` then copies everything except them. It keeps three kinds of files out of images: **secrets** (`.env`, keys), **junk** (logs, `.git`, editor folders) and **things the image builds itself** (`node_modules`, `dist/`). The syntax looks like `.gitignore`, with one important difference you will hit in Break it.

VISUAL



COMMANDS

Pattern	Matches
<code>.env</code>	the file <code>.env</code> at the root of the context
<code>node_modules</code>	the folder <code>node_modules</code> at the root, and everything inside it
<code>*.log</code>	<code>.log</code> files at the root only
<code>**/*.log</code>	<code>.log</code> files in any folder
<code>!README.md</code>	an exception: include it even if an earlier pattern excluded it
<code># comment</code>	a comment line

RECAP

- `.dockerignore` sits at the root of the context and excludes files from `COPY / ADD`.
- Exclude secrets, junk and anything the image builds itself (`node_modules`, `dist/`).
- Patterns are anchored at the context root: use `**/` to match at any depth.
- An allowlist (`*` plus `!` exceptions) is the safest form.

Full lesson with every command and its real output: [037-dockerignore](#)

Lesson 038 · Build layers

Every `RUN`, `COPY` and `ADD` instruction produces a **layer**: the set of files that step added, changed or deleted. An image is its base image's layers plus one layer per such instruction, stacked; other instructions (`WORKDIR`, `CMD`, `ENV`, `EXPOSE`) only change metadata. Layers are **immutable**: a later step can hide a file, but never remove it from an earlier layer. That is why the way you split a Dockerfile into steps decides how big the image is.

VISUAL

Dockerfile.wasteful		image layers (each one is stored and pulled)
FROM alpine:3.23	→	base layer 9 MB
RUN head -c 30000000 ... > download	→	+ /tmp/download.bin 30 MB
RUN sha256sum ... > /checksum.txt	→	+ /checksum.txt < 1 kB
RUN rm /tmp/download.bin	→	"whiteout": hide download.bin ≈ 8 kB
		total (unpacked) ≈ 39 MB (the 30 MB are still there)
Dockerfile.fixed		
FROM alpine:3.23	→	base layer 9 MB
RUN download && checksum && rm	→	+ /checksum.txt only < 1 kB
		total ≈ 9 MB

COMMANDS

Command	What it shows
<code>docker history IMAGE</code>	every layer: the instruction that created it and its size
<code>--format '{{.Size}}\t{{.CreatedBy}}'</code>	only those two columns (<code>--no-trunc</code> shows full commands)
<code>docker image ls IMAGE</code>	the total size of the image
<code>docker image inspect --format '{{len .RootFS.Layers}}' IMAGE</code>	how many file system layers the image has

RECAP

- `RUN`, `COPY` and `ADD` each add a layer; other instructions are metadata.
- Layers are immutable: deleting a file in a later step hides it but does not shrink the image.
- Create, use and delete temporary files in one `RUN`.
- `docker history` shows every layer and its size; use it to find what makes an image big.

Full lesson with every command and its real output: [038-build-layers](#)

Lesson 039 · The build cache

BuildKit remembers every layer it built. Before running a step, it checks whether it has already built **the same instruction, on the same parent layer, with the same input files**; if so, it reuses the stored layer (`CACHED`) in milliseconds. As soon as one step changes, that step **and every step after it** must run again. The order of the Dockerfile therefore decides whether a one-line code change rebuilds in one second or reinstalls all dependencies.

VISUAL

```
Dockerfile.slow (code before dependencies)      Dockerfile (dependencies before code)

FROM python:3.14-slim          CACHED          FROM python:3.14-slim          CACHED
WORKDIR /app                  CACHED          WORKDIR /app                  CACHED
COPY . .                      X app.py changed COPY requirements.txt .      CACHED (unchanged)
RUN pip install ...           X runs again  RUN pip install ...          CACHED
CMD ...                       X app.py changed
                                CMD ...
                                edit app.py → copy one file again

edit app.py → reinstall every package

cache key of a step = parent layer + instruction text + checksum of the files it copies
a miss invalidates every step below it
```

COMMANDS

Command / output	Meaning
<code>#N CACHED</code>	step N was reused from the cache
a step without <code>CACHED</code>	it ran; so does every later step
<code>docker build --no-cache</code>	ignore the cache for every step
<code>docker build --no-cache-filter STAGE</code>	ignore it for one stage only (multi-stage, lesson 087)
<code>docker builder du</code> / <code>docker builder prune</code>	how much disk the cache uses / delete it

RECAP

- A step is reused when its parent layer, instruction and input files are unchanged.
- The first cache miss invalidates every later step.
- Order Dockerfiles from rarely-changing to often-changing: dependencies before code.
- `--no-cache` forces a full rebuild; `docker builder prune` frees the cache's disk space.

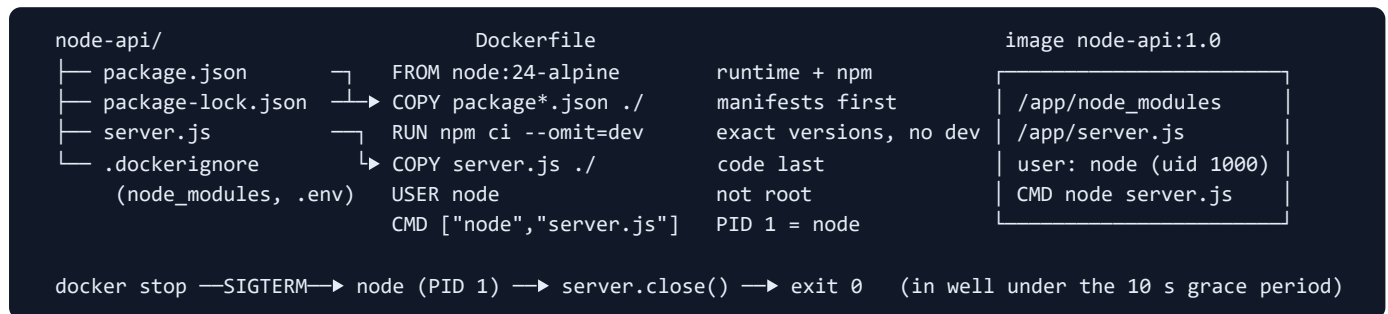
Full lesson with every command and its real output: [039-build-cache](#)

Module 06 · Application containerization

Lesson 040 · Containerizing a Node.js application

How to write a proper Dockerfile for a Node.js service, step by step: an official, pinned base image, a reproducible dependency install from the lockfile (`npm ci`), the cache-friendly order from lesson 039, an unprivileged user, and a start command that lets Node.js receive the stop signal. The same pattern works for Express, Fastify or NestJS APIs.

VISUAL



COMMANDS

Instruction	Why
<code>FROM node:24-alpine</code>	the official image, pinned to a major version (lesson 021); Alpine keeps it small
<code>ENV NODE_ENV=production</code>	many libraries switch to production behaviour with it
<code>COPY package.json package-lock.json ./</code> then <code>RUN npm ci</code>	install exactly the locked versions; cached until they change
<code>--omit=dev</code>	skip development dependencies (test tools, linters) in the runtime image
<code>USER node</code>	the official image's unprivileged user
<code>CMD ["node", "server.js"]</code>	exec form, not <code>npm start</code> : node is PID 1 and gets <code>SIGTERM</code>

RECAP

- Base image pinned, manifests copied first, `npm ci --omit=dev` from the lockfile, code last.
- Run as the image's `node` user, with an exec-form `CMD` so Node.js receives `SIGTERM`.
- Commit `package-lock.json` : without it `npm ci` fails and builds are not reproducible.
- Configure the application with environment variables (`PORT` , `GREETING`), not by rebuilding.

Full lesson with every command and its real output: [040-containerize-node](#)

Lesson 041 · Containerizing a Python application

How to containerize a Python web API (Flask here; FastAPI works the same way with `uvicorn` instead of `gunicorn`): a slim official base image, pinned requirements installed before the code, two Python settings that matter in containers, an unprivileged user, and a **production server** listening on `0.0.0.0`. The last point is the most common reason a containerized Python app "runs but cannot be reached".

VISUAL



COMMANDS

Instruction	Why
<code>FROM python:3.14-slim</code>	Debian-based and small; the <code>-alpine</code> variant is smaller but uses musl, which some wheels do not support
<code>PYTHONDONTWRITEBYTECODE=1</code>	do not write <code>.pyc</code> files into the container
<code>PYTHONUNBUFFERED=1</code>	print logs immediately instead of buffering them (otherwise <code>docker logs</code> lags or loses lines)
<code>pip install --no-cache-dir</code>	do not keep pip's download cache in the layer
<code>useradd --uid 10001 appuser + USER appuser</code>	run without root
<code>gunicorn --bind 0.0.0.0:5000</code>	a production WSGI server, listening on all interfaces of the container

RECAP

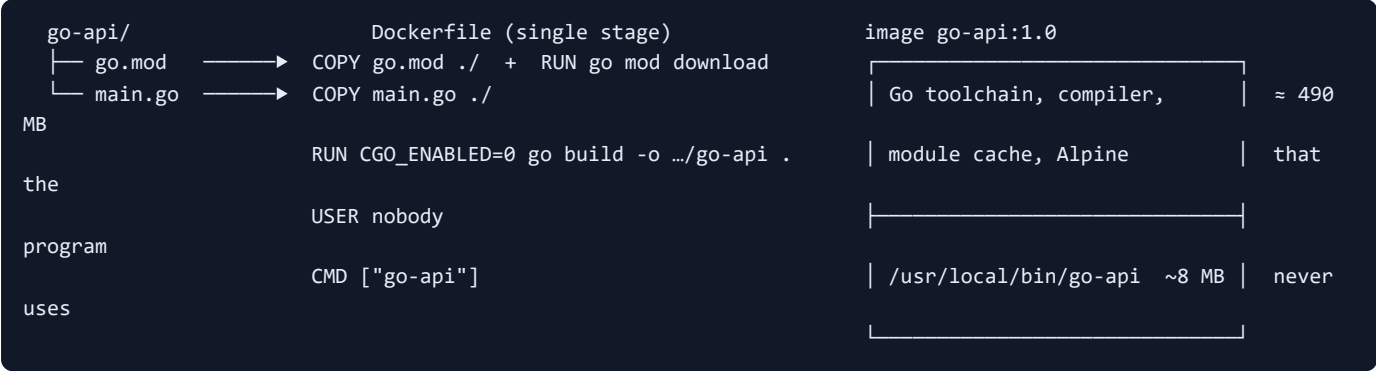
- Slim base image, requirements before code, `--no-cache-dir`, an unprivileged user.
- `PYTHONUNBUFFERED=1` makes logs appear in `docker logs` immediately.
- Run a production server (gunicorn, uvicorn), not the framework's development server.
- A server bound to `127.0.0.1` cannot be reached through a published port: bind to `0.0.0.0`.

Full lesson with every command and its real output: [041-containerize-python](#)

Lesson 042 · Containerizing a Go application

Go compiles to a single binary. Containerizing it means: compile inside the official Go image (no Go installation on your computer), with the module files copied first so dependencies are cached, as a **static** binary (`CGO_ENABLED=0`), and run it as an unprivileged user. This lesson builds a working single-stage image and measures what it costs: the whole Go toolchain ships with an 8 MB program. Lesson 089 removes it with a multi-stage build.

VISUAL



COMMANDS

Instruction / setting	Why
COPY go.mod ./ (and go.sum) then RUN go mod download	dependencies in their own cached layer
CGO_ENABLED=0	pure-Go static binary: no dependency on the C library of the image
go build -o /usr/local/bin/go-api .	build the package in the current folder into one file
USER nobody	Alpine's unprivileged user (uid 65534)
CMD ["go-api"]	exec form: the binary is PID 1 and receives signals

RECAP

- Build inside `golang:1.26-alpine` ; copy `go.mod / go.sum` first and run `go mod download` for caching.
- `CGO_ENABLED=0` produces a static binary that runs on any Linux image.
- A single-stage Go image carries the whole toolchain: hundreds of MB for a program of a few MB.
- `docker create` + `docker cp` copies files out of an image.

Full lesson with every command and its real output: [042-containerize-go](#)

Lesson 043 · Containerizing a Java application

How to containerize a Java service: compile and package it into an executable **jar** inside an official JDK image (Eclipse Temurin), run it with `java -jar` as an unprivileged user, and understand how the JVM sizes its memory inside a container. The example is a plain JDK HTTP server, so the build needs no Maven download; the Spring Boot pattern, which follows the same principles, is shown as an annotated Dockerfile.

VISUAL

java-api/src/Main.java

Dockerfile (single stage)

image java-api:1.0

FROM eclipse-temurin:25-jdk-alpine

COPY src ./src

RUN javac -d classes ...

&& jar --create --file app.jar

--main-class Main -C classes .

USER app

CMD ["java", "-jar", "app.jar"]

JDK: compiler, jar tool,

JVM, Alpine

/app/app.jar

JVM in a container: docker run -m 512m ... → the JVM sees 512 MB → default max heap = 25 % = 128 MB

-XX:MaxRAMPercentage=75 → max heap = 384 MB

COMMANDS

Instruction / option	Why
FROM eclipse-temurin:25-jdk-alpine	an official OpenJDK build (Java 25 LTS) with the compiler
javac -d classes + jar --create --main-class Main	compile, then package into an executable jar
adduser -S app + USER app	an unprivileged system user (Alpine's adduser)
CMD ["java", "-jar", "app.jar"]	exec form: the JVM is PID 1 and handles SIGTERM
-m 512m	a memory limit for the container (lesson 098)
-XX:MaxRAMPercentage=75	let the heap use 75 % of the container's memory

RECAP

- Compile and package inside an official JDK image; run with `java -jar` as an unprivileged user.
- The JVM respects container limits; by default the heap is 25 % of the container's memory.
- Set `-XX:MaxRAMPercentage` (or `JAVA_TOOL_OPTIONS`) instead of a fixed `-Xmx` .
- `Could not find or load main class` : check the class name and the jar's manifest.

Full lesson with every command and its real output: [043-containerize-java](#)

Lesson 044 · Containerizing a PHP application

A PHP application usually runs as **two** containers: **PHP-FPM**, which executes PHP code and speaks the FastCGI protocol on port 9000, and **Nginx**, which receives HTTP requests, serves static files itself and forwards `.php` requests to PHP-FPM. Each container does one job, the PHP image contains the code, and the two find each other by name on a shared network. This is also how Laravel and Symfony applications are deployed.

VISUAL



COMMANDS

Command / setting	Meaning
FROM php:8.5-fpm-alpine	the official PHP image, FPM variant (no web server inside)
php.ini-production -> php.ini	the image ships recommended production settings; activate them
docker network create php-net	a network on which containers find each other by name
--name php --network php-net	the PHP container is reachable as <code>php</code> from that network
-v "\$(pwd)/nginx.conf:/etc/nginx/conf.d/default.conf:ro"	replace Nginx's default site with ours, read-only
fastcgi_pass php:9000	forward PHP requests to host <code>php</code> , port 9000, using FastCGI

RECAP

- PHP runs as PHP-FPM (FastCGI, port 9000) behind Nginx (HTTP): two containers, one job each.
- The PHP image contains the code; Nginx reaches it by container name on a shared network.
- Nginx resolves `fastcgi_pass` hosts at start-up: `host not found in upstream` means the backend is not on the network.
- Static files are served by Nginx; everything else goes to PHP-FPM.

Full lesson with every command and its real output: [044-containerize-php](#)

Lesson 045 · Comparing the stacks

Lessons 040–044 containerized the same small API in five languages. Every Dockerfile followed the same principles (official pinned base image, dependencies before code, unprivileged user, `exec-form` start command), but the results are very different, because each language needs something different **at run time**. This lesson builds all five side by side and measures them: image size, memory in use, and what is inside each image.

VISUAL

	needs at build time	needs at run time	single-stage image contains
Node.js	npm + package-lock.json	node runtime + node_modules	runtime + npm + code
Python	pip + requirements.txt	interpreter + site-packages	interpreter + pip + code
Go	Go toolchain + go.mod	nothing (static binary)	toolchain + binary ← most waste
Java	JDK (javac, Maven/Gradle)	JRE (JVM)	JDK + jar ← waste
PHP	Composer (for frameworks)	PHP-FPM + extensions (+ Nginx)	PHP-FPM + code

the gap between "build time" and "run time" is what multi-stage builds remove (lessons 087–091)

Stack	Start command	Typical production base
Node.js	<code>node server.js</code>	<code>node:24-alpine</code> (or a distroless Node.js image)
Python	<code>gunicorn app:app</code> / <code>uvicorn main:app</code>	<code>python:3.14-slim</code>
Go	<code>/go-api</code>	<code>gcr.io/distroless/static</code> or <code>scratch</code>
Java	<code>java -jar app.jar</code>	<code>eclipse-temurin:25-jre-alpine</code>
PHP	<code>php-fpm</code> behind Nginx	<code>php:8.5-fpm-alpine</code>

COMMANDS

Command	Meaning
<code>docker build -t NAME FOLDER</code>	build with FOLDER as the context (and FOLDER/Dockerfile)
<code>docker image ls --filter 'reference=stack-*'</code>	only images whose name matches the pattern
<code>docker stats --no-stream</code>	one snapshot of CPU, memory, network and disk I/O per container (lesson 095)
<code>--format '{{.Name}}\t{{.MemUsage}}'</code>	only the name and memory columns

RECAP

- The same principles apply to every stack; what differs is what the application needs at run time.
- Measure, do not guess: `docker image ls` for size, `docker stats` for memory.
- Build tools (compilers, package managers) are only needed at build time.
- Swapping in a runtime-only base image breaks the build: separate build and run stages instead (lesson 087).

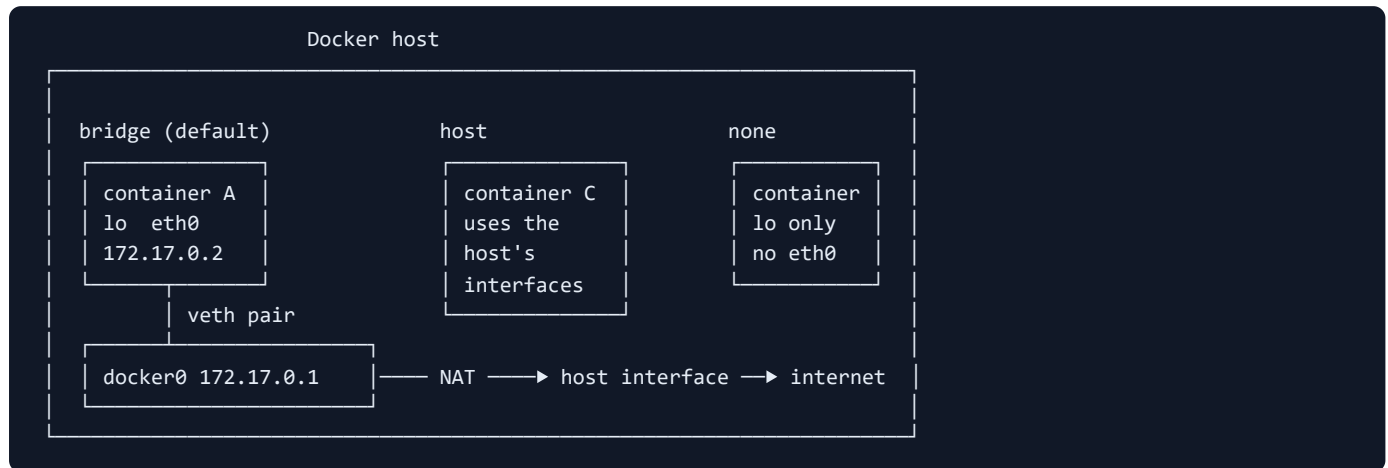
Full lesson with every command and its real output: [045-comparing-stacks](#)

Module 07 · Networking

Lesson 046 · Networking fundamentals

Every container gets its own **network namespace**: its own network interfaces, IP addresses, routing table and ports. Docker connects that namespace to a **network** chosen with `--network`. The three built-in networks show the three basic choices: `bridge` (the default: a private network behind the host, with outgoing access), `host` (no isolation: the container uses the host's network directly) and `none` (no network at all, only loopback).

VISUAL



Network	Driver	Isolation	Typical use
bridge	bridge	own IP on a private subnet; outgoing traffic through NAT	the default for single containers
user-defined bridge	bridge	the same, plus DNS between containers	applications with several containers (lesson 048)
host	host	none: shares the host's interfaces and ports	special cases (network tools, performance), Linux only in full
none	null	no interface except loopback	jobs that must not talk to anything

COMMANDS

Command / flag	What it does
<code>docker network ls</code>	list the networks of the engine
<code>--network NAME</code>	attach the new container to that network (default: <code>bridge</code>)
<code>ip -4 -o addr</code>	the container's IPv4 addresses, one line per interface
<code>ip route</code>	the container's routing table; <code>default via</code> is its gateway

RECAP

- Each container has its own network namespace: interfaces, IP address, routes, ports.
- `bridge` (default): private address, outgoing access through NAT; `host`: the host's network; `none`: loopback only.
- `docker network ls`, `docker inspect ... .NetworkSettings` and `ip addr` / `ip route` inside the container show where a container is connected.
- For any connection problem, first check which network the container is on.

Full lesson with every command and its real output: [046-networking-fundamentals](#)

Lesson 047 · The default bridge network

Containers started without `--network` join the **default bridge** network (`bridge` , the `docker0` interface on the host). Containers on it can reach each other **by IP address**, but **not by name**: the default bridge has no service discovery. IP addresses change whenever a container is recreated, so the default bridge is fine for single containers and a trap for applications made of several containers.

VISUAL

```
default "bridge" network 172.17.0.0/16 (docker0 = 172.17.0.1)
```

web (nginx)
172.17.0.2

client (busybox)
172.17.0.3

← http://172.17.0.2 — |
← http://web — |

✓ by IP

✗ "bad address 'web'"
(no DNS for container names)

recreate "web" → it may get another IP → every hard-coded IP breaks

COMMANDS

Command	What it does
<code>docker network inspect bridge</code>	the network's configuration and the containers attached to it
<code>--format '{{range .Containers}}...{{end}}'</code>	loop over the attached containers
<code>docker inspect C --format '{{.NetworkSettings.Networks.bridge.IPAddress}}'</code>	a container's IP on the <code>bridge</code> network
<code>wget -qO- URL</code>	(BusyBox) fetch a URL and print it, quietly

RECAP

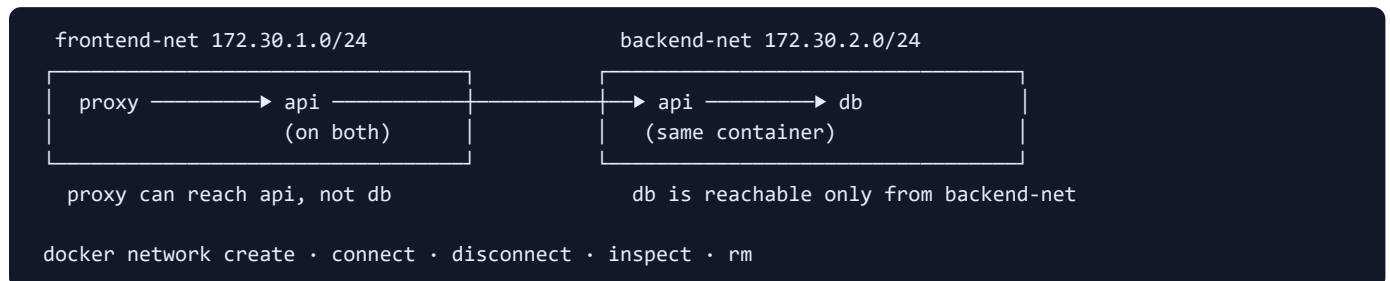
- Containers without `--network` join the default `bridge` network.
- On the default bridge, containers reach each other by IP only; names give `bad address` .
- Container IPs are not stable across recreation: never hard-code them.
- User-defined networks add DNS for container names (lesson 048).

Full lesson with every command and its real output: [047-default-bridge](#)

Lesson 048 · Custom networks

A **user-defined network** (`docker network create NAME`) is a bridge network you own. Compared with the default bridge it gives you three things: **DNS** (containers find each other by name), **isolation** (only the containers you attach can talk to each other) and **control** (you choose the subnet, and connect or disconnect running containers). Every multi-container application should get its own network.

VISUAL



COMMANDS

Command	What it does
<code>docker network create [--subnet CIDR] NAME</code>	create a user-defined bridge network
<code>docker run --network NAME</code>	start a container on that network
<code>docker network connect NET CONTAINER</code>	add a running container to another network (a new interface)
<code>docker network disconnect NET CONTAINER</code>	remove it from that network
<code>docker network inspect NAME</code>	subnet, gateway and attached containers
<code>docker network rm NAME</code>	delete a network (no containers may be attached)

RECAP

- `docker network create` makes a user-defined bridge with DNS, isolation and a subnet you can choose.
- Containers resolve each other's names only on networks they share.
- `docker network connect / disconnect` changes a running container's networks.
- Split networks by tier (frontend, backend) so each container reaches only what it needs.

Full lesson with every command and its real output: [048-custom-networks](#)

Lesson 049 · Container-to-container communication

Real applications are several containers: an API and its cache, a worker and its queue. They talk to each other over a shared user-defined network, using the **container name** as the host name and the **container port** (the port the service listens on inside its container). Publishing with `-p` is only for traffic from outside Docker. Configuration tells each service where its dependencies are, usually through environment variables such as `REDIS_HOST=redis`.

VISUAL



COMMANDS

Part	What it does
<code>--network cafe-net</code>	both containers on one user-defined network, so names resolve
<code>--name redis</code>	the container name is also its DNS name on the network
<code>-e REDIS_HOST=redis</code>	configuration: where the API finds Redis (lesson 059)
<code>-p 8080:5000</code>	publish the API's container port 5000 on the host's port 8080
<code>redis:6379</code>	container-to-container: the name and the container port, never the host port

RECAP

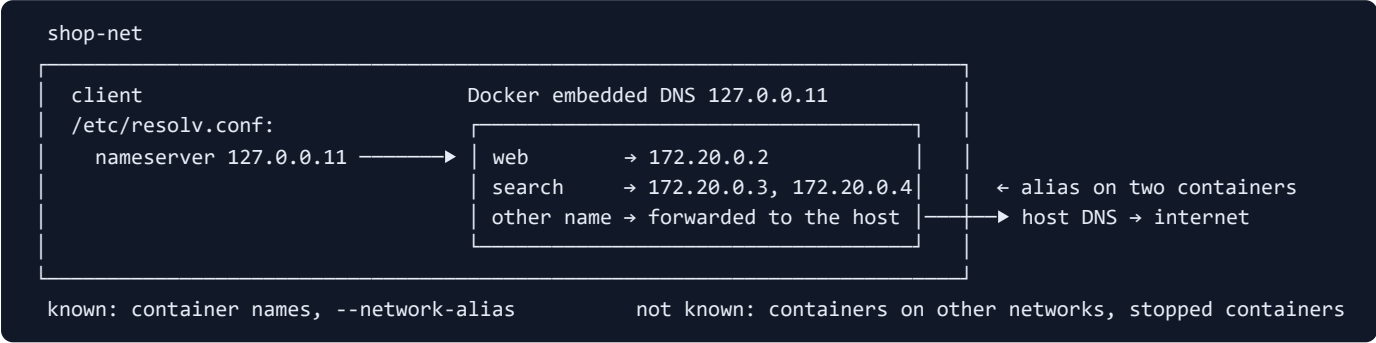
- Containers on a shared user-defined network reach each other by **name** and **container port**.
- `-p` is only for traffic from outside Docker; internal services such as Redis need no published port.
- Pass dependency addresses as configuration (`-e REDIS_HOST=redis`); a 500 with a name-resolution error in the logs means the configured name does not exist on the network.
- Network aliases add extra DNS names to a container.

Full lesson with every command and its real output: [049-container-to-container](#)

Lesson 050 · Container DNS

On a user-defined network, every container uses Docker's **embedded DNS server** at `127.0.0.11`. It answers the names of the containers on the same network (their **container names** and **network aliases**) and forwards every other name (`example.com`) to the host's DNS servers. Knowing exactly which names it knows explains most "could not resolve host" errors between containers.

VISUAL



COMMANDS

Command / flag	What it does
<code>nslookup NAME</code>	(BusyBox) ask the configured DNS server for NAME
<code>/etc/resolv.conf</code>	the DNS configuration of the container (<code>127.0.0.11</code> on user-defined networks)
<code>--network-alias NAME</code>	an extra DNS name for the container on that network; may be shared
<code>docker inspect C --format '{{...DNSNames}}'</code>	the names Docker's DNS answers for C on each network
<code>--dns IP</code>	the server that Docker's DNS forwards external names to, for this container

RECAP

- On user-defined networks, containers use Docker's embedded DNS at `127.0.0.11`.
- It knows container names and network aliases on the same network, and forwards all other names.
- `docker inspect ... DNSNames` lists exactly which names resolve; add others with `--network-alias`.
- Several containers can share an alias: DNS returns all running ones.

Full lesson with every command and its real output: [050-container-dns](#)

Lesson 051 · EXPOSE vs publishing ports

`EXPOSE 80` in a Dockerfile is **documentation**: it records which port the application listens on. It opens nothing. **Publishing** (`-p HOST:CONTAINER` on `docker run`) is what makes a container port reachable from outside Docker: the engine forwards a port of the host to the port of the container. Containers on a shared network reach each other's ports directly, exposed or not.

VISUAL

Dockerfile: `EXPOSE 80`

→ metadata only: "this image listens on 80/tcp"

`docker run nginx`

→ reachable from other containers on its network on :80
NOT reachable from the host

`docker run -p 8080:80 nginx`

→ host 0.0.0.0:8080 → container :80 (every interface of the host)

`docker run -p 127.0.0.1:8080:80`

→ host 127.0.0.1:8080 → container :80 (only this computer)

`docker run -P nginx`

→ host <random> → every EXPOSEd port

browser

→ :8080

→ forward →

container

nginx :80

← other containers (no -p needed)

COMMANDS

Form	Meaning
<code>EXPOSE 80</code> (Dockerfile)	metadata: the application listens on 80/tcp; publishes nothing
<code>-p 8080:80</code>	host port 8080 on all interfaces → container port 80
<code>-p 127.0.0.1:8080:80</code>	the same, but only reachable from this computer
<code>-p 8080:80/udp</code>	a UDP port (the default is TCP)
<code>-P</code>	publish every exposed port on a random free host port
<code>docker port C</code>	the published ports of a container

RECAP

- `EXPOSE` is documentation of the listening port; it never makes anything reachable from the host.
- `-p HOST:CONTAINER` publishes a port; add `127.0.0.1:` to keep it local; `-P` publishes exposed ports randomly.
- Containers on the same network reach each other's ports without `-p`.
- `docker port` and the `PORTS` column of `docker ps` show what is really published.

Full lesson with every command and its real output: [051-expose-vs-publish](#)

Lesson 052 · Network troubleshooting

"I can't reach my container" has a short list of causes, and each one has a command that proves or rules it out. Work from the outside in: is the container running, is the port published, is the application listening on the right port, is it listening on the right **address**, are client and server on the same network, is the name right? The classic trap is an application listening on `127.0.0.1` inside its container: it works from inside, and nothing else can reach it.

VISUAL



Symptom	Likely cause	Proof
Connection refused (curl exit 7) on the host	port not published, or wrong host port	docker port C
Empty reply / Connection reset (exit 52/56)	published, but the app listens on another port or on 127.0.0.1	netstat -tln in the container's namespace
bad address / Could not resolve host	wrong name, or not on the same network	nslookup NAME , docker inspect ... Networks
request hangs, then times out	a firewall, or a dependency that hangs (lesson 049)	test the dependency from inside the container

COMMANDS

Command	What it proves
<code>docker ps --filter name=C</code>	the container runs (and has not exited)
<code>docker port C</code>	which host ports forward to which container ports
<code>docker run --rm --network container:C busybox:1.37 netstat -tln</code>	the listening sockets inside C, with tools C does not have
<code>docker inspect C --format '{{range \$n, \$_ := .NetworkSettings.Networks}}{{ \$n }} {{end}}'</code>	C's networks
<code>docker run --rm --network NET busybox:1.37 nslookup NAME</code>	whether NAME resolves on NET

RECAP

- Troubleshoot from the outside in: running → published → listening port → listening address → network → name.
- `127.0.0.1` inside a container means only that container; servers in containers must listen on `0.0.0.0`.
- `--network container:NAME` runs a debugging tool in another container's network namespace.
- `-p HOST:CONTAINER` : the container port must be the application's real listening port.

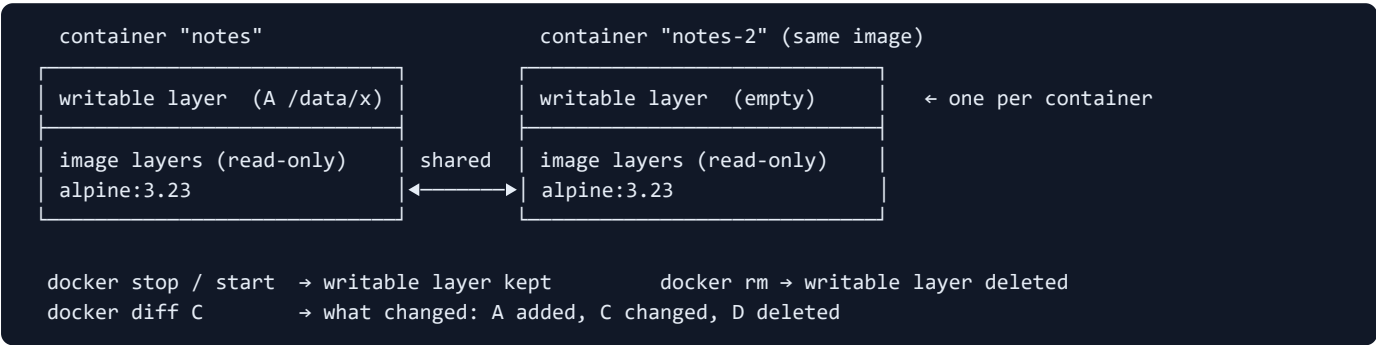
Full lesson with every command and its real output: [052-network-troubleshooting](#)

Module 08 · Storage

Lesson 053 · The container's writable layer

An image is read-only. When a container starts, Docker adds a thin **writable layer** on top of the image's layers: every file the container creates, changes or deletes is recorded there (copy-on-write). That layer belongs to the container: it survives a stop and a restart, and it is **deleted with the container**. Anything that must outlive a container (uploads, databases, logs you want to keep) belongs in a volume or a bind mount (lessons 054–058).

VISUAL



COMMANDS

Command	What it does
<code>docker diff C</code>	changes in C's writable layer: A added, C changed, D deleted
<code>docker start -a C</code>	start an existing (stopped) container again, attached
<code>docker container ls -s</code>	include the size of the writable layer

RECAP

- Each container has a writable layer on top of the read-only image layers (copy-on-write).
- It survives stop and start, and is deleted with the container: never keep important data there.
- `docker diff` shows what a container changed; `docker container ls -s` shows how much.
- Persistent data goes into volumes or bind mounts.

Full lesson with every command and its real output: [053-writable-layer](#)

Lesson 054 · Bind mounts

A **bind mount** makes a file or folder of the host appear at a path inside the container. Both sides see the same files: change one on the host and the container sees it immediately. That makes bind mounts the tool for development (edit code on your laptop, run it in a container) and for handing configuration files to a container. The host path must be **absolute**: a bare name such as `site` means something else entirely (a named volume).

VISUAL

host: ~/docker-practice/lesson-054/site/

container: /usr/share/nginx/html/

index.html styles.css

← bind mount →

index.html styles.css

same files

edit on the host → visible in the container at once :ro → the container cannot write

-v "\$(pwd)/site:/usr/share/nginx/html" absolute host path → bind mount ✓

-v site:/usr/share/nginx/html a bare name → named volume "site" ✗ (not your folder)

--mount type=bind,source="\$(pwd)/site",target=/usr/share/nginx/html,readonly explicit form

COMMANDS

Form	Meaning
-v /abs/host/path:/container/path	bind mount (short form); the host path must be absolute
-v "\$(pwd)/site:/path:ro"	read-only: the container cannot change the files
--mount type=bind,source=...,target=...[,readonly]	the explicit form: fails if the source does not exist
docker inspect C --format '{{range .Mounts}}{{end}}'	the container's mounts

A bind mount **hides** whatever the image had at the target path: here nginx's default page is invisible while the mount is in place.

RECAP

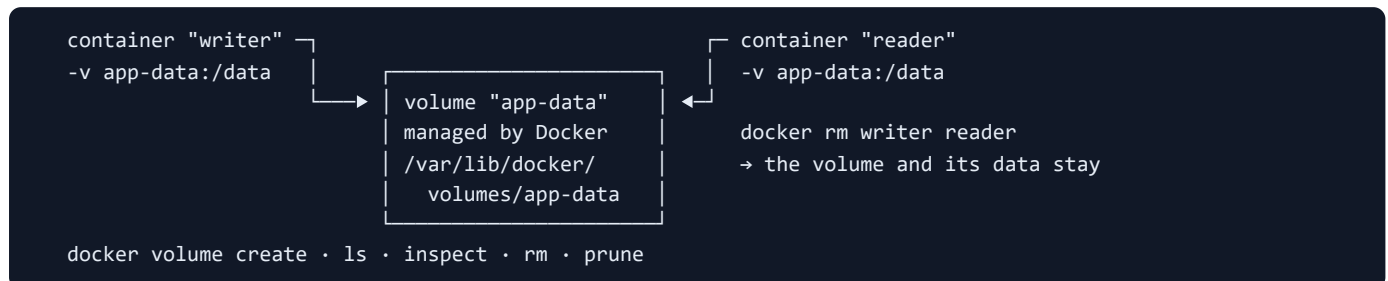
- A bind mount shares a host file or folder with the container; changes are visible on both sides at once.
- The host path must be absolute (`"$(pwd)/..."`); a bare name creates a named volume instead.
- Add `:ro` (or `readonly`) unless the container must write; a mount hides the image's files at the target path.
- `docker inspect ... .Mounts` shows the type, source and destination of every mount.

Full lesson with every command and its real output: [054-bind-mounts](#)

Lesson 055 • Named volumes

A **named volume** is storage that Docker creates and manages, independent of any container. You refer to it by name (`-v app-data:/data`), several containers can use it, and it survives when containers are removed. Volumes are the default choice for data that an application owns: databases, uploads, queues. Docker never deletes a volume on its own; you remove it explicitly, once no container uses it.

VISUAL



COMMANDS

Command	What it does
<code>docker volume create NAME</code>	create an empty volume (<code>docker run -v NAME:...</code> also creates it if missing)
<code>-v NAME:/path[:ro]</code>	mount the volume at <code>/path</code>
<code>--mount</code> <code>type=volume,source=NAME,target=/path</code>	the explicit form
<code>docker volume ls</code>	list volumes
<code>docker volume rm NAME</code>	delete a volume and its data (only when no container uses it)

RECAP

- Named volumes are managed by Docker, independent of containers, and survive `docker rm`.
- Several containers can mount the same volume (use `:ro` for readers).
- A volume in use by any container, even a stopped one, cannot be deleted: `docker ps -a --filter volume=NAME`.
- Back up a volume with a temporary container that mounts it and a host folder.

Full lesson with every command and its real output: [055-named-volumes](#)

Lesson 056 · Inspecting volumes

Volumes outlive containers, so they pile up and need looking after: which volumes exist, which container uses each one, where the data is, how big it is, and which ones nobody uses any more (**dangling**). Some volumes have no meaningful name at all: an image that declares `VOLUME /path` gets an **anonymous volume** (a random 64-character name) every time a container starts without a mount for that path. Anonymous volumes are where "lost" data is usually found.

VISUAL

```
docker volume ls          all volumes: named (app-data) and anonymous (3f9c1e...)
docker volume ls -f dangling=true  volumes no container references → candidates for cleanup (or lost
data!)
docker volume inspect NAME      driver, mountpoint, creation date, labels
docker ps -a --filter volume=NAME  which containers use it
docker system df -v             disk usage of images, containers and volumes

image with VOLUME /data
|  docker run IMAGE             (no -v for /data)
▼
container A → anonymous volume 3f9c1e...  docker rm A → volume stays, dangling
container B → anonymous volume 8a27d0...  a NEW, empty one: the data "disappeared"
```

COMMANDS

Command	What it does
<code>docker volume inspect NAME</code>	details: <code>Driver</code> , <code>Mountpoint</code> , <code>CreatedAt</code> , <code>Labels</code> , <code>Scope</code>
<code>docker volume create --label K=V NAME</code>	a volume with metadata
<code>docker volume ls --filter label=K=V</code>	volumes with that label (also <code>dangling=true</code> , <code>name=...</code>)
<code>docker system df -v</code>	disk usage per image, container and volume
<code>docker volume prune</code>	delete unused anonymous volumes (<code>-a</code> : unused named ones too)

RECAP

- `docker volume ls`, `inspect` and `docker ps -a --filter volume=...` answer "what exists and who uses it".
- Images with `VOLUME` create an anonymous volume for every container started without a mount for that path.
- After `docker rm`, anonymous volumes stay behind as dangling volumes: that is where "disappeared" data usually is.
- Use named volumes for everything you care about; prune dangling ones regularly.

Full lesson with every command and its real output: [056-inspecting-volumes](#)

Lesson 057 · Volumes vs bind mounts

Both put data outside the container's writable layer, but they serve different owners. A **bind mount** is a host path that **you** manage: ideal for source code during development and for configuration files. A **named volume** is storage that **Docker** manages: ideal for application data, portable between hosts, independent of the host's folder layout, and pre-filled from the image when it is empty. A third type, **tmpfs**, lives only in memory. The visible difference that surprises people: an empty volume is filled with the image's files, an empty bind mount hides them.

VISUAL

	bind mount	named volume	tmpfs
source	a host path you choose	managed by Docker	memory
managed with	your file manager, git	docker volume ...	nothing (gone at stop)
empty at first mount	hides the image's files	filled with the image's files (copy-up)	empty
performance (Desktop)	slower (crosses into the VM)	fast (inside the VM)	fastest
typical use	code in development, config	databases, uploads, caches	secrets, scratch files
example	-v "\$(pwd)/src:/app/src"	-v pgdata:/var/lib/postgresql	--tmpfs /tmp

COMMANDS

Form	Type
-v NAME:/path	named volume (created if missing, pre-filled from the image when empty)
-v /abs/path:/path or -v "\$(pwd)/dir:/path"	bind mount (never pre-filled)
--tmpfs /path or --mount type=tmpfs,target=/path,tmpfs-size=64m	in-memory file system
--mount type=volume,source=NAME,target=/path,volume-nocopy	a volume without the copy-up

RECAP

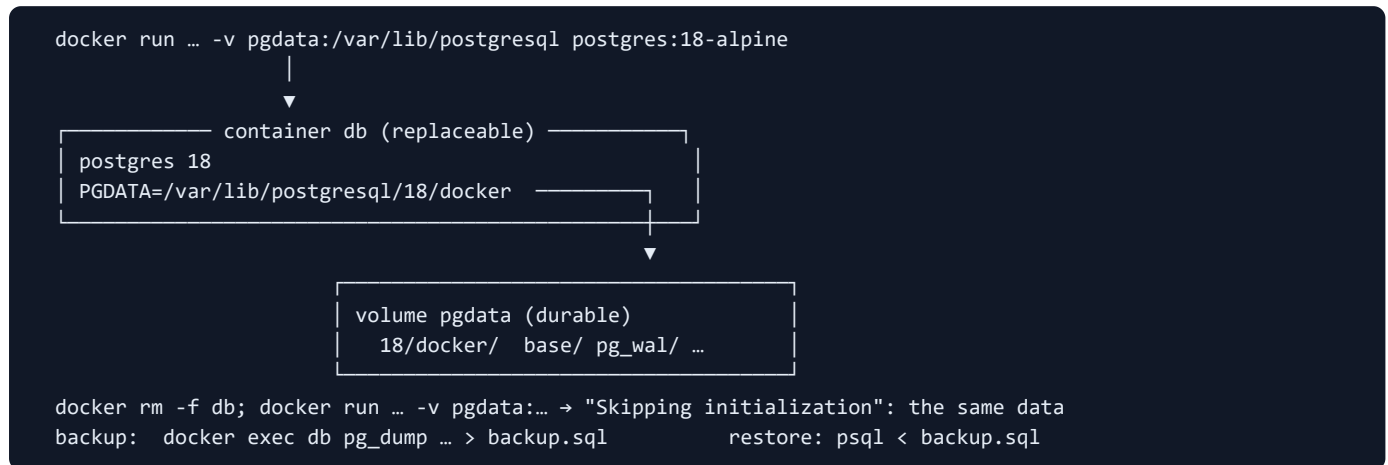
- Bind mount: a host path you manage; hides the image's files; best for code in development and config files.
- Named volume: managed by Docker; pre-filled from the image when empty; best for application data.
- tmpfs: in memory only, gone when the container stops; for scratch files and secrets.
- `docker inspect ... .Mounts` shows which type a container really got.

Full lesson with every command and its real output: [057-volumes-vs-bind-mounts](#)

Lesson 058 · A persistent database

A database in a container is only as durable as its storage. With a named volume at the right path, you can remove the container, upgrade the image within the same major version, or move to a new container, and the data stays. With the wrong path, the database writes into the container (or into an anonymous volume) and the next redeploy starts empty. This lesson runs Postgres 18 the right way, proves the data survives a new container, and backs it up.

VISUAL



COMMANDS

Part	What it does
<code>-v pgdata:/var/lib/postgresql</code>	the named volume at the path the image declares (<code>VOLUME</code>)
<code>PGDATA=/var/lib/postgresql/18/docker</code>	where Postgres 18 really writes, inside that volume
<code>pg_isready -h 127.0.0.1 -U postgres</code>	is the server accepting TCP connections?
<code>psql -U postgres -tAc "SQL"</code>	run one SQL statement; <code>-tA</code> : only the values, no table formatting
<code>pg_dump -U postgres DB</code>	a logical backup (SQL) of one database

RECAP

- Mount a named volume at the path the image declares (`/var/lib/postgresql` for Postgres 18).
- The container is replaceable; the volume holds the data: a new container skips initialisation.
- Wait for readiness with `pg_isready -h 127.0.0.1` , not just "the container is running".
- Volumes are not backups: `pg_dump` regularly and test the restore.

Full lesson with every command and its real output: [058-persistent-database](#)

Module 09 · Environment configuration

Lesson 059 · Environment variables

The same image must run in development, staging and production with different settings: ports, URLs, feature flags, log levels. The standard way to hand those settings to a container is **environment variables**, set with `-e` when the container is created. The image can define defaults (`ENV` in the Dockerfile, lesson 030); `-e` overrides them. Build once, configure per environment: that is the twelve-factor rule "store config in the environment".

VISUAL



COMMANDS

Form	Meaning
<code>-e NAME=value</code>	set a variable in the container
<code>-e NAME</code>	copy NAME's value from the shell that runs <code>docker</code> (unset there → not set in the container)
<code>--env NAME=value</code>	the long form of <code>-e</code>
<code>docker inspect C --format '{{json .Config.Env}}'</code>	every variable the container was created with
<code>ENV NAME=value</code> (Dockerfile)	a default baked into the image (lesson 030)

RECAP

- Configuration goes into the container as environment variables: `-e NAME=value`, or `-e NAME` from the shell.
- `-e` overrides `ENV` defaults of the image; a container's environment is fixed when it is created.
- A container that exits immediately often lacks a required variable: `docker logs` tells you which.
- Passwords in `-e` are visible in `docker inspect` and shell history (lessons 060, 061).

Full lesson with every command and its real output: [059-environment-variables](#)

Lesson 060 · Environment files

Ten `-e` flags make an unreadable command and leak values into your shell history. An **env file** collects them: one `NAME=value` per line, comments with `#`, passed with `--env-file FILE`. One file per environment (`dev.env`, `staging.env`) keeps configuration versioned and reviewable. The format is deliberately simple, and that is the trap: `docker run` does **not** interpret quotes, so `NAME="value"` gives a value that includes the quotes.

VISUAL

app.env

```
# cafe API, development
GREETING=Welcome to the cafe
APP_VERSION=1.4.0
LOG_LEVEL=info
```

```
docker run --env-file app.env -e LOG_LEVEL=debug IMAGE
```

↓

```
container environment
  GREETING=Welcome to the cafe
  APP_VERSION=1.4.0
  LOG_LEVEL=debug      ← -e wins over the file
```

rules: NAME=value · no spaces around = · everything after = is the value, quotes included · # comments

commit: app.env.example (no secrets) never commit: .env files with real passwords

COMMANDS

Form	Meaning
<code>--env-file FILE</code>	read <code>NAME=value</code> lines from FILE (repeatable: later files override earlier ones)
<code># ...</code>	a comment line
<code>NAME</code> alone on a line	copy NAME from the shell, like <code>-e NAME</code>
<code>-e NAME=value</code>	overrides the same variable from any env file

RECAP

- `--env-file FILE` passes many variables at once; `#` lines are comments.
- Values are literal: no quotes, no `$` expansion; quotes become part of the value.
- Later files override earlier ones; `-e` overrides any file.
- Keep env files without secrets in Git; never commit real passwords in them.

Full lesson with every command and its real output: [060-env-files](#)

Lesson 061 · Configuration vs secrets

Configuration (a log level, a host name, a feature flag) may be seen by anyone who operates the application. A **secret** (a password, an API key, a private key) must be seen only by the application. Environment variables are fine for configuration but leak secrets: they appear in `docker inspect`, in the environment of every process in the container, and in crash reports that dump the environment. The safer pattern is a **secret file**: mounted read-only, typically under `/run/secrets/`, and read by the application (many official images support `*_FILE` variables for exactly this). Never bake either into the image (lesson 083).

VISUAL

	configuration	secret
examples	LOG_LEVEL, DB_HOST, PORT	DB password, API token, TLS key
who may see it	operators, logs, tickets	only the application
how to pass it	-e / --env-file	a read-only file: /run/secrets/NAME
visible in inspect	yes (fine)	-e: YES X file: only the mount path ✓
in Git	yes (reviewed)	never (a secret manager, CI secrets)

```
docker run -e POSTGRES_PASSWORD=... → docker inspect shows the password to anyone with Docker access
```

```
docker run -v ./db_password.txt:/run/secrets/db_password:ro
```

```
-e POSTGRES_PASSWORD_FILE=/run/secrets/db_password → inspect shows only the file's path
```

COMMANDS

Part	What it does
<code>-v "\$(pwd)/db_password.txt:/run/secrets/db_password:ro"</code>	the secret as a read-only file inside the container
<code>-e POSTGRES_PASSWORD_FILE=/run/secrets/db_password</code>	configuration: where the secret is (not the secret itself)
<code>pg_isready -h 127.0.0.1</code>	is the server accepting TCP connections?
<code>psql -h 127.0.0.1 -U postgres</code>	log in over TCP, which requires the password

RECAP

- Configuration may be visible; secrets must not be. Treat every environment variable as visible.
- `-e` secrets leak through `docker inspect`, process environments and crash dumps.
- Pass secrets as read-only files (`/run/secrets/NAME`) and pass only their path as configuration (`*_FILE`).
- A leaked secret must be rotated, not just removed.

Full lesson with every command and its real output: [061-config-vs-secrets](#)

Module 10 · Docker Compose

Lesson 062 · Why Docker Compose?

Real applications are several containers: an API, a cache, a database, a proxy. With plain `docker` commands you create the network, start each container with the right flags, in the right order, and remove everything again by hand. **Docker Compose** describes the whole application in one file, `compose.yaml`, and manages it with one command: `docker compose up`. The file lives in Git next to the code, so every developer and every CI job starts exactly the same stack.

VISUAL

By hand: 5 commands to start, 4 to stop

```
docker network create cafe
docker build -t cafe-api .
docker run -d --name redis --network cafe redis
docker run -d --name api --network cafe \
  -p 8080:5000 -e REDIS_HOST=redis cafe-api
... and the same in reverse to clean up
```

With Compose: 1 file, 1 command

```
compose.yaml
services:
  api: build . - port 8080 - REDIS_HOST
  redis: redis:8-alpine

docker compose up -d      docker compose down
```

↓

```
network lesson-062_default └─ lesson-062-api-1      :8080
                             └─ lesson-062-redis-1
```

COMMANDS

Command	What it does
<code>compose.yaml</code>	the file Compose reads by default (also <code>docker-compose.yaml</code> , older name)
<code>docker compose up -d</code>	create the network, volumes and containers, and start them in the background
<code>--build</code>	build the images of services with <code>build:</code> first (lesson 073)
<code>docker compose ps</code>	the project's containers
<code>docker compose down</code>	stop and remove the project's containers and network (lesson 074)
project name	the folder name by default; prefixes every container, network and volume

RECAP

- Multi-container applications need networks, names, ports, variables and an order: easy to get wrong by hand.
- Compose describes the application in `compose.yaml` and manages it as one project.
- `docker compose up -d` creates everything; `docker compose down` removes it.
- Object names are prefixed with the project name (the folder name by default).

Full lesson with every command and its real output: [062-why-compose](#)

Lesson 063 · Your first Compose file

How a `compose.yaml` file is structured, how to check it before running it (`docker compose config`), and the basic cycle: `up` , `ps` , use the application, `down` . Compose files are YAML: indentation is the structure, so most mistakes are a few misplaced spaces.

VISUAL

compose.yaml

services:
 api:
 build: .
 ports: ["8080:5000"]
 environment: {REDIS_HOST: redis}
 redis:
 image: redis:8-alpine

what Compose creates (project "lesson-063")

network lesson-063_default
container lesson-063-api-1 image lesson-063-api (built)
8080 on your computer → 5000 in the container
env REDIS_HOST=redis
container lesson-063-redis-1 image redis:8-alpine

indentation = structure: two spaces per level, never tabs

COMMANDS

Command	What it does
<code>docker compose config</code>	validate the file and print the resolved configuration
<code>docker compose config --services</code>	only the service names
<code>docker compose up -d</code>	create and start everything in the background
<code>--build --quiet-build</code>	build the images first, without printing the build log
<code>docker compose ps</code>	the project's containers, with <code>--format</code> like <code>docker ps</code>
<code>docker compose down</code>	stop and remove the containers and the network
<code>-f FILE</code>	use another file than <code>compose.yaml</code>

RECAP

- `compose.yaml` lists services; each one uses an `image:` or a `build:` , plus ports, environment and more.
- Indentation is structure: keys of one mapping line up; never use tabs.
- `docker compose config` validates and shows the resolved file; use it after every edit.
- `up -d` → `ps` → use → `down` .

Full lesson with every command and its real output: [063-first-compose-file](#)

Lesson 064 · Services

A **service** is one component of the application: Compose runs one or more containers for it, all from the same configuration. This lesson covers the keys every service uses: where the image comes from (`image:` or `build:`), the command, the ports, the restart policy, and how many copies run (`--scale`).

VISUAL



Key	Meaning
<code>image:</code>	run an existing image (pulled if missing)
<code>build:</code>	build the image from a Dockerfile (lesson 073)
<code>command:</code>	replace the image's default command (<code>CMD</code>)
<code>ports:</code>	publish <code>HOST:CONTAINER</code> ports
<code>restart:</code>	<code>no</code> (default), <code>on-failure</code> , <code>unless-stopped</code> , <code>always</code>

COMMANDS

Command	What it does
<code>docker compose config --services</code>	the services in the file
<code>docker compose ps --format '{{.Name}} ...'</code>	one line per container, with the fields you choose
<code>docker compose exec api sh -c 'kill 1'</code>	stop process 1 of the container (its main process); <code>kill</code> is a shell command
<code>{{.RestartCount}}</code>	how many times Docker restarted the container
<code>docker compose up -d --scale api=2</code>	run two containers for <code>api</code>

RECAP

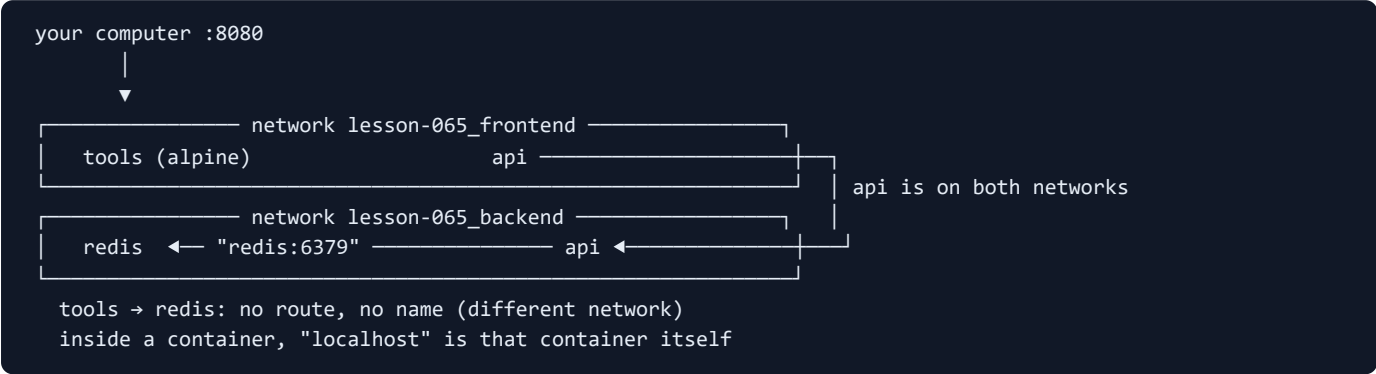
- A service = configuration for one or more identical containers.
- `image:` uses an image, `build:` builds one; `command:` replaces the default command.
- `restart: unless-stopped` restarts crashed containers, not ones you stopped.
- Replicas cannot share a fixed host port: use a range, or a proxy in front.

Full lesson with every command and its real output: [064-compose-services](#)

Lesson 065 · Networking in Compose

Compose creates a network for every project (`PROJECT_default`) and attaches every service to it. On that network, each **service name is a DNS name**: the API reaches Redis at `redis` , not at an IP address. You can also declare your own networks and attach each service only where it belongs, so a front-end component cannot reach the database at all.

VISUAL



COMMANDS

Key / command	What it does
<code>networks:</code> (top level)	declare networks; Compose names them <code>PROJECT_NAME</code>
<code>networks:</code> [a, b] (service)	attach the service to these networks (instead of default)
service name	the DNS name of the service on every network it is attached to
<code>docker network inspect NET --format '{{range .Containers}}...</code>	the containers on a network
<code>docker compose exec SERVICE CMD</code>	run a command in the service's container (lesson 072)

RECAP

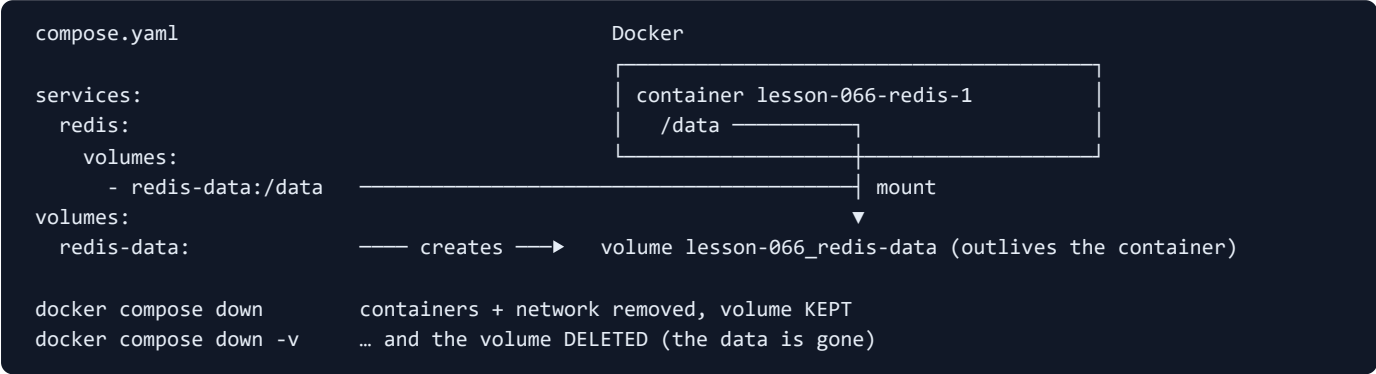
- Compose creates `PROJECT_default` and attaches every service; custom `networks:` give finer control.
- Service names are DNS names on every network the service joins.
- Services talk on container ports; published ports are only for your computer.
- `localhost` inside a container is the container itself.

Full lesson with every command and its real output: [065-compose-networking](#)

Lesson 066 · Volumes in Compose

A container's writable layer disappears with the container (lesson 053). Data that must survive (a database, a cache that must not start empty, uploads) belongs in a **volume**. In Compose you declare named volumes at the top level and mount them into services; Compose creates them once and keeps them across `down` and `up`, until you explicitly delete them with `down -v`.

VISUAL



COMMANDS

Key / command	What it does
volumes: (top level)	declare named volumes; Compose names them <code>PROJECT_NAME</code>
- redis-data:/data (service)	mount the named volume at <code>/data</code> in the container
- ./src:/app/src (service)	a bind mount: a folder of your computer (lesson 054)
docker compose down	remove containers and networks; volumes stay
docker compose down -v	also remove the project's named volumes

RECAP

- Named volumes are declared at the top level and mounted per service (`name:/path`).
- They outlive containers: `down` keeps them, `down -v` deletes them.
- A name in a service's `volumes:` means a named volume, a path (`./dir`) means a bind mount.
- `docker inspect --format '{{range .Mounts}}...'` shows what is mounted where.

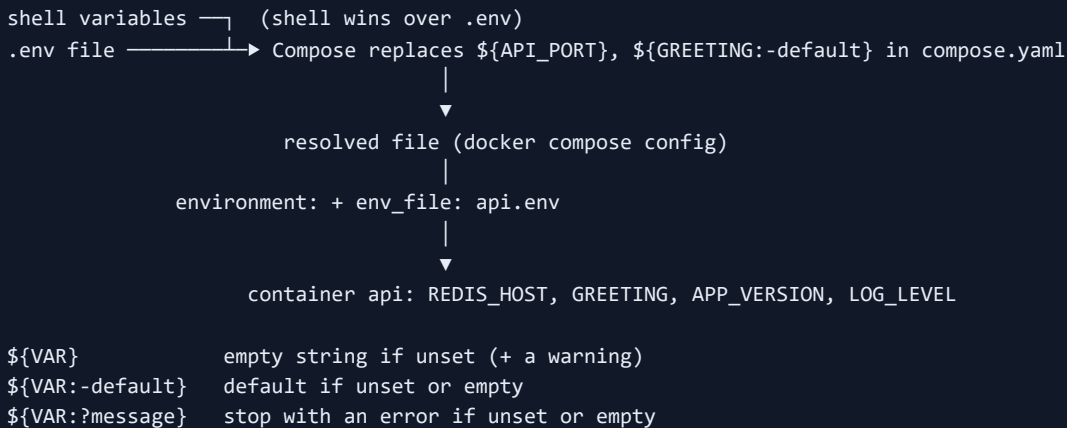
Full lesson with every command and its real output: [066-compose-volumes](#)

Lesson 067 · Environment variables in Compose

Compose uses environment variables in two different places, and mixing them up is a classic source of confusion:

1. **Interpolation:** `${VAR}` inside `compose.yaml` is replaced by Compose itself, from your shell or from a `.env` file next to `compose.yaml`. It configures the file (ports, image tags, defaults).
2. **Container environment:** `environment:` and `env_file:` set variables **inside** the container, for the application.

VISUAL



COMMANDS

Syntax / command	Meaning
<code>\${VAR}</code> , <code>\${VAR:-default}</code> , <code>\${VAR:?error}</code>	interpolation in <code>compose.yaml</code>
<code>.env</code> (next to <code>compose.yaml</code>)	values for interpolation; not passed to containers
<code>environment:</code>	variables inside the container
<code>env_file: [file]</code>	variables inside the container, from a file
<code>docker compose config</code>	the file after interpolation: check values here first
<code>--env-file FILE</code>	use another file instead of <code>.env</code> (for example <code>.env.staging</code>)

RECAP

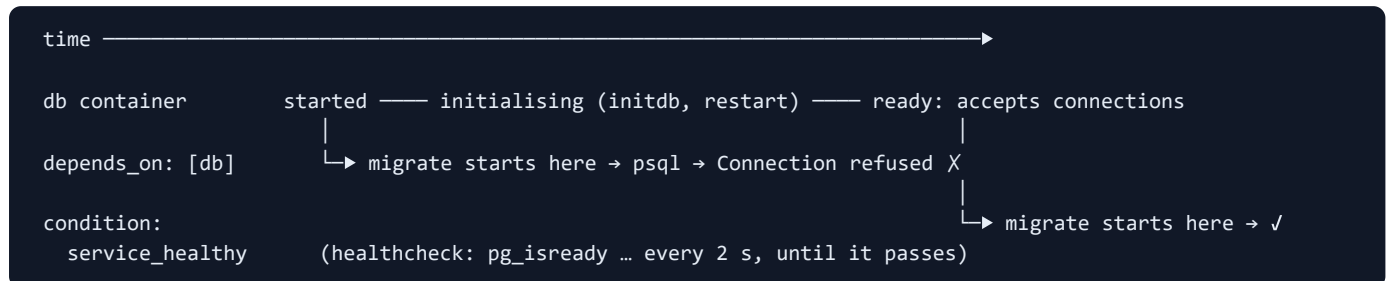
- `${VAR}` in `compose.yaml` is interpolation: from the shell, then `.env`. `docker compose config` shows the result.
- `environment:` and `env_file:` set variables inside containers.
- An unset `${VAR}` is only a warning: use `${VAR:-default}` or `${VAR:?message}`.
- Commit `.env.example`, never `.env`.

Full lesson with every command and its real output: [067-compose-environment](#)

Lesson 068 · depends_on vs readiness

`depends_on: [db]` only controls the **start order**: Compose starts `db`'s container first, then the dependent service. It does not wait until the database inside is **ready** to accept connections. A database needs seconds to initialise; an application that connects immediately fails. The fix is to define what "ready" means (a **healthcheck**) and wait for it: `condition: service_healthy`.

VISUAL



condition:	Waits until the dependency ...
<code>service_started</code> (what <code>depends_on: [db]</code> means)	has a running container
<code>service_healthy</code>	reports <code>healthy</code> (its healthcheck passes)
<code>service_completed_successfully</code>	has exited with status 0 (for one-off jobs)

COMMANDS

Command / key	What it does
<code>depends_on: [db]</code>	start <code>db</code> before this service (order only)
<code>depends_on: {db: {condition: service_healthy}}</code>	wait until <code>db</code> is healthy
<code>healthcheck: test: [...]</code>	the command Docker runs to decide whether the service is healthy (lesson 069)
<code>pg_isready -h 127.0.0.1</code>	PostgreSQL's own readiness check (exit 0 = accepting connections)
<code>docker compose up</code> (without <code>-d</code>)	run in the foreground, showing every service's output

RECAP

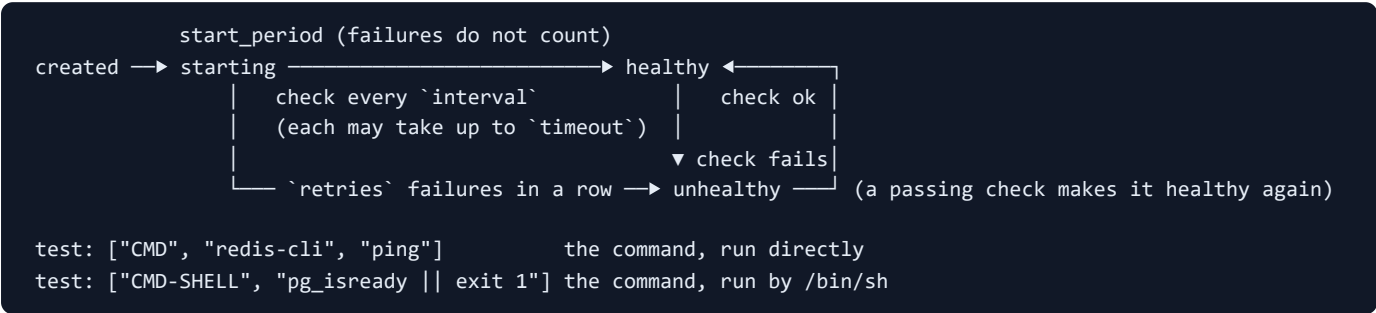
- `depends_on` = start order. It does not wait for readiness.
- Readiness needs a definition: a healthcheck (`pg_isready` , an HTTP `/health` , `redis-cli ping`).
- `condition: service_healthy` waits for the healthcheck; `service_completed_successfully` waits for a job.
- Applications should still retry connections: dependencies restart in production too.

Full lesson with every command and its real output: [068-depends-on-vs-readiness](#)

Lesson 069 · Healthchecks in Compose

A **healthcheck** is a command Docker runs inside the container at a regular interval. Exit code 0 means healthy, any other code means a failed check; after a number of failures in a row, the container is **unhealthy**. Compose uses health for `depends_on` conditions (lesson 068) and for `docker compose up --wait`; you see it in `docker compose ps`. A process that is running but not answering is exactly what a healthcheck catches (lesson 093).

VISUAL



COMMANDS

Key / command	Meaning
healthcheck: test:	the check; <code>CMD</code> runs it directly, <code>CMD-SHELL</code> through <code>/bin/sh -c</code>
interval, timeout, retries, start_period	how often, how long each check may take, failures before unhealthy, grace at start
docker compose up --wait	return when all services are running and healthy; fail otherwise
--wait-timeout N	give up after N seconds
{{.State.Health.Status}}, {{.State.Health.Log}}	the health status and the last checks (with their output)

RECAP

- A healthcheck is a command run inside the container; exit 0 = check passed.
- `retries` failures in a row → `unhealthy`; `start_period` gives the service time to start.
- `docker compose up --wait` and `condition: service_healthy` use health.
- Check the check: it runs inside the container, on container ports, and must fail when the service is unusable.

Full lesson with every command and its real output: [069-compose-healthchecks](#)

Lesson 070 · Profiles

Some services are not needed every time: a debugging shell, an admin UI, a job that loads test data, a load generator. **Profiles** keep them in the same `compose.yaml` but start them only on request. A service without `profiles:` always starts; a service with `profiles: [debug]` starts only when the `debug` profile is active.

VISUAL

```
compose.yaml      docker compose up      --profile seed      --profile debug
api               ✓                      ✓                  ✓
redis            ✓                      ✓                  ✓
seed  profiles: [seed]  ·                ✓ (runs once)  ·
tools  profiles: [debug] ·                ·                ✓

activate:  --profile NAME    (several: --profile a --profile b)
          COMPOSE_PROFILES=a,b  (environment variable or .env)
```

COMMANDS

Command / key	What it does
<code>profiles: [name]</code>	the service belongs to that profile and is skipped by default
<code>docker compose --profile NAME up</code>	also start the services of that profile
<code>COMPOSE_PROFILES=a,b</code>	the same, as an environment variable (or in <code>.env</code>)
<code>docker compose --profile "*" ...</code>	all profiles
<code>docker compose config --profiles</code>	the profiles defined in the file
<code>docker compose run --rm SERVICE</code>	run one service, even one with a profile (lesson 072)

RECAP

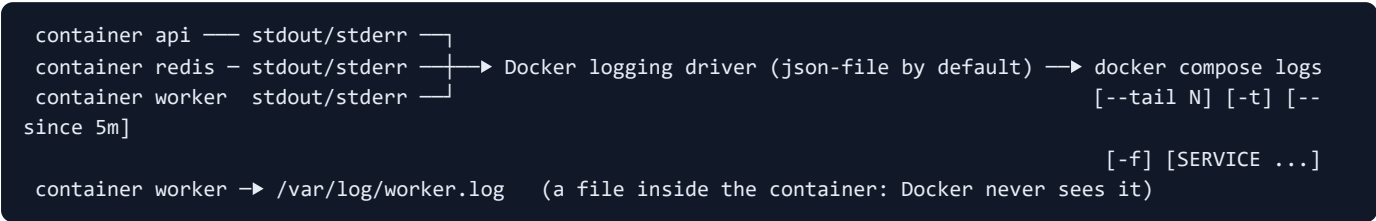
- `profiles: [name]` makes a service optional; services without profiles always start.
- Activate with `--profile name` or `COMPOSE_PROFILES` .
- An always-on service cannot depend on an inactive one (or use `required: false`).
- `docker compose run --rm SERVICE` runs a one-off job, profile or not.

Full lesson with every command and its real output: [070-compose-profiles](#)

Lesson 071 · Logs

Docker collects everything a container's main process writes to **standard output and standard error**. `docker compose logs` shows that output for every service of the project, interleaved and prefixed with the container name, or for one service. The rule that makes it work: containers log to stdout/stderr, not to files inside the container.

VISUAL



COMMANDS

Command / flag	What it does
<code>docker compose logs</code>	output of all services, interleaved
<code>docker compose logs SERVICE ...</code>	only these services
<code>--tail N</code>	the last N lines per container
<code>-t</code> , <code>--timestamps</code>	prefix each line with the time Docker received it
<code>--since 10m</code> , <code>--until ...</code>	a time window (<code>10s</code> , <code>5m</code> , <code>2h</code> or a date)
<code>-f</code> , <code>--follow</code>	keep streaming new lines
<code>--no-log-prefix</code>	without the <code>service-1 </code> prefix

RECAP

- Docker captures a container's stdout and stderr; `docker compose logs` shows them per project or per service.
- `--tail` , `-t` , `--since` , `-f` , `--no-log-prefix` narrow and shape the output.
- Log files inside the container are invisible to Docker: log to stdout (or link the file to `/dev/stdout`).

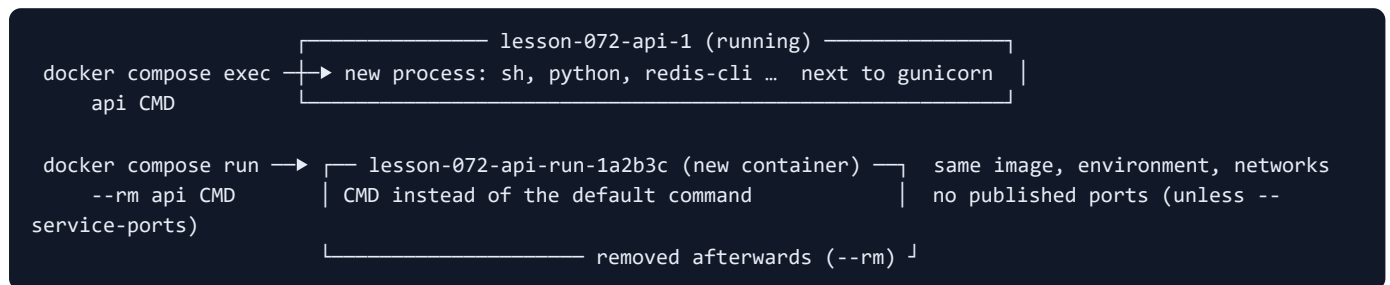
Full lesson with every command and its real output: [071-compose-logs](#)

Lesson 072 · exec and run

Two ways to run a command for a service, which are easy to confuse:

- `docker compose exec SERVICE CMD` runs the command **in the running container** of that service: same processes, same files, same memory. For investigating and administering what is running.
- `docker compose run SERVICE CMD` creates a **new, separate container** from the service's configuration and runs the command there. For one-off jobs: migrations, tests, scripts.

VISUAL



COMMANDS

Command	What it does
<code>docker compose exec SERVICE CMD</code>	run CMD in the service's running container
<code>exec -T</code>	no pseudo-terminal: for scripts and pipes (... <code>exec -T db psql < dump.sql</code>)
<code>exec -u USER</code> , <code>exec -w DIR</code> , <code>exec -e VAR=x</code>	as another user, in another directory, with an extra variable
<code>docker compose run --rm SERVICE CMD</code>	a new one-off container of the service, removed afterwards
<code>run --no-deps</code>	do not start the service's dependencies
<code>run --service-ports</code>	publish the service's ports (off by default for <code>run</code>)

RECAP

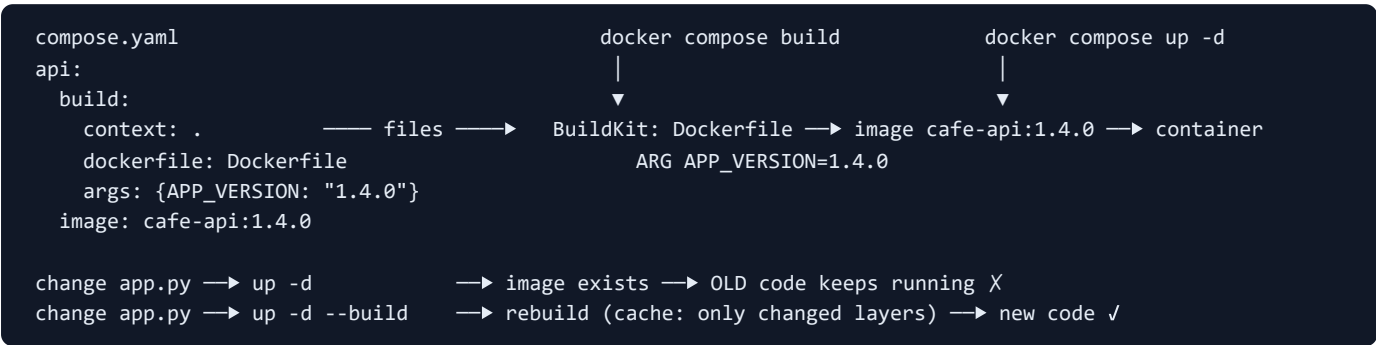
- `exec` : a new process in the running container (needs it running).
- `run` : a new container from the service's configuration (use `--rm`).
- `exec -T` for scripts and pipes; `run --no-deps` , `--service-ports` as needed.
- A stopped or crashed service: read its logs, then `start` it or investigate with `run` .

Full lesson with every command and its real output: [072-compose-exec](#)

Lesson 073 · Building images with Compose

A service with `build:` gets its image from a Dockerfile. Compose builds it when the image does not exist yet, and only then: after you change the code, `docker compose up` keeps running the **old** image until you rebuild it (`docker compose build`, or `up --build`). The `build:` section also sets the context, the Dockerfile, build arguments and, with `image:`, the name of the result.

VISUAL



COMMANDS

Key / command	What it does
<code>build: .</code>	short form: the context folder, <code>Dockerfile</code> inside it
<code>build: {context, dockerfile, args, target}</code>	long form; <code>target</code> picks a stage of a multi-stage build (lesson 087)
<code>image:</code> next to <code>build:</code>	the name of the built image (and what <code>push</code> uploads)
<code>docker compose build [SERVICE]</code>	build (or rebuild) the images
<code>docker compose up -d --build</code>	rebuild, then recreate containers whose image changed
<code>docker compose build --no-cache</code>	ignore the build cache (lesson 039)
<code>docker compose images</code>	the images used by the project's containers

RECAP

- `build:` builds an image for a service; `image:` names it.
- Compose builds only when the image is missing: after code changes, use `up --build` or `build`.
- `args:` pass build arguments (`ARG`); `target:` selects a stage.
- The build cache keeps rebuilds fast when the Dockerfile is ordered well.

Full lesson with every command and its real output: [073-compose-build](#)

Lesson 074 · up, down and the project lifecycle

`up` is not only "start": it **converges** the running project to the file. It creates what is missing, recreates containers whose configuration changed, and leaves the rest alone. `stop / start` pause and resume containers without removing them; `down` removes them. What `down` removes can be widened: volumes (`-v`), images (`--rmi`), and containers of services that are no longer in the file (`--remove-orphans`).

VISUAL



COMMANDS

Command	What it does
<code>docker compose up -d</code>	create missing, recreate changed, start stopped; leave the rest
<code>up -d --force-recreate</code>	recreate every container, changed or not
<code>up -d SERVICE</code>	only this service (and its dependencies)
<code>docker compose stop / start / restart</code>	stop, start, restart without removing
<code>docker compose down</code>	remove containers and networks
<code>down -v / down --rmi local</code>	also the named volumes / the images the project built
<code>down --remove-orphans , up --remove-orphans</code>	also containers of services no longer in the file

RECAP

- `up -d` converges to the file: creates, recreates on change, starts; unchanged containers stay.
- `stop / start` keep containers; `down` removes them (`-v` volumes, `--rmi local` built images).
- Removed services leave orphans: `--remove-orphans` .
- `--force-recreate` and `--no-deps` give finer control.

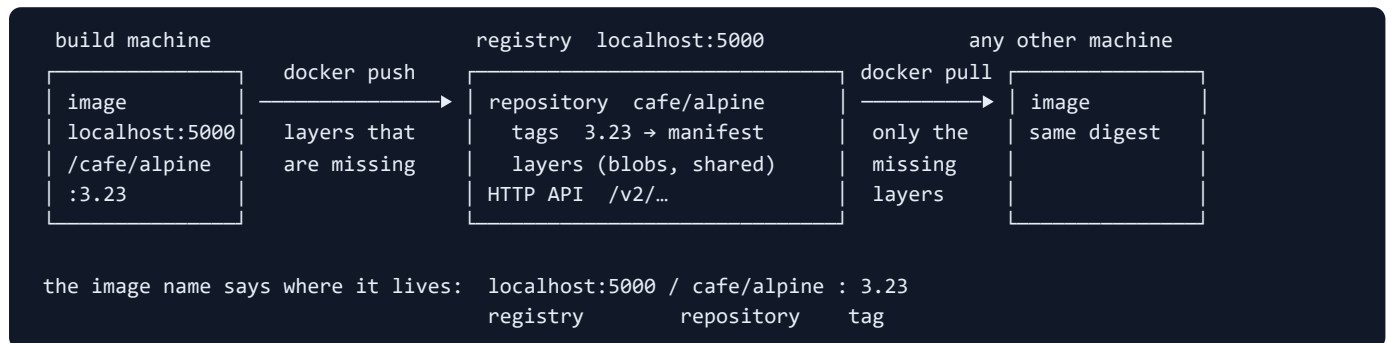
Full lesson with every command and its real output: [074-compose-up-down](#)

Module 11 · Registries

Lesson 075 · What is a registry?

A **registry** is a server that stores images and hands them out: `docker push` uploads an image, `docker pull` downloads it. It is how an image built on one machine (a laptop, a CI runner) reaches every other machine (servers, Kubernetes nodes). Docker Hub is the default registry; GitHub, GitLab and every cloud run their own; and you can run one yourself, with the official `registry` image. This lesson does exactly that, on your computer.

VISUAL



COMMANDS

Command	What it does
<code>docker run -d -p 5000:5000 registry:3</code>	a private registry (the CNCF Distribution project)
<code>docker tag SOURCE TARGET</code>	give an image another name (no copy)
<code>docker push NAME:TAG</code>	upload the image to the registry in its name
<code>docker pull NAME:TAG</code>	download it
<code>GET /v2/_catalog</code>	the repositories of the registry
<code>GET /v2/REPO/tags/list</code>	the tags of a repository

RECAP

- A registry stores images as repositories with tags; push uploads, pull downloads (only missing layers).
- The image's name decides the registry: `REGISTRY/REPOSITORY:TAG`.
- `registry:3` runs a private registry; its HTTP API (`/v2/...`) shows repositories and tags.
- `manifest unknown` = the registry works, the tag does not exist.

Full lesson with every command and its real output: [075-what-is-a-registry](#)

Lesson 076 · Docker Hub

Docker Hub (`docker.io`) is Docker's default registry: every image name without a registry address comes from there. It hosts **official images** (`nginx`, `python`, `postgres`, maintained with Docker), images of verified publishers, and every user's and organisation's own repositories. To push, you log in with an **access token** (not your password); anonymous pulls are rate-limited.

VISUAL

what you type	what Docker means
<code>nginx:1.30-alpine</code>	<code>docker.io / library / nginx : 1.30-alpine</code> official image
<code>bitnami/redis:8</code>	<code>docker.io / bitnami / redis : 8</code> a publisher's namespace
<code>alice/cafe-api:1.0</code>	<code>docker.io / alice / cafe-api : 1.0</code> a user's repository

push: `docker login (user + access token) → docker push alice/cafe-api:1.0`
pull: anonymous (rate-limited per IP) or logged in (higher limit, private repositories)

COMMANDS

Command	What it does
<code>docker login [-u USER]</code>	log in to Docker Hub (or <code>docker login REGISTRY</code> for another registry)
<code>--password-stdin</code>	read the token from standard input, for scripts and CI
<code>docker logout</code>	remove the stored credentials
<code>{ {.RepoDigests} }</code>	the registry digests of a local image
<code>IMAGE@sha256:...</code>	refer to an image by digest instead of tag

RECAP

- A name without a registry is a Docker Hub name: `nginx` = `docker.io/library/nginx`.
- Log in with an access token (`--password-stdin` in scripts); push only to a namespace you own.
- Tags move, digests do not: pin by digest for reproducibility.
- Anonymous pulls are rate-limited: log in or mirror in CI.

Full lesson with every command and its real output: [076-docker-hub](#)

Lesson 077 · Image naming

An image name is an address: **registry / namespace / repository : tag**, optionally **@digest**. Every part except the repository has a default, which is why `nginx` works. Knowing the parts tells you where `docker push` will send an image, and good tags tell everyone which version it is. One image can have many names; `docker tag` adds one, without copying anything.

VISUAL

```
ghcr.io / cafe-team / cafe-api : 1.4.2 @sha256:3f1a...
```

registry (default docker.io)	namespace (default for Docker Hub: library)	repository (required)	tag (default latest)	digest (content address, immutable)
------------------------------------	--	--------------------------	----------------------------	-------------------------------------

rules: lowercase letters, digits, separators . _ - in names; a tag is up to 128 of [A-Za-z0-9_-.]
one image — many names: `cafe-api:1.4.2` `cafe-api:1.4` `cafe-api:1` `cafe-api:sha-3f1a2b4`

COMMANDS

Command	What it does
<code>docker tag SOURCE[:TAG] TARGET[:TAG]</code>	add a name to an existing image
<code>docker push --all-tags REPOSITORY</code>	push every local tag of that repository
<code>docker image rm NAME:TAG</code>	remove one name; the image goes only when its last name is removed
<code>IMAGE@sha256:...</code>	refer to exact content, whatever the tags say

RECAP

- `registry/namespace/repository:tag@digest`; defaults: `docker.io`, `library`, `latest`.
- Repository names are lowercase; tags allow `[A-Za-z0-9_-.]`, up to 128 characters.
- `docker tag` adds a name; one image can have many; `rm` of a name only untags.
- Tag releases with full versions and commits; pin deployments to versions or digests.

Full lesson with every command and its real output: [077-image-naming](#)

Lesson 078 · Private registries

Company images belong in a **private registry**: only people and machines with credentials may push or pull. It can be a managed service (Amazon ECR, Azure Container Registry, Google Artifact Registry, GitHub, GitLab, Harbor, Nexus ...) or the `registry` image you already know, with authentication switched on. Whatever the product, the client side is the same: `docker login REGISTRY`, then push and pull with names that start with that registry.

VISUAL

```
docker login localhost:5001 — user + password/token —> registry checks htpasswd —> "Login Succeeded"
|
| — stores the credential (credential store, or base64 in ~/.docker/config.json)
|
docker push localhost:5001/cafe/api:1.0
|
| — sends the stored credential with every request
|
| — registry:3 (REGISTRY_AUTH=htpasswd) —
| — no / wrong credential —> 401 unauthorized
| — valid credential —> push / pull
```

COMMANDS

Command / setting	What it does
<code>htpasswd -Bbn USER PASSWORD</code>	print a user line with a bcrypt hash (<code>-B</code>), from the command line (<code>-b</code>), to stdout (<code>-n</code>)
<code>REGISTRY_AUTH=htpasswd</code> , <code>..._PATH</code> , <code>..._REALM</code>	switch on basic authentication in <code>registry:3</code>
<code>docker login REGISTRY -u USER --password-stdin</code>	log in to that registry
<code>docker logout REGISTRY</code>	remove its stored credential
<code>credsStore</code> / <code>credHelpers</code> in <code>config.json</code>	keep credentials in a keychain, or get them from a cloud CLI

RECAP

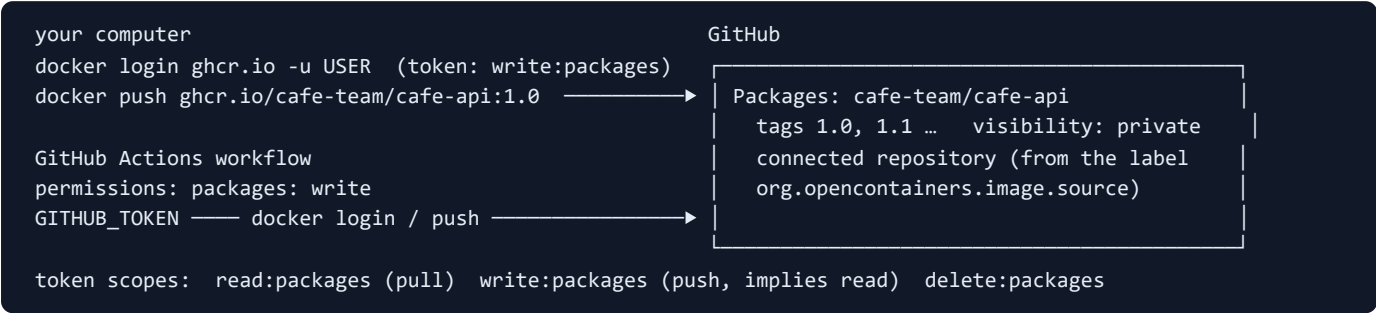
- A private registry requires credentials: `docker login REGISTRY`, then push/pull by its name.
- `registry:3` supports `htpasswd` authentication; managed registries use tokens from their CLI.
- `no basic auth credentials` = not logged in; `401 Unauthorized` = rejected; `denied` = no permission.
- Use `--password-stdin` and a credential helper; base64 in `config.json` is not protection.

Full lesson with every command and its real output: [078-private-registries](#)

Lesson 079 · GitHub Container Registry

GitHub Container Registry (GHCR, `ghcr.io`) stores images next to the code: `ghcr.io/OWNER/IMAGE:TAG`, where `OWNER` is a GitHub user or organisation. Access follows GitHub's permissions. From your computer you log in with a personal access token that has the `write:packages` scope; in GitHub Actions the job's built-in `GITHUB_TOKEN` is enough, once the workflow grants it `packages: write`. A label in the image links the package to its repository.

VISUAL



COMMANDS

Item	Meaning
<code>ghcr.io/OWNER/IMAGE:TAG</code>	the GHCR name; <code>OWNER</code> = user or organisation, lowercase
<code>write:packages</code> / <code>read:packages</code> / <code>delete:packages</code>	token scopes for pushing / pulling / deleting
<code>gh auth refresh -s write:packages</code>	add the scope to the GitHub CLI's token
<code>permissions: packages: write</code>	lets a workflow's <code>GITHUB_TOKEN</code> push packages
<code>LABEL org.opencontainers.image.source=URL</code>	connect the package to a repository

RECAP

- GHCR names: `ghcr.io/owner/image:tag`, lowercase.
- From your computer: a token with `write:packages` (`gh auth refresh -s write:packages`), `--password-stdin`.
- From Actions: `GITHUB_TOKEN` with `permissions: packages: write`.
- `org.opencontainers.image.source` connects the package to its repository; new packages are private.

Full lesson with every command and its real output: [079-github-container-registry](#)

Module 12 · Container security

Lesson 080 · The attack surface of a container

The **attack surface** is everything an attacker could use once they get code running in your container: a shell, a package manager, network tools, setuid programs, the root user, Linux capabilities, a writable file system, access to host devices. Each lesson of this module removes one part of it. This lesson shows you how to **see** the attack surface, and the most dangerous single flag: `--privileged`.

VISUAL

What an attacker finds inside the container	How this module shrinks it
shell, package manager, curl, perl (082)	minimal or distroless base image
the root user (081)	USER appuser
secrets in ENV, ARG, image layers (083)	build secrets, runtime secrets
writable file system (084)	--read-only + --tmpfs
Linux capabilities (chown, kill, ...) (085)	--cap-drop ALL, add back only what is needed
unlimited CPU, memory, processes (086)	--memory, --cpus, --pids-limit
host devices, all capabilities (080)	never --privileged

———— host kernel (shared by every container) ————

each item above is a step from "code runs in the container" towards "code controls the host"

COMMANDS

Command	What it shows
<code>ls /bin /sbin /usr/bin /usr/sbin \ wc -l</code>	how many programs are installed
<code>find / -xdev -perm -4000 -type f</code>	setuid programs (they run with the rights of their owner)
<code>gcr.io/distroless/static-debian12:nonroot</code>	an image with no shell or package manager, running as a non-root user
<code>--privileged</code>	gives the container every capability and every host device: almost the host itself

RECAP

- The attack surface is every tool, permission and resource available inside the container.
- Fewer programs, no setuid binaries, no shell (distroless) = less for an attacker to use.
- `--privileged` gives the container all capabilities and all host devices: never use it to "make something work".
- Find privileged containers with `docker inspect --format '{{.HostConfig.Privileged}}'`.

Full lesson with every command and its real output: [080-attack-surface](#)

Lesson 081 · Running as a non-root user

Unless the image says otherwise, the process in a container runs as **root** (user ID 0). It is the same user ID 0 that the host kernel knows: if the process breaks out of the container, through a kernel bug or a careless bind mount, it is root on the host. The `USER` instruction makes the application run as an ordinary user. The usual problem that follows is file permissions, and this lesson fixes it properly.

VISUAL

Default: root in the container

container python app.py uid 0 (root)
| escape / bind mount
▼
host uid 0 = root: can change anything

Files in the image: /app/app.py root:root
/app/data/ app:app

With USER app

container python app.py uid 100 (app)
| escape / bind mount
▼
host uid 100: an unprivileged user

read-only for app (the code cannot be changed)
writable for app (only what the app must write)

COMMANDS

Instruction / command	What it does
<code>RUN addgroup -S app && adduser -S -G app -H app</code>	create a system group and user <code>app</code> (Alpine; Debian: <code>groupadd -r / useradd -r</code>)
<code>USER app</code>	every later <code>RUN</code> , and the container's command, run as <code>app</code>
<code>RUN mkdir data && chown app:app data</code>	give the user only the folder it must write to
<code>COPY --chown=app:app SRC DEST</code>	copy files owned by a user (only when the app must change them)
<code>docker run --user 1000:1000</code>	override the user at start time
<code>docker top NAME -o user,pid,args</code>	the container's processes, with their users, as the host sees them

RECAP

- Containers run as root (uid 0) by default, the same uid 0 as on the host kernel.
- `USER` makes the application run as an unprivileged user.
- "Permission denied" after adding `USER` means the user has nowhere to write: `chown` only the data folder.
- Keep the code owned by root, so a compromised application cannot rewrite itself.

Full lesson with every command and its real output: [081-non-root-users](#)

Lesson 082 · Image security: trusted, minimal, pinned, scanned

Your image is only as secure as everything inside it, and most of what is inside comes from the base image. Four habits keep it under control:

- 1. **Trusted:** start from official images or verified publishers, never a random repository.
- 2. **Minimal:** the fewer packages, the fewer vulnerabilities and tools for an attacker (`-slim` , `alpine` , `distroless`).
- 3. **Pinned:** a tag can be moved to a different image; a **digest** (`@sha256:...`) cannot.
- 4. **Scanned:** a scanner compares the packages in the image with databases of known vulnerabilities (CVEs).

VISUAL

Trusted	Minimal		Pinned	Scanned
IMAGE	python:3.14	big	python:3.14-slim	docker scout cves
Docker Official Image	python:3.14-slim		(a tag: can move)	trivy image IMAGE
Verified Publisher	python:3.14-alpine		▼	
your own registry	distroless	small	python@sha256:c3e5...	▼
	fewer packages =		(a digest: always the	CVEs per package
X someuser/python-fast	fewer CVEs and tools		same bytes)	→ update the base

COMMANDS

Command / syntax	What it does
<code>docker image inspect --format '{{.Size}}' IMG</code>	the image's size in bytes
<code>docker export CONTAINER \ tar -t</code>	list every file of a container, even without a shell inside
<code>apk list --installed / dpkg -l</code>	the packages and versions a scanner checks
<code>docker image inspect --format '{{index .RepoDigests 0}}'</code>	the image's digest reference <code>repo@sha256:...</code>
<code>IMAGE@sha256:DIGEST</code>	refer to an image by content, not by tag
<code>docker scout cves IMG / trivy image IMG</code>	list known vulnerabilities

RECAP

- Use official or verified base images; prefer the smallest variant that works (`-slim` , `alpine` , `distroless`).
- A tag can move; a digest (`@sha256: + 64 hex characters`) always means the same bytes.
- Scanners compare the installed package versions against CVE databases; rebuild on fixed base images regularly.
- `docker export` inspects images that have no shell.

Full lesson with every command and its real output: [082-image-security](#)

Lesson 083 · Secrets in images

Anyone who can pull an image can read **everything** in it: every layer, the build history and the metadata. A secret that reached the image in any of these ways is leaked, even if a later step deletes it:

- ARG TOKEN used in a RUN step → stored in the **history**,
- ENV PASSWORD=... → stored in the image **configuration**,
- COPY credentials ... then RUN rm ... → still in the **layer** that the COPY created.

The fix: **build secrets** (RUN --mount=type=secret) for secrets needed during the build, and **runtime** configuration for secrets the application needs (lessons 060, 061).

VISUAL

Leaky Dockerfile

ARG API_TOKEN
ENV DB_PASSWORD=...
COPY credentials.txt /tmp/
RUN rm /tmp/credentials.txt

Where the secret ends up (forever, in every copy)

docker history

docker inspect

layer 3

layer 4

"RUN |1 API_TOKEN=example-token..."

Config.Env

tmp/credentials.txt ← still here

"file deleted" marker only

Fixed Dockerfile

RUN --mount=type=secret,id=api_token ...
docker build --secret id=api_token,src=api_token.txt

/run/secrets/api_token exists during this step only:

nothing in layers, history or metadata

COMMANDS

Command / instruction	What it does
docker history --no-trunc IMG	the instruction that created every layer, in full
docker image inspect --format '{{.Config.Env}}'	the environment variables baked into the image
docker save IMG -o FILE.tar	the image as an archive: every layer as a tar file
RUN --mount=type=secret,id=NAME	mount a secret at /run/secrets/NAME for this one step
docker build --secret id=NAME,src=FILE	provide that secret from a file on the build machine

RECAP

- Anything in an image's layers, history or configuration is readable by everyone who can pull it.
- ARG values used in RUN appear in docker history; ENV values appear in docker inspect .
- Deleting a file in a later layer does not remove it from the earlier layer.
- Build-time secrets: RUN --mount=type=secret + docker build --secret . Run-time secrets: given at start, never in the image. A leaked secret must be rotated.

Full lesson with every command and its real output: [083-secrets-in-images](#)

Lesson 084 · Read-only file systems

By default every container has a **writable layer**: a process can change any file it has permission to, including the application's own code, configuration and binaries. An attacker who gets in can therefore install tools or plant a backdoor that survives until the container is removed. `--read-only` makes the container's whole file system read-only. The few places the application really writes to become **tmpfs** mounts (in memory, empty at every start) or volumes.

VISUAL

Default

writable layer (anything)

image layers (read-only)

attacker can change code, add tools

`--read-only --tmpfs /var/cache/nginx --tmpfs /run`

`/var/cache/nginx`

tmpfs (RAM)

`/run`

tmpfs (RAM)

everything else

READ-ONLY

← writable, empty at start

← writable, empty at start

← code, config, binaries

attacker cannot change the application

COMMANDS

Flag	What it does
<code>--read-only</code>	mount the container's root file system read-only
<code>--tmpfs /path</code>	an empty, writable, in-memory folder at <code>/path</code> (lost when the container stops)
<code>--tmpfs /path:size=16m</code>	the same, limited to 16 MB
<code>-v NAME:/path</code>	a writable volume, for data that must be kept (lesson 055)
<code>docker diff NAME</code>	the files a container changed in its writable layer (A added, C changed, D deleted)

RECAP

- `--read-only` prevents any change to the container's file system: code, configuration and binaries stay intact.
- Find what an application writes with `docker diff` ; give it `--tmpfs` (scratch) or volumes (data).
- `Read-only file system` errors name the exact path that needs to be writable.

Full lesson with every command and its real output: [084-read-only-fileystems](#)

Lesson 085 · Linux capabilities

Linux splits the power of root into about 40 **capabilities**: `CHOWN` (change file owners), `NET_BIND_SERVICE` (listen on ports below 1024), `SYS_ADMIN` (mount file systems and much more), `SYS_TIME` (set the clock) and so on. Docker gives containers a reduced default set of 14. Most applications need none of them. The secure pattern is **drop all, add back only what is needed**:

```
docker run --cap-drop ALL --cap-add CHOWN ...
```

VISUAL

Root on the host
all ~40 capabilities

`SYS_ADMIN SYS_TIME
SYS_MODULE NET_ADMIN
CHOWN SETUID KILL ...`

Container (Docker default)
14 capabilities

`CHOWN DAC_OVERRIDE FOWNER
FSETID KILL SETGID SETUID
SETPCAP NET_BIND_SERVICE
NET_RAW SYS_CHROOT MKNOD
AUDIT_WRITE SETFCAP`

`--cap-drop ALL --cap-add CHOWN ...`
only the ones the app needs

`CHOWN SETUID SETGID`

`--privileged` = all of them
(never, lesson 080)

COMMANDS

Flag	What it does
<code>--cap-drop ALL</code>	remove every capability
<code>--cap-add NAME</code>	add one capability (without the <code>CAP_</code> prefix), e.g. <code>NET_BIND_SERVICE</code>
<code>--cap-drop NAME</code>	remove one capability from the default set
<code>--security-opt no-new-privileges</code>	setuid programs cannot gain more privileges than the process already has
<code>docker inspect --format '{{.HostConfig.CapAdd}} {{.HostConfig.CapDrop}}'</code>	the capability changes of a container

RECAP

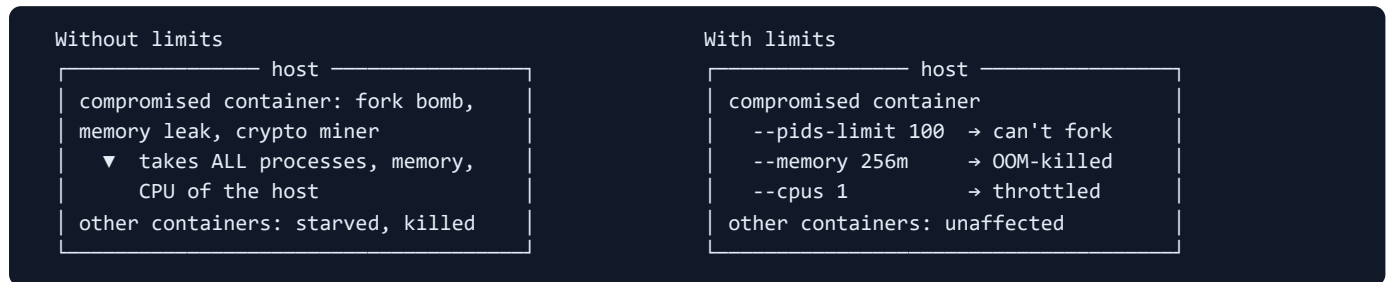
- Capabilities split root's power into pieces; containers get 14 of about 40 by default.
- `--cap-drop ALL`, then `--cap-add` only what the application proves it needs (read its `Operation not permitted` errors).
- `--security-opt no-new-privileges` stops setuid programs from gaining privileges.
- Docker allows every port without `NET_BIND_SERVICE` (`ip_unprivileged_port_start = 0`); most applications need no capabilities at all.

Full lesson with every command and its real output: [085-linux-capabilities](#)

Lesson 086 · Resource limits for security

Containers share the host's CPU, memory, process table and open-file table. Without limits, **one** container, buggy or compromised, can exhaust them and take every other container and the host down with it: a denial of service from the inside. Limits turn "the host is down" into "one container failed". This lesson covers the security view and the limits that are easy to forget: **processes** (`--pids-limit`) and **open files** (`--ulimit nofile`). Module 15 covers CPU and memory in depth.

VISUAL



Flag	Protects against	cgroup / kernel setting
<code>--pids-limit 100</code>	fork bombs, runaway process creation	<code>pids.max</code>
<code>--memory 256m</code>	memory exhaustion (lesson 098)	<code>memory.max</code>
<code>--cpus 1</code>	CPU starvation (lesson 097)	<code>cpu.max</code>
<code>--ulimit nofile=1024:1024</code>	file-descriptor exhaustion	<code>RLIMIT_NOFILE</code>

COMMANDS

Command	What it does
<code>--pids-limit N</code>	at most N processes and threads in the container
<code>--ulimit nofile=SOFT:HARD</code>	maximum open files per process
<code>--memory</code> , <code>--cpus</code>	memory and CPU ceilings (module 15)
<code>docker stats --no-stream</code>	current CPU, memory and PIDs per container
<code>docker inspect --format '{{.HostConfig.PidsLimit}}'</code>	the configured process limit

RECAP

- Unlimited containers can exhaust the host's processes, memory and CPU: a denial of service from the inside.
- `--pids-limit` stops fork bombs; `--ulimit nofile` bounds open files; `--memory` and `--cpus` bound the rest.
- "Resource temporarily unavailable" / `fork() failed` = the pids limit was reached.
- Measure normal usage, then set limits with headroom.

Full lesson with every command and its real output: [086-resource-limits-for-security](#)

Module 13 · Multi-stage builds

Lesson 087 · Why multi-stage builds?

Building an application needs compilers, package managers and source code; **running** it usually needs only the result. A single-stage image ships both (lesson 045 measured the waste). A **multi-stage** Dockerfile has several `FROM` instructions: each starts a new **stage**, and a later stage copies only the files it needs from an earlier one with `COPY --from=STAGE`. Only the last stage becomes the image. Smaller images pull faster, start faster and contain far fewer programs an attacker could use or a scanner could flag.

VISUAL

Dockerfile.single	Dockerfile (multi-stage)
<pre>FROM golang:1.26-alpine COPY go.mod / main.go RUN go build -o /usr/local/bin/go-api CMD ["go-api"]</pre>	<pre>FROM golang:1.26-alpine AS build COPY go.mod / main.go RUN go build -o /out/go-api FROM alpine:3.23 COPY --from=build /out/go-api ... CMD ["go-api"]</pre> stage "build" (thrown away, stays in the build cache) final stage = the image
image = toolchain + Alpine + binary (≈ 490 MB)	image = Alpine + binary (≈ 26 MB)

COMMANDS

Instruction / flag	Meaning
<code>FROM IMAGE AS NAME</code>	start a new stage and give it a name
<code>COPY --from=NAME SRC DEST</code>	copy files from an earlier stage (or <code>--from=IMAGE</code> from any image)
the last <code>FROM</code>	the final stage: the only one that becomes the image
<code>docker build --target NAME</code>	build only up to stage NAME and make that the image
stages not needed by the target	skipped entirely by BuildKit

RECAP

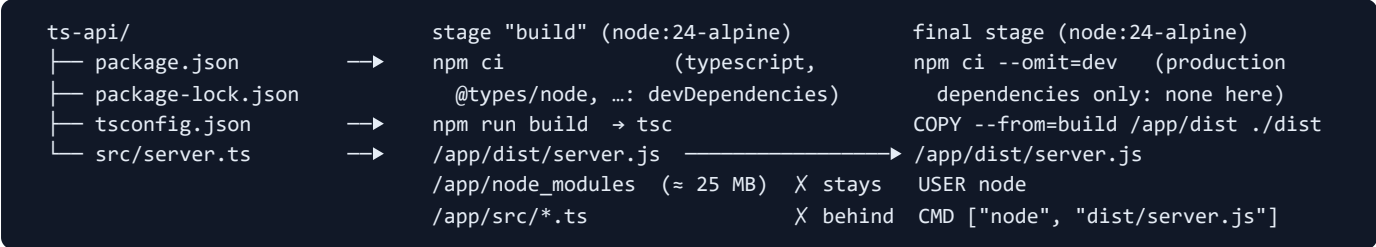
- Several `FROM` s = several stages; only the last one is the image.
- `COPY --from=STAGE` copies files from a stage's file system: build tools never reach the final image.
- `--target STAGE` builds up to a stage: for debugging, tests or a development image.
- `"...": not found` in `COPY --from`: inspect the stage with `--target` to find the real path.

Full lesson with every command and its real output: [087-why-multistage](#)

Lesson 088 · Multi-stage builds for Node.js

A Node.js service written in TypeScript (or with a bundler, a frontend framework, test tools) needs **development dependencies** to build, and only **production dependencies** plus the compiled JavaScript to run. A two-stage Dockerfile installs everything and compiles in the first stage, then starts the final stage fresh: `npm ci --omit=dev` and `COPY --from=build /app/dist`. The TypeScript compiler, the type definitions and the `.ts` sources never reach the image.

VISUAL



COMMANDS

Step	Why
FROM node:24-alpine AS build + npm ci	install production and development dependencies to build
RUN npm run build	compile TypeScript to <code>dist/</code>
second FROM node:24-alpine	a clean start: nothing from the build stage unless copied
npm ci --omit=dev	production dependencies only, from the same lockfile
COPY --from=build /app/dist ./dist	the only thing taken from the build stage

RECAP

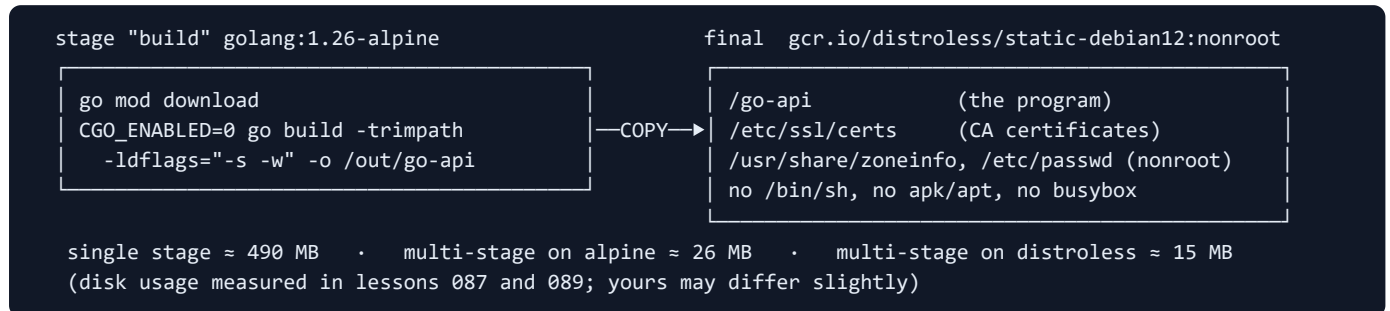
- Build stage: all dependencies + compile; final stage: production dependencies + compiled output.
- `npm ci --omit=dev` in the final stage keeps development tools out.
- A missing `COPY --from` builds fine and fails at run time: smoke-test every image you build.
- `--target build` gives a stage with the full toolchain for checks and debugging.

Full lesson with every command and its real output: [088-multistage-node](#)

Lesson 089 · Multi-stage builds for Go

A static Go binary needs no runtime, no shell and no C library, so its final stage can be almost empty. **Distroless** images (`gcr.io/distroless/static-debian12`) contain only what such a program may still need: CA certificates for HTTPS, time zone data, `/etc/passwd` with a non-root user. No shell, no package manager. The image shrinks to little more than the binary, and an attacker who gets into the process finds no tools. The price: you cannot `docker exec ... sh` into it, and every command must be in exec form.

VISUAL



Final base	Contains	Use when
<code>scratch</code>	nothing at all	the binary needs no certificates, users or time zones
<code>gcr.io/distroless/static-debian12:nonroot</code>	CA certs, tzdata, passwd, non-root user	static binaries (Go, Rust)
<code>alpine:3.23</code>	a shell and <code>apk</code>	you need tools in the image (debugging, scripts)

COMMANDS

Instruction / flag	Meaning
<code>-trimpath</code>	remove local file system paths from the binary (reproducible builds)
<code>-ldflags="-s -w"</code>	strip the symbol table and debug information: a smaller binary
<code>FROM gcr.io/distroless/static-debian12:nonroot</code>	minimal base with a non-root default user (uid 65532)
<code>ENTRYPOINT ["/go-api"]</code>	exec form: started directly, no shell involved
<code>docker image inspect --format '{{.Config.User}}'</code>	the user the image runs as by default

RECAP

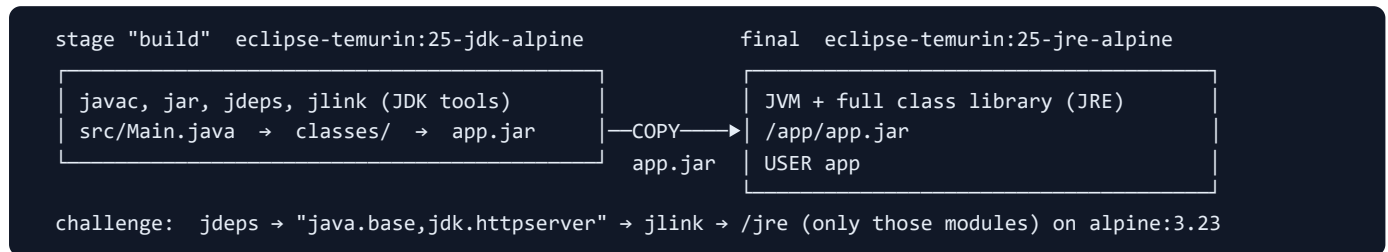
- Static Go binaries run on `distroless/static` (or `scratch`): images of a few MB.
- `:nonroot` distroless images run as uid 65532 by default.
- No shell: `ENTRYPOINT / CMD` must use the exec form (JSON array), and `docker exec ... sh` is impossible.
- Debug through a sidecar container that shares the network namespace (`--network container:NAME`).

Full lesson with every command and its real output: [089-multistage-go](#)

Lesson 090 · Multi-stage builds for Java

Java needs the **JDK** (compiler, `jar`, Maven or Gradle) to build and only the **JRE** (the JVM and the class libraries) to run. A two-stage Dockerfile compiles in `eclipse-temurin:25-jdk-alpine` and runs in `eclipse-temurin:25-jre-alpine`, copying just the jar. One step further, `jlink` builds a **custom runtime** containing only the Java modules the application uses.

VISUAL



COMMANDS

Instruction / tool	Meaning
<code>FROM eclipse-temurin:25-jdk-alpine AS build</code>	build stage with the full JDK
<code>FROM eclipse-temurin:25-jre-alpine</code>	final stage: the runtime only
<code>COPY --from=build /src/app.jar ./app.jar</code>	the jar is the only artefact that crosses over
<code>ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]</code>	start the JVM with a container-friendly heap
<code>jdeps --print-module-deps app.jar</code>	list the JDK modules a jar uses
<code>jlink --add-modules ... --output /jre</code>	build a Java runtime containing only those modules
<code>docker run --entrypoint sh IMAGE -c '...'</code>	replace the image's entrypoint for one container

RECAP

- Build with the JDK, run with the JRE: `COPY --from=build` only the jar.
- Relative `COPY` destinations and `ENTRYPOINT` paths depend on `WORKDIR`; absolute ones ignore it.
- `--entrypoint` overrides an image's entrypoint to inspect it.
- `jdeps` + `jlink` build a custom runtime with only the modules the application needs.

Full lesson with every command and its real output: [090-multistage-java](#)

Lesson 091 · Measuring image size

"Make the image smaller" only means something if you measure. Docker reports two sizes for an image: the **disk usage** (the unpacked files on the engine's disk) and the **content size** (the compressed layers a registry stores and a `docker pull` downloads). `docker history` breaks the size down by instruction, so you can see which line of the Dockerfile is responsible. This lesson measures the module's Node.js images and finds an oversized layer.

VISUAL

```
docker image ls ts-multi
IMAGE          DISK USAGE  CONTENT SIZE
ts-multi:1.0   ...MB       ...MB
```

```
docker history ts-multi:1.0 (newest first)
SIZE          CREATED BY
0B            CMD ["node" "dist/server.js"]
...kB         COPY /app/dist ./dist
...kB         RUN npm ci --omit=dev
...           ... node:24-alpine's own layers
```

compressed: what is pushed and pulled over the network

unpacked: what the image occupies on disk

your share of the image = the layers above the base image's layers

COMMANDS

Command	What it measures
<code>docker image ls</code> → DISK USAGE	unpacked size on the engine's disk (shared layers counted in every image)
<code>docker image ls</code> → CONTENT SIZE	compressed size: what a registry stores and a pull downloads
<code>docker history IMAGE</code>	size of each layer and the instruction that created it
<code>docker image inspect --format '{{.Size}}'</code>	the content size in bytes, for scripts
<code>docker system df</code>	disk used by all images, containers, volumes and the build cache

RECAP

- Disk usage = unpacked size; content size = compressed, what is pulled and pushed.
- `docker history` attributes size to Dockerfile instructions; your layers are above the base image's.
- `docker image inspect --format '{{.Size}}'` gives exact bytes for scripts and CI checks.
- `COPY --from` copies exactly what you name: copy build output, not build folders.

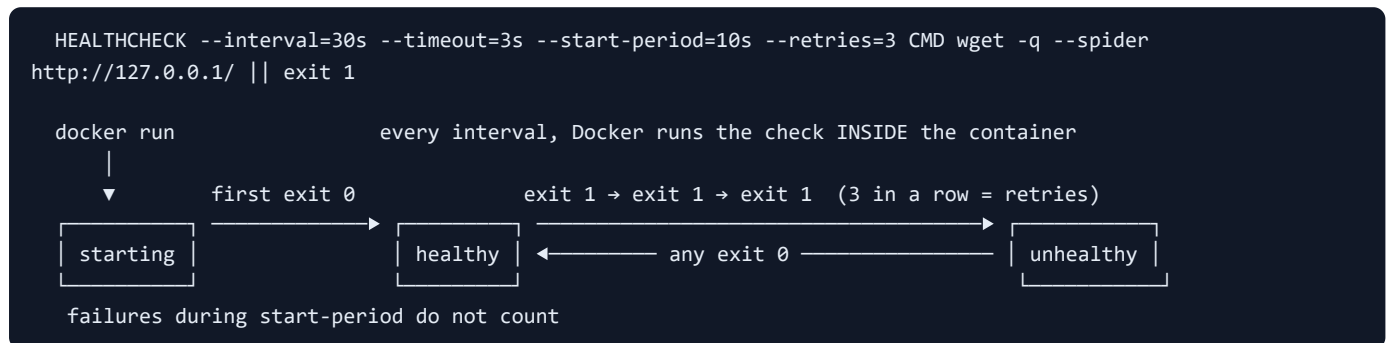
Full lesson with every command and its real output: [091-measuring-image-size](#)

Module 14 · Observability

Lesson 092 · HEALTHCHECK

Docker knows whether a container's **process** is running. It does not know whether the application **works**. A **health check** is a command that Docker runs inside the container at regular intervals: exit status 0 means healthy, anything else counts as a failure. After `--retries` failures in a row, the container is marked **unhealthy**. Compose (`depends_on: condition: service_healthy`, lesson 069), Swarm and monitoring tools act on that status.

VISUAL



COMMANDS

Option (Dockerfile / docker run)	Meaning
HEALTHCHECK CMD ... / --health-cmd	the check; exit 0 = healthy, 1 = unhealthy
--interval / --health-interval	time between checks (default 30 s)
--timeout / --health-timeout	a check running longer counts as failed (default 30 s)
--start-period / --health-start-period	failures during the start do not count (default 0 s)
--retries / --health-retries	failures in a row before unhealthy (default 3)
HEALTHCHECK NONE / --no-healthcheck	disable a check inherited from the base image
.State.Health	status, failing streak and the last 5 check results

RECAP

- A health check is a command run inside the container; exit 0 = healthy.
- Status: `starting` → `healthy` or, after `retries` failures in a row, `unhealthy`.
- Check from inside the container: use the container's port, and a tool that exists in the image.
- Read failures in `docker inspect --format '{{json .State.Health}}'`.

Full lesson with every command and its real output: [092-healthcheck](#)

Lesson 093 · Running is not healthy

`docker ps` says `Up`, the process exists, and yet nobody gets an answer: the application is stuck (a deadlock, a request waiting forever, an exhausted connection pool). Docker restarts containers only when their process **exits**; a hung process never exits. This lesson shows the difference between **running** and **healthy**, how to detect a hung application, and why Docker on its own does not repair it.

VISUAL

policy	process alive?	answers requests?	docker ps	docker restart
working app	yes	yes	Up 2 minutes (healthy)	nothing to do
crashed app	no (exited)	no	Exited (1)	restarts it ✓
hung app	yes	NO	Up 2 minutes (unhealthy)	does nothing X

Only a health check sees the third case. Acting on it is the job of an orchestrator (Swarm, Kubernetes), a monitoring system, or you.

COMMANDS

Command	What it shows
<code>docker inspect --format '{{.State.Status}}'</code>	the process state: <code>running</code> , <code>exited</code> , <code>restarting</code> , ...
<code>docker inspect --format '{{.State.Health.Status}}'</code>	the health state: <code>starting</code> , <code>healthy</code> , <code>unhealthy</code>
<code>docker inspect --format '{{.RestartCount}}'</code>	how often the restart policy restarted the container
<code>curl -m 2 URL</code>	give up after 2 seconds instead of waiting forever
<code>docker ps --filter health=unhealthy</code>	every unhealthy container on the host

RECAP

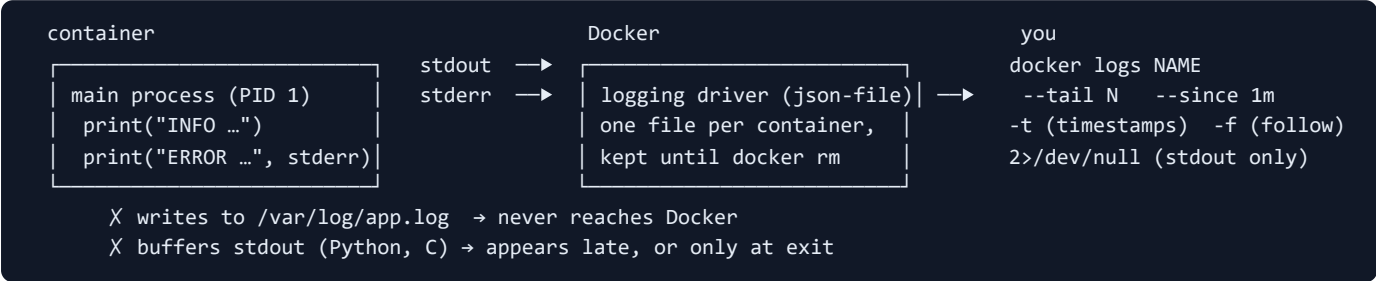
- `running` describes the process; `healthy` describes the application. They can disagree.
- Restart policies act only when the process exits; a hung process is never restarted by them.
- Find hung containers with health checks: `docker ps --filter health=unhealthy`.
- Restarting fixes the symptom; timeouts and concurrency in the code fix the cause.

Full lesson with every command and its real output: [093-running-is-not-healthy](#)

Lesson 094 · Logs in depth

Docker captures everything a container's main process writes to **standard output** (stdout) and **standard error** (stderr), and stores it through a **logging driver** (by default `json-file`, lesson 102). `docker logs` reads it back. Lesson 014 covered the basics; this lesson covers the parts that matter when something is wrong: the two streams, time filters, timestamps, and the most common reason logs are missing: **output buffering**.

VISUAL



COMMANDS

Command	What it does
<code>docker logs NAME</code>	everything the container wrote to stdout and stderr since it was created
<code>--tail N</code>	only the last N lines
<code>-t</code>	prefix each line with the time Docker received it
<code>--since 10m / --until 2026-10-05T10:00:00</code>	a time window (relative or absolute)
<code>-f</code>	follow: keep printing new lines until Ctrl+C or until the container stops
<code>docker logs NAME 2>/dev/null</code>	the container's stdout only
<code>docker logs NAME 2>&1 >/dev/null</code>	the container's stderr only

RECAP

- Docker captures the main process's stdout and stderr; `docker logs` keeps them as separate streams.
- `--tail`, `--since`, `--until`, `-t` and `-f` narrow down what you read.
- Missing log lines with a running process: suspect output buffering (`PYTHONUNBUFFERED=1`) or logs written to a file.
- Write errors to stderr and everything else to stdout, one event per line.

Full lesson with every command and its real output: [094-logs-in-depth](#)

Lesson 095 · docker stats

`docker stats` shows, for every running container, how much CPU, memory, network and disk I/O it uses right now, and how many processes it runs. It reads the numbers from the kernel's **cgroups**, the same mechanism that enforces the limits of lessons 097–099. When a host is slow, `docker stats` answers the first question: **which container** is it?

VISUAL

```
docker stats --no-stream
```

NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
web	0.00%	9.1MiB / 15.4GiB	0.06%	1.2kB / 0B	0B / 4kB	15
worker	99.87%	1.2MiB / 15.4GiB	0.01%	872B / 0B	0B / 0B	2
cache	0.21%	3.4MiB / 64MiB	5.31%	1kB / 0B	0B / 0B	6

← the busy one
← limited to 64 MiB

the container's memory limit (or the whole engine's memory)
memory the container uses now
100% = one full CPU core (a container can use more than 100% on several cores)

COMMANDS

Command / placeholder	What it shows
<code>docker stats</code>	live usage of all running containers (Ctrl+C to stop)
<code>--no-stream</code>	one snapshot, then exit (for scripts)
<code>--format 'table {{.Name}}\t{{.CPUPerc}}...'</code>	chosen columns; <code>.MemPerc</code> , <code>.NetIO</code> , <code>.BlockIO</code> , <code>.PIDs</code> exist too
<code>docker stats NAME ...</code>	only these containers
<code>docker top NAME</code>	the processes inside one container
<code>docker update --cpus 0.5 NAME</code>	change a running container's CPU limit without restarting it

RECAP

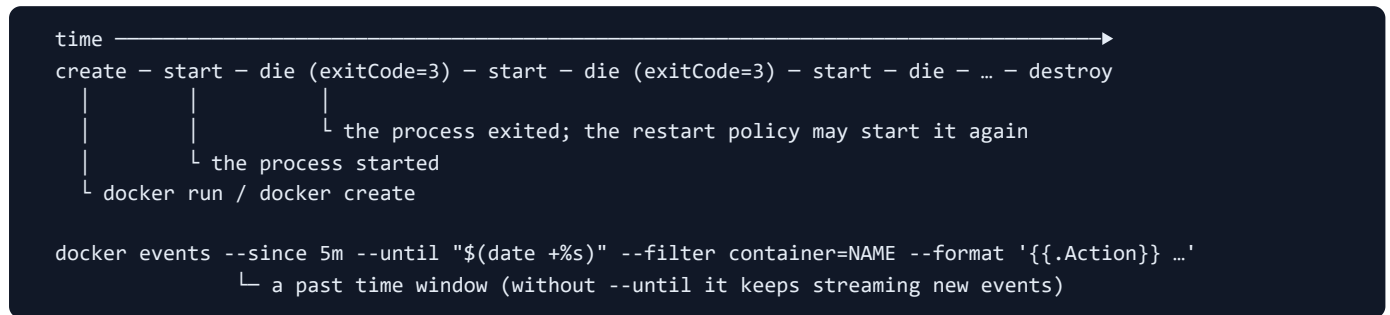
- `docker stats` shows per-container CPU, memory (usage / limit), network, disk I/O and process count.
- `--no-stream` and `--format` make it scriptable; `sort` finds the top consumer.
- CPU 100% = one full core; the memory limit column shows the container's limit, or the engine's memory.
- Contain with `docker update --cpus/--memory` , then fix the cause.

Full lesson with every command and its real output: [095-docker-stats](#)

Lesson 096 · docker events

The Docker engine records an **event** for everything that happens: a container is created, started, dies (with its exit code), is killed by the kernel for using too much memory (`oom`), changes health status, is destroyed; images are pulled, networks connected, volumes mounted. `docker events` streams them live or replays a time window. When a container "keeps restarting" or "disappeared", the events tell you what happened and when.

VISUAL



COMMANDS

Command / option	What it does
<code>docker events</code>	stream new events until Ctrl+C
<code>--since 10m</code> / <code>--since 2026-10-05T10:00:00</code>	start from a moment in the past
<code>--until "\$(date +%s)"</code>	stop at that moment (then exit)
<code>--filter container=NAME</code>	one container (also <code>image=</code> , <code>network=</code> , <code>volume=</code> , <code>type=container</code>)
<code>--filter event=die</code>	one action (<code>create</code> , <code>start</code> , <code>die</code> , <code>oom</code> , <code>kill</code> , <code>health_status</code> , <code>destroy</code> , ...)
<code>--format '{{.Action}} {{.Actor.Attributes.exitCode}}'</code>	chosen fields; <code>{{json .}}</code> shows them all

RECAP

- The engine emits an event for every change: create, start, die, oom, kill, health_status, destroy, ...
- `--since` / `--until` replay a window; without `--until`, the command streams forever.
- Filter by `container=`, `type=`, `event=`; read attributes such as `exitCode` with `--format`.
- Restart loops: count `die` events, read the exit code, then the logs.

Full lesson with every command and its real output: [096-docker-events](#)

Module 15 · Resource management

Lesson 097 · CPU limits

Without limits, one container can use every CPU core of the host and slow down everything else. Docker offers three controls, all enforced by the kernel's **cgroups**:

- `--cpus 1.5`: a hard ceiling, at most one and a half cores' worth of CPU time (the usual choice),
- `--cpuset-cpus 0,1`: run only on these cores,
- `--cpu-shares 512`: a relative weight that matters only when containers compete (default 1024).

VISUAL

```
--cpus 0.5 → cgroup cpu.max = "50000 100000"
              in every 100 ms period the container may run for 50 ms (summed over all its threads)
```



docker stats CPU % → about 50% the application is not killed, only slowed down

COMMANDS

Flag	Effect	cgroup file
<code>--cpus 0.5</code>	at most half a core of CPU time	<code>cpu.max = 50000 100000</code>
<code>--cpuset-cpus 0,1</code>	only on cores 0 and 1	<code>cpuset.cpus.effective</code>
<code>--cpu-shares 512</code>	half the default weight when CPUs are contended	<code>cpu.weight</code>
<code>docker update --cpus 1 NAME</code>	change the limit of a running container	
<code>docker inspect --format '{{.HostConfig.NanoCpus}}'</code>	the limit in billionths of a core	

RECAP

- `--cpus N` sets a hard ceiling of N cores' worth of CPU time (cgroup `cpu.max`).
- A limited container is throttled (slowed down), never killed, for using too much CPU.
- `--cpuset-cpus` pins to cores; `--cpu-shares` only weights containers against each other under contention.
- `docker update --cpus` changes the limit of a running container.

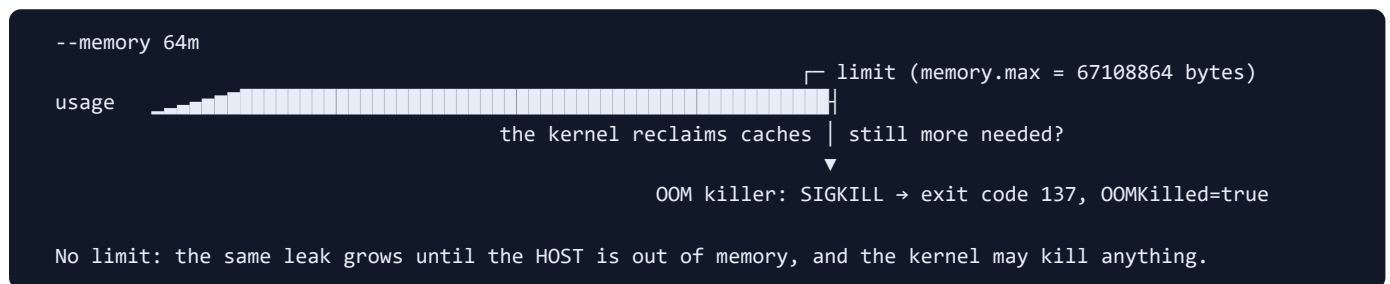
Full lesson with every command and its real output: [097-cpu-limits](#)

Lesson 098 · Memory limits

Memory is different from CPU: CPU can be slowed down, memory cannot be "slowed". When a container reaches its memory limit, the kernel first reclaims what it can (file caches), and if the container still needs more, the kernel's **OOM killer** (out of memory) kills a process in it. Without a limit, a leaking container can eat the host's memory until the kernel starts killing processes of **other** containers, or of the host.

Flag	Meaning	cgroup file
<code>--memory 256m</code>	hard limit: above it, the OOM killer acts	<code>memory.max</code>
<code>--memory-swap 256m</code>	memory + swap; equal to <code>--memory</code> = no swap	<code>memory.swap.max</code>
<code>--memory-reservation 128m</code>	soft limit: memory the kernel reclaims first when the host is short	<code>memory.low</code>

VISUAL



COMMANDS

Command	What it does
<code>--memory 64m</code> (<code>-m 64m</code>)	hard memory limit (<code>b</code> , <code>k</code> , <code>m</code> , <code>g</code> units)
<code>--memory-swap 64m</code>	total memory + swap; same value as <code>--memory</code> disables swap
<code>--memory-reservation 32m</code>	soft limit (<code>memory.low</code>)
<code>docker inspect --format '{{.HostConfig.Memory}}'</code>	the limit in bytes (0 = none)
<code>docker inspect --format '{{.State.OOMKilled}}'</code>	whether the OOM killer stopped the container

RECAP

- `--memory` is a hard limit enforced by the kernel's OOM killer; `--memory-swap` equal to it disables swap.
- An OOM-killed container exits with 137 (SIGKILL) and `OOMKilled=true`.
- Limits below 6 MB, or reservations above the limit, are refused before the start.
- Measure real usage with `docker stats`, then set a limit with headroom.

Full lesson with every command and its real output: [098-memory-limits](#)

Lesson 099 · Memory pressure and OOM kills

How to recognise and investigate a container that runs out of memory. The signs differ depending on **which** process the kernel kills:

- the container's main process (PID 1): the container stops, exit code **137**, `OOMKilled=true` ;
- a child process (a worker under a process manager, a script started by a shell): the container keeps running and `docker ps` shows `Up` . Docker still sets `OOMKilled=true` , but only `docker inspect` , Docker's `oom` event and the kernel's counters reveal it.

Exit code 137 on its own only means "killed with SIGKILL": `docker kill` and a `docker stop` time-out produce it too.

VISUAL

	container stops?	exit code	OOMKilled	docker events	memory.events
oom_kill					
PID 1 killed by OOM	yes	137	true	oom, die	1
child killed by OOM	no (still Up)	-	true	oom	1 ← easy to miss
docker kill / stop timeout	yes	137	false	kill, die	0

Investigation: `docker inspect (State) → docker events --filter event=oom → docker stats (usage vs limit) → docker logs (what was happening) → cat /sys/fs/cgroup/memory.events (inside)`

COMMANDS

Command	What it tells you
<code>docker inspect --format '{{.State.ExitCode}} {{.State.OOMKilled}}'</code>	137 + true: the main process was OOM-killed; running + true: a child was
<code>docker events --filter event=oom</code>	every OOM kill in the container, main process or not
<code>cat /sys/fs/cgroup/memory.events</code> (inside)	<code>oom_kill N</code> : how many processes the kernel killed
<code>docker stats --no-stream</code>	usage vs limit right now
<code>docker inspect --format '{{.HostConfig.Memory}}'</code>	the limit in bytes

RECAP

- 137 = SIGKILL. `OOMKilled=true` with exit 137: the main process was OOM-killed; `docker kill` gives 137 without it.
- OOM kills of child processes leave the container `Up` : check `OOMKilled` , `memory.events` and `docker events --filter event=oom` .
- Logs and `docker stats` show the growth pattern: steady climb = leak, sudden jump = one large operation.
- Fix the memory use first; raise the limit only when the application legitimately needs more.

Full lesson with every command and its real output: [099-memory-pressure](#)

Module 16 · Advanced Docker

Lesson 100 · Metadata and docker inspect

Every Docker object (image, container, network, volume) has a JSON document with everything the engine knows about it. `docker inspect` prints it, and `--format` with Go templates extracts exactly the fields you need, for scripts, for debugging, and for audits. Lesson 016 introduced it; this lesson shows how to navigate the document, the template functions that matter, and the difference between an **image's** and a **container's** metadata.

VISUAL

image metadata (docker image inspect)

Id, RepoTags, RepoDigests

Architecture, Os, Size

Config Env, Cmd, Entrypoint,

ExposedPorts, User,

Labels, Healthcheck

RootFS Layers

—defaults→

container metadata (docker container inspect)

Id, Name, Image, Created, RestartCount

State Status, ExitCode, OOMKilled,

Health, StartedAt

Config what this container runs with

HostConfig limits, ports, mounts, caps

Mounts volumes and bind mounts

NetworkSettings Networks, IPAddress, Ports

`docker inspect --format '{{.State.Status}}' · '{{json .Config.Env}}' · '{{range ...}}{{end}}'`

COMMANDS

Template	Meaning
<code>{{.A.B}}</code>	field B inside A (names are case-sensitive)
<code>{{json .X}}</code>	X as JSON (for lists and maps)
<code>{{range .List}}{{.Field}}{{end}}</code>	repeat for every element
<code>{{range \$k, \$v := .Map}}{{\$k}}={{\$v}}{{end}}</code>	repeat for every key of a map
<code>{{index .Map "key"}}</code>	a map value whose key has special characters (<code>80/tcp</code>)
<code>{{len .List}}</code>	number of elements
<code>{{println .}}</code>	the value and a line break
<code>docker inspect --type image NAME</code>	inspect the image even if a container has the same name

RECAP

- `docker inspect` prints the engine's JSON document for any object; `--format` extracts fields.
- Images describe defaults (`Config` , `RootFS`); containers add `State` , `HostConfig` , `Mounts` , `NetworkSettings` .
- `json` , `range` , `index` , `len` , `if` cover almost every need.
- Use `docker container inspect` / `docker image inspect` in scripts to get clear errors.

Full lesson with every command and its real output: [100-metadata-and-inspect](#)

Lesson 101 · Labels

A **label** is a key-value pair of metadata attached to an image, a container, a network or a volume. Docker itself ignores labels; people and tools use them to answer "what is this?": which source code and version an image was built from, which team owns a container, which environment it belongs to. Filters (`--filter label=...`) then select objects by label for listing, monitoring or cleanup.

VISUAL

Image labels (LABEL in the Dockerfile, fixed at build)
container)

```
org.opencontainers.image.source = https://github.com/...
org.opencontainers.image.revision = 3f2c1ab
org.opencontainers.image.version = 1.4.0
```

Container labels (docker run --label, per
container)

```
team = payments
env = staging
com.example.cost-center = 4711
```

```
docker ps --filter label=team=payments
label=org.opencontainers.image.version=1.4.0
docker container prune --filter label=env=test
```

```
docker image ls --filter
docker system prune --filter label!=keep
```

The `org.opencontainers.image.*` keys are a standard (the OCI image specification): registries such as GitHub Container Registry use `...source` to link an image to its repository.

COMMANDS

Command	What it does
<code>LABEL key="value" ...</code>	add labels to the image (Dockerfile)
<code>docker run --label key=value</code>	add a label to the container
<code>docker build --label key=value</code>	add a label at build time without changing the Dockerfile
<code>--filter label=key / label=key=value</code>	objects that have the label / that value
<code>--filter label!=key=value</code>	objects without it (prune commands)
<code>{{.Label "key"}} in docker ps --format</code>	one label's value
<code>{{json .Config.Labels}} in docker inspect</code>	all labels

RECAP

- Labels are key-value metadata on images (`LABEL` , `--label` at build) and containers (`docker run --label`).
- Use the `org.opencontainers.image.*` keys for source, revision, version and license.
- `--filter label=key=value` selects objects; keys and values match exactly, case included.
- List what a filter selects before you delete it.

Full lesson with every command and its real output: [101-labels](#)

Lesson 102 · Logging drivers

Where container output goes is decided by the **logging driver**. The default, `json-file`, writes one JSON line per log line into a file on the Docker host, **without size limit** unless you configure one: a chatty container can fill the host's disk. Other drivers keep logs in a compact local format (`local`), send them to the system journal (`journald`), to syslog, or to a central service (`fluentd`, `gelf`, `awslogs`, `gcplogs`, ...), or discard them (`none`). Some of them cannot be read back with `docker logs`.

VISUAL

container stdout/stderr → logging driver —┐ json-file file on the host (default; rotate with max-size/max-file)			
logs ✓	└─ local	compact files, rotated by default	docker
logs ✓	└─ journald	systemd journal	docker
logs ✓*	└─ syslog / fluentd / gelf / awslogs / ...	sent away	docker
logs ✗	└─ none	discarded	docker
* since Docker 20.10, "dual logging" keeps a local copy so docker logs still works for remote drivers			

COMMANDS

Option	What it does
<code>--log-driver NAME</code>	the driver for this container (<code>json-file</code> , <code>local</code> , <code>journald</code> , <code>none</code> , ...)
<code>--log-opt max-size=10m</code>	rotate the file at 10 MB (<code>json-file</code> , <code>local</code>)
<code>--log-opt max-file=3</code>	keep 3 files: at most 30 MB per container with the line above
<code>--log-opt tag=...</code>	a tag for remote drivers (e.g. <code>{{.Name}}</code>)
<code>daemon.json</code> <code>"log-driver"</code> , <code>"log-opts"</code>	the engine-wide default (restart the engine; applies to new containers)
<code>docker inspect --format '{{.HostConfig.LogConfig}}'</code>	a container's driver and options

RECAP

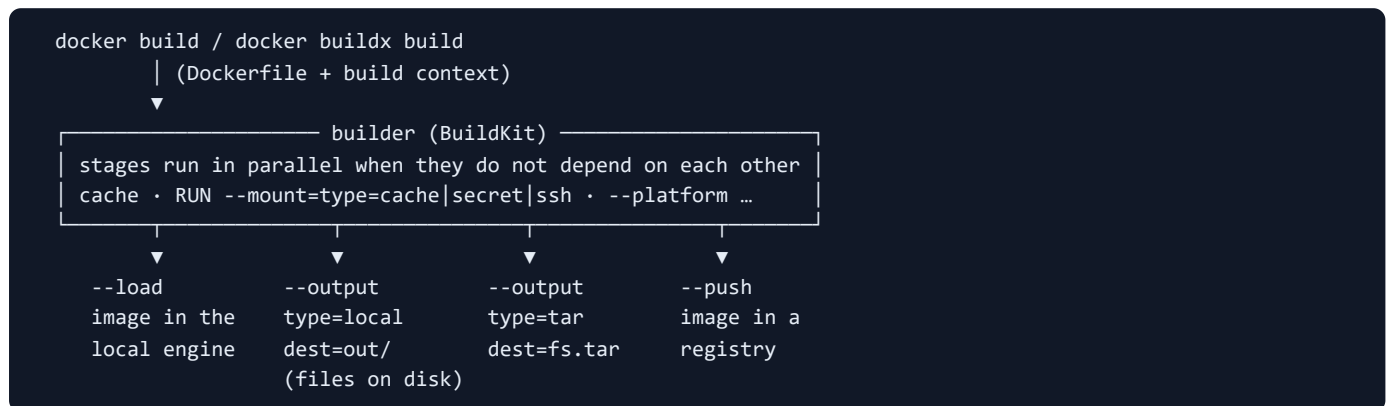
- The logging driver decides where stdout/stderr go; the default is `json-file`, unrotated.
- Rotate with `--log-opt max-size=...` `--log-opt max-file=...`, or use the `local` driver; set defaults in `daemon.json`.
- `none` discards logs: `docker logs` fails with `does not support reading`, and the lines are lost.
- The driver is fixed when the container is created: recreate the container to change it.

Full lesson with every command and its real output: [102-logging-drivers](#)

Lesson 103 · BuildKit and buildx

Since Docker 23, every `docker build` runs on **BuildKit**, the build engine, through the **buildx** client plugin (`docker build` is an alias of `docker buildx build`). BuildKit is what makes the features of the last lessons possible: parallel stages, cache mounts and secret mounts (`RUN --mount`), multi-platform builds (lesson 104), and **outputs** other than an image: files on your disk, a tar archive, an OCI layout, or a push straight to a registry. buildx also manages **builders**: the engine's own, or separate BuildKit instances in a container, on another machine, or in Kubernetes.

VISUAL



COMMANDS

Command / option	What it does
<code>docker buildx ls</code>	the builders and their platforms
<code>docker buildx build --progress plain</code>	full log of every step (<code>auto</code> , <code>tty</code> , <code>quiet</code> also exist)
<code>--target STAGE</code>	build up to that stage only
<code>--output type=local,dest=DIR</code>	write the result's files into DIR
<code>--output type=tar,dest=FILE.tar</code>	the same, as one tar file
<code>--load</code> / <code>--push</code>	the result as an image in the engine / in a registry
<code>docker buildx du</code>	disk used by the build cache
<code>docker buildx prune</code>	remove build cache
<code>docker buildx create --driver docker-container</code>	a separate BuildKit instance (pulls the <code>moby/buildkit</code> image)

RECAP

- `docker build` runs on BuildKit through buildx; builders can be local, containerised or remote.
- `--target` builds one stage; `--output type=local|tar` writes files instead of an image.
- `--progress plain` shows every step; `docker buildx du` and `prune` manage the build cache.
- Wrong stage names fail before any step runs: stage names come from `FROM ... AS name`.

Full lesson with every command and its real output: [103-buildx](#)

Lesson 104 · Multi-architecture images

A container image contains machine code for one CPU architecture: an `amd64` (x86-64) binary does not run on an `arm64` processor (Apple silicon, AWS Graviton, Raspberry Pi) without emulation. A **multi-architecture image** is one name, one tag, with a variant per platform behind it (an **image index**, also called a manifest list). `docker pull` and `docker run` pick the variant that matches the machine automatically. `docker buildx build --platform ...` builds all variants in one command.

VISUAL

```
cafe-hello:1.0 (image index)
├─ linux/amd64 → manifest → layers with x86-64 machine code ← picked on a laptop / most servers
└─ linux/arm64 → manifest → layers with ARM64 machine code ← picked on Apple silicon, Graviton
```

How it is built (cross-compilation, fast):

```
FROM --platform=$BUILDPLATFORM golang runs natively, once per target
GOOS=$TARGETOS GOARCH=$TARGETARCH go build → hello (amd64), hello (arm64)
FROM scratch + COPY hello → one small image per platform
```

Alternative: build every stage under emulation (QEMU), slower but works for any language.

COMMANDS

Command / instruction	What it does
<code>docker buildx build --platform linux/amd64,linux/arm64</code>	build one variant per platform
<code>--load</code> / <code>--push</code>	store the result in the local engine / push it to a registry (lesson 076)
<code>FROM --platform=\$BUILDPLATFORM ...</code>	run this stage on the build machine's own platform
<code>ARG TARGETOS TARGETARCH</code>	the platform being built, set by buildx for every variant
<code>docker image ls --tree NAME</code>	the platform variants of a local image
<code>docker run --platform linux/arm64</code>	run a specific variant (emulated if it is not the machine's)
<code>docker buildx imagetools inspect REF</code>	the variants of an image in a registry

RECAP

- An image index groups one variant per platform under one tag; the engine picks the matching one.
- `docker buildx build --platform a,b` builds all variants; `--push` (or `--load` with the containerd image store).
- `FROM --platform=$BUILDPLATFORM + TARGETOS/TARGETARCH` cross-compiles natively instead of emulating.
- "does not provide the specified platform": the variant does not exist; "exec format error": no emulation.

Full lesson with every command and its real output: [104-multi-architecture-images](#)

Lesson 105 · Build cache optimization

Lesson 039 explained the build cache: a step is reused when it and everything before it are unchanged. This lesson applies it to make real builds fast:

1. **Order** steps from least to most frequently changed: dependencies before code.
2. Copy **only** what a step needs (`COPY requirements.txt` before `COPY . .`).
3. Use **cache mounts** (`RUN --mount=type=cache`) so package managers keep their downloads between builds.
4. Put values that change on every build (time, commit) **last**.

VISUAL

Before		After	
FROM python:3.14-slim	cached	FROM python:3.14-slim	cached
COPY . .	X changed	COPY requirements.txt .	cached
RUN pip install ...	X re-runs (downloads everything again)	RUN --mount=type=cache... pip install ...	cached
		COPY . .	X changed (only this)
one changed line of app.py		one changed line of app.py	
→ every step after COPY . . runs again		→ only the last, cheap step runs again	
ARG BUILD_TIME at the top → every RUN after it re-runs on every build (put it at the end)			

COMMANDS

Technique	Dockerfile
dependencies first	<code>COPY requirements.txt .</code> → <code>RUN pip install ...</code> → <code>COPY . .</code>
cache mount	<code>RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt</code>
changing values last	<code>ARG BUILD_TIME</code> + <code>LABEL ...</code> at the end
keep the context small	<code>.dockerignore</code> (lesson 037): changes to ignored files never invalidate <code>COPY . .</code>
share cache in CI	<code>docker buildx build --cache-to type=registry,... --cache-from type=registry,...</code>

The same order works for every stack: `package.json` + `package-lock.json` → `npm ci`; `go.mod` + `go.sum` → `go mod download`; `pom.xml` → `mvn dependency:go-offline`; `composer.json` + `composer.lock` → `composer install`.

RECAP

- One cache miss invalidates every later step: order steps from least to most frequently changed.
- Copy dependency manifests before the code; copy the code last.
- `RUN --mount=type=cache,target=...` keeps package manager caches between builds, outside the image.
- Values that change on every build (`ARG BUILD_TIME` , commit) belong at the end.
- Diagnose with `--progress plain` : the first step that ran is the culprit.

Full lesson with every command and its real output: [105-cache-optimization](#)

Module 18 · Production

Lesson 130 · Production Dockerfile

A Dockerfile that builds and runs is a starting point. A **production** Dockerfile also makes the image small, safe, fast to rebuild, easy to operate and predictable: a pinned base image, dependencies installed from the lock file in their own layer, only the files the application needs, a non-root user, a healthcheck, metadata labels, and an exec-form command so the application receives the stop signal. This lesson puts the principles of modules 04–14 together in one file and shows what each one changes.

VISUAL

before/Dockerfile	final/Dockerfile	principle
FROM node:24-alpine	FROM node:24-alpine	pinned, official, minimal base
	ENV NODE_ENV=production	fixed runtime settings
	WORKDIR /app	a home for the app, not /
COPY . .	COPY package.json package-lock.json ./	dependencies first: cached
layer		
RUN npm install	RUN npm ci --omit=dev && npm cache ...	reproducible, no dev deps
	COPY server.js ./	only the files needed (+
.dockerignore)		
	USER node	never root
	EXPOSE 3000	documented port
	HEALTHCHECK ... /health	"running" means "working"
	LABEL org.opencontainers.image....	who, what, where from
CMD sh start.sh	CMD ["node", "server.js"]	exec form: PID 1 gets SIGTERM
(start.sh: echo ...; node server.js)		

COMMANDS

Instruction	Production rule
FROM node:24-alpine	official image, pinned major version, small variant (lesson 023); digest pin: lesson 134
COPY package.json package-lock.json ./ then RUN npm ci	dependencies in their own cached layer, exactly as locked (lesson 039)
--omit=dev, npm cache clean --force	no development tools or caches in the image
.dockerignore	node_modules, .git, .env and build files never enter the build context (lesson 037)
USER node	the application cannot modify its own code or the system (lesson 081)
HEALTHCHECK	Docker knows whether the application answers (lesson 092)
LABEL org.opencontainers.image.*	standard metadata that registries and scanners show (lesson 101)
CMD ["node", "server.js"]	exec form: Node.js is PID 1 and receives SIGTERM (lesson 029)

RECAP

- Pinned minimal base, dependencies from the lock file in their own layer, only the needed files.
- Non-root USER, HEALTHCHECK, OCI labels, no secrets.
- Exec-form CMD: the application is process 1 and receives SIGTERM; handle it and exit cleanly.
- Measure: user, command, health and stop time are all visible with docker inspect, docker top and docker stop.

Full lesson with every command and its real output: [130-production-dockerfile](#)

Lesson 131 · Container design

A container that works is not automatically a container that is easy to run. Production platforms (Docker hosts, Kubernetes, ECS) expect every container to follow a few rules: **one concern per container**, **logs on standard output**, **no important state inside the container**, **configuration from outside**, and a **clean shutdown on SIGTERM**. A container that follows them can be restarted, replaced, scaled and debugged by anyone, with the standard tools.

VISUAL



Principle	Why	Lesson
One concern per container	scale, update and restart each part on its own	013, 062
Logs to stdout/stderr	docker logs and every log collector read exactly that	014, 094
State outside the container	the container can be deleted and recreated at any time	055, 058
Configuration from outside	the same image runs in every environment	059, 133
Handle SIGTERM	stopping or replacing a container drops no requests	130

COMMANDS

Command	What it does
ls -l /var/log/nginx	shows that the log files are links (->) to /dev/stdout and /dev/stderr
docker logs NAME	what the container wrote to its standard output and error
docker top NAME -o pid,args	the processes running in the container, with chosen columns
curl -sI URL \> head -1	request only the headers and print the status line (HTTP/1.1 200 OK)

RECAP

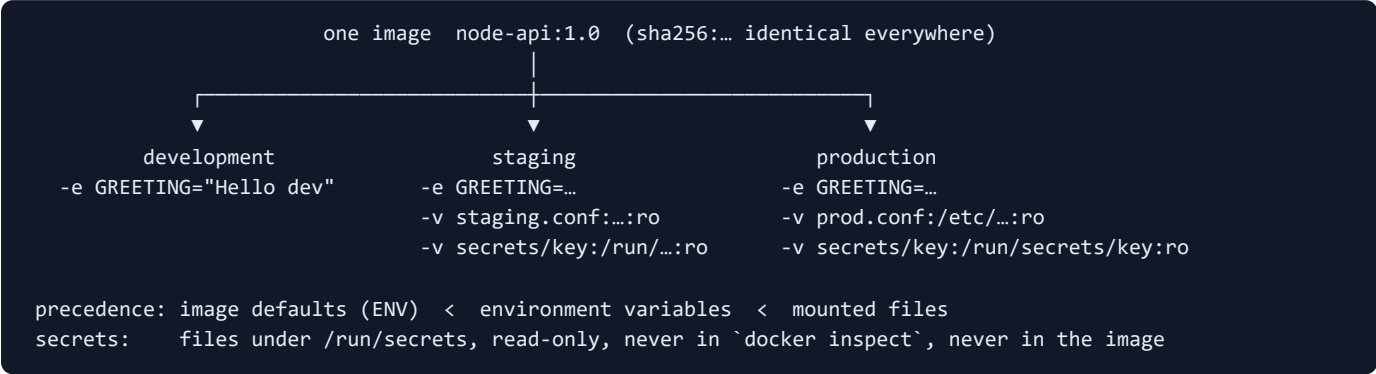
- One concern per container; scale and update the parts separately.
- Logs go to stdout/stderr; docker logs and log collectors read nothing else.
- Keep state in volumes or external services, so any container can be deleted and recreated.
- Configuration comes from outside the image; SIGTERM must lead to a clean exit (lesson 130).

Full lesson with every command and its real output: [131-container-design](#)

Lesson 133 · Production configuration

Build once, configure per environment. The image that passed the tests in staging must be the same bytes that run in production; only its configuration changes. Configuration therefore comes from outside the image, in three forms: **environment variables** for simple values, **configuration files mounted read-only** for structured settings, and **secret files** (never environment variables, never the image) for passwords and keys.

VISUAL



COMMANDS

Option	What it does
<code>-e NAME=value</code>	one environment variable (lesson 059)
<code>-v "\$(pwd)/file:/path/in/container:ro"</code>	mount one file, read-only (:ro)
<code>/run/secrets/NAME</code>	the conventional place for secret files (Compose and Swarm use it)
<code>docker inspect --format '{{.Config.Env}}'</code>	every environment variable of a container, readable by any Docker user
<code>curl -sI</code>	request only the response headers

RECAP

- Build once, configure per environment: the image never contains environment-specific values.
- Environment variables for simple settings; read-only mounted files for structured configuration.
- Secrets as files under `/run/secrets`, not as environment variables (visible in `docker inspect`) and never in the image.
- Verify the configuration the application actually loaded (`nginx -T`, a config endpoint), not just that it runs.

Full lesson with every command and its real output: [133-production-configuration](#)

Lesson 134 · Security review

Before an image and its deployment go to production, someone reviews them against a checklist, and proves every finding with a command instead of guessing from the file. In this lesson you review a real Dockerfile and Compose file that "work", find nine problems, check the hardened version, and see a classic side effect of hardening: a read-only container that can no longer start.

VISUAL

Review checklist	how to prove it
IMAGE 1 base image pinned, official, minimal 2 runs as a non-root user 3 no secrets in ENV / ARG / layers 4 no tools the app does not need 5 only the needed files (.dockerignore) 6 exec-form CMD, HEALTHCHECK { {.Config.Healthcheck}}'	FROM line; docker image ls docker image inspect --format '{{.Config.User}}' docker image inspect (Env), docker history --no-trunc docker run --rm IMG sh -c 'command -v curl bash' docker run --rm IMG ls docker image inspect --format '{{.Config.Cmd}}'
RUN 7 not privileged, no Docker socket 8 only the needed ports published 9 secrets as files, resource limits, read-only file system, dropped capabilities	docker compose config grep docker compose config grep published docker compose config

COMMANDS

Command	What it proves
docker image inspect --format '{{.Config.User}}'	empty = root
docker image inspect --format '{{range .Config.Env}}...'	every environment variable baked into the image
docker history --no-trunc --format '{{.CreatedBy}}'	the full instruction of every layer, including ARG values used in RUN
docker run --rm IMG sh -c 'command -v ...'	which tools exist in the image
docker compose -f FILE config	the final, resolved deployment that Compose would run

RECAP

- Review with a checklist and prove every finding with a command (inspect , history , run , compose config).
- Common findings: root user, secrets in ENV/ARG, extra tools, privileged mode, the Docker socket, published database ports, no limits.
- Harden with USER , secret files, read_only , cap_drop: [ALL] , no-new-privileges , resource limits.
- Hardening can break applications that write files: give them tmpfs exactly where they need it.

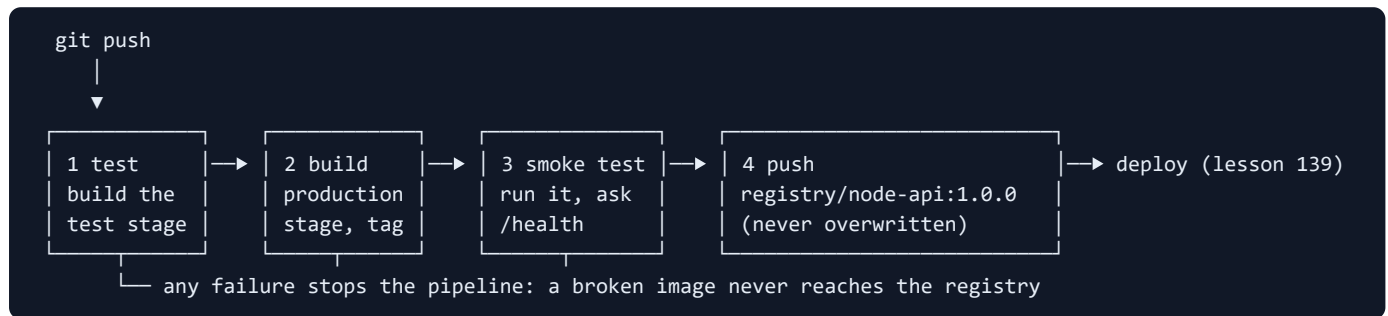
Full lesson with every command and its real output: [134-security-review](#)

Module 19 · Docker in CI/CD

Lesson 135 · Docker in CI

In a team, nobody builds production images on a laptop. A **CI pipeline** does it on every change, always the same way: **test** the code, **build** the image, **smoke-test** the container, and only then **push** it to a registry with an **immutable tag** (the version or the Git commit). If any step fails, nothing is pushed. This lesson runs exactly those steps with a small script you can read in a minute; lesson 136 runs the same steps in GitHub Actions.

VISUAL



One Dockerfile, three stages: `base` (dependencies and code), `test` (`base` + the tests, which run during the build) and `production` (`base` + a non-root user). The tests never ship.

COMMANDS

Command	Pipeline step
<code>docker build --target test</code>	build up to the <code>test</code> stage: its <code>RUN node --test</code> runs the tests
<code>docker build --target production -t REGISTRY/node-api:VERSION</code>	the image that ships, tagged with its version
<code>docker run -d + docker exec ... /health</code>	the smoke test: does the container start and answer?
<code>docker push</code>	publish; reached only if every earlier step passed (<code>set -e</code>)
<code>curl .../v2/node-api/tags/list</code>	the registry's API: which tags exist (lesson 078)

RECAP

- A Docker pipeline: test → build → smoke test → push, and a failing step stops everything.
- A multi-stage Dockerfile gives the tests their own stage; the shipped image contains no tests.
- Tag with immutable versions (version, commit); deploy by tag or, even safer, by digest.
- The same steps run on a laptop (`ci.sh`) and in CI (lesson 136).

Full lesson with every command and its real output: [135-docker-in-ci](#)

Lesson 136 · Build and push with GitHub Actions

How the pipeline of lesson 135 looks in a real CI system: a GitHub Actions workflow that tests and builds the image on every pull request, and pushes it to the **GitHub Container Registry** (`ghcr.io`) on every push to `main` and every version tag. The workflow logs in with the job's own short-lived `GITHUB_TOKEN` , so no password is stored anywhere. This lesson keeps the GitHub Actions details to what Docker needs; the workflow syntax itself belongs to a CI course.

VISUAL

```
.github/workflows/docker-image.yml
```

```
pull request → checkout → buildx → build --target test → build --target production      (no push)
push to main →      "      "      "      "      + login ghcr.io → push
tag v1.4.0   →      "      "      "      "      + login ghcr.io → push
```

```
tags written (docker/metadata-action):
```

```
ghcr.io/OWNER/node-api:sha-3f9c2a1e...  every push: the exact commit
ghcr.io/OWNER/node-api:main              the branch (moves with each push)
ghcr.io/OWNER/node-api:1.4.0             a release tag v1.4.0 (never moves)
```

```
GITHUB_TOKEN + "permissions: packages: write" = the job may push to ghcr.io/OWNER/*
```

Workflow step	Same as in <code>ci.sh</code> (lesson 135)
<code>docker/build-push-action</code> with <code>target: test</code>	<code>docker build --target test .</code>
<code>docker/metadata-action</code>	choosing the tags (<code>VERSION</code>)
<code>docker/login-action</code> with <code>GITHUB_TOKEN</code>	<code>docker login ghcr.io</code>
<code>docker/build-push-action</code> with <code>target: production</code> , <code>push: true</code>	<code>docker build --target production -t ... && docker push ...</code>
<code>cache-from/cache-to: type=gha</code>	the local build cache, stored between CI runs

COMMANDS

Workflow element	What it does
<code>on: push / pull_request</code>	when the workflow runs; pull requests build and test but never push
<code>permissions: packages: write</code>	allows the job's <code>GITHUB_TOKEN</code> to push packages (images)
<code>docker/login-action</code> + <code>secrets.GITHUB_TOKEN</code>	log in to <code>ghcr.io</code> with a token valid only for this job
<code>images: ghcr.io/\${{ github.repository_owner }}</code> <code>}}/node-api</code>	the image name: registry host / owner / name
<code>type=sha,format=long</code>	tag <code>sha-<full commit SHA></code>
<code>push: \${{ github.event_name != 'pull_request' }}</code>	push only for pushes and tags

On your own GitHub repository, the push is the only manual step: commit this folder, push, and watch the **Actions** tab:

```
git remote add origin https://github.com/YOUR-USER/node-api.git
git push -u origin main
gh run watch                # follow the workflow (GitHub CLI)
docker pull ghcr.io/YOUR-USER/node-api:main
```

RECAP

- GitHub Actions runs the same Docker steps as a local script: test stage, build, login, push.
- `GITHUB_TOKEN` with `permissions: packages: write` pushes to `ghcr.io/OWNER/...`; no stored password.
- Pin actions to commit SHAs; tag images with the commit, the branch and the release version.
- The image name chooses the registry: no host means Docker Hub.

Full lesson with every command and its real output: [136-build-and-push-with-github-actions](#)

Lesson 137 · Scanning and testing in CI

Unit tests prove the code works. Before an image is pushed, a pipeline should also prove the **image** is acceptable: it starts and answers (**smoke test**), it follows the team's rules (**policy gate**: non-root, healthcheck, no secrets, a size budget, a version tag), and it has no known serious vulnerabilities (**vulnerability scan**). Each gate is a command that exits non-zero when the image fails it, so the pipeline stops before the push.

VISUAL

```
build → unit tests → smoke test → policy gate → vulnerability scan → push
      (test stage)  (it starts,  image-policy.sh:  trivy / docker scout
                   /health = ok) version tag · non-root · known CVEs in OS packages
                                healthcheck · no ENV secrets and libraries, HIGH/CRITICAL
                                · size budget                  with a fix → fail
exit ≠ 0 at any gate → pipeline stops, nothing pushed
```

COMMANDS

Gate	Command	Fails when
unit tests	<code>docker build --target test</code>	a test fails
smoke test	<code>docker run -d + docker inspect --format '{{.State.Health.Status}}'</code>	not <code>healthy</code> in time
policy	<code>bash image-policy.sh IMAGE</code>	root, no healthcheck, secret-like ENV, too big, <code>latest</code>
vulnerabilities	<code>trivy image --exit-code 1 IMAGE</code>	known HIGH/CRITICAL vulnerabilities with a fix

RECAP

- Gate the image, not only the code: smoke test, policy, vulnerability scan.
- Each gate is a command with a non-zero exit code on failure, so the pipeline stops before the push.
- A policy gate is a few `docker image inspect` queries: user, healthcheck, environment, size, tag.
- Scanners (Trivy, Docker Scout) find known vulnerabilities in OS packages and libraries; run them in CI.

Full lesson with every command and its real output: [137-scanning-and-testing-in-ci](#)

Module 20 · Docker and Kubernetes

Lesson 138 · From Docker to Kubernetes

Docker runs containers on one machine, when you tell it to. **Kubernetes** runs the same images on a cluster of machines and keeps them running: you describe the desired state in YAML ("two copies of `go-api:1.0`, 64 MiB each, reachable as `go-api`"), and Kubernetes creates, restarts, replaces and spreads the containers to match. Almost every `docker run` option you learned has a direct equivalent in a Kubernetes manifest. This lesson maps them, and creates a local cluster with **kind** (Kubernetes in Docker), whose "machines" are Docker containers.

VISUAL

```
docker run -d --name go-api -p 8080:8080 -e PORT=8080 --memory 64m --cpus 0.25 --restart always go-api:1.0
```

Deployment "go-api"

└ Pod template

└ container

└ image: go-api:1.0

└ env: PORT=8080

└ ports: containerPort 8080

└ resources.limits

└ readinessProbe

Service "go-api" port 80 → 8080

restartPolicy: Always (default)

← --name (and how many copies: replicas)

← the image, unchanged

← -e

← EXPOSE (documentation, like in Docker)

← --memory, --cpus

← HEALTHCHECK

← a user-defined network alias + load balancing (not -p)

← --restart always

kind cluster: each Kubernetes "node" is a Docker container running kubelet + containerd

└ Docker

└ container docker-course-control-plane

→ Pods (run by containerd inside it)

docker run	Kubernetes (Pod spec)
image go-api:1.0	containers[].image
--name	metadata.name (Pods get generated names)
-e NAME=value, --env-file	env, envFrom (ConfigMap, Secret)
-p 8080:8080	a Service (and <code>kubectl port-forward</code> for testing)
-v volume:/path	volumes + volumeMounts (PersistentVolumeClaim)
--memory, --cpus	resources.limits.memory, resources.limits.cpu
HEALTHCHECK	readinessProbe, livenessProbe
--user, --read-only, --cap-drop	securityContext
--restart	restartPolicy (Deployments: always)

COMMANDS

Command	What it does
<code>kubectl create deployment ... --dry-run=client -o yaml</code>	print a manifest instead of creating anything
<code>kind create cluster --name NAME</code>	start a local cluster in Docker containers, and write its kubeconfig
<code>kubectl get nodes</code>	the cluster's machines and whether they are <code>Ready</code>
<code>kind delete cluster --name NAME</code>	remove the cluster and its containers

RECAP

- Kubernetes runs the same OCI images as Docker, on many machines, and keeps the desired state.
- `docker run` options map to Pod spec fields: image, env, ports, resources, probes, securityContext.
- `-p` has no direct equivalent: Services give Pods a stable address; `port-forward` is for testing.
- kind runs a whole cluster as Docker containers; `kubectl` reads its target from the kubeconfig.

Full lesson with every command and its real output: [138-from-docker-to-kubernetes](#)

Lesson 139 · Deploy an image to kind

The complete path from source code to a running Kubernetes workload: **build** the Go API's image with Docker (the multi-stage Dockerfile of lesson 089), **load** it into a kind cluster (in place of pushing it to a registry the cluster can pull from), **deploy** it with a Deployment and a Service, and **reach** it with `kubectl port-forward`. Then the most common first failure: a Pod stuck in `ImagePullBackOff` because of the `latest` tag.

VISUAL

```
examples/go-api —docker build→ go-api:1.0 (local Docker engine)
                                | kind load docker-image go-api:1.0
                                v
┌────────── kind node (a Docker container) ───────────┐
│ containerd image store: go-api:1.0                    │
│                                                       │
│ Deployment go-api → ReplicaSet → Pod go-api-...-abcde  (8080)
│                               ↳ Pod go-api-...-fghij    (8080)
│ Service go-api :80 ————— selects app=go-api → both Pods :8080
│                                                       │
│ ▲ kubectl port-forward service/go-api 8090:80
│ curl localhost:8090
│
│ imagePullPolicy: tag :latest (or none) → Always (must pull from a registry)
│                  any other tag         → IfNotPresent (a loaded image is used)
```

COMMANDS

Command	What it does
<code>kind load docker-image IMAGE --name CLUSTER</code>	copy an image from the Docker engine into every node of the cluster
<code>kubectl apply -f FILE</code>	create or update the objects described in the file
<code>kubectl rollout status deployment/NAME</code>	wait until the new Pods are ready (or time out)
<code>kubectl get pods -l app=go-api</code>	the Pods with that label
<code>kubectl port-forward service/NAME LOCAL:PORT</code>	forward a local port to the Service, for testing
<code>kubectl logs deployment/NAME</code>	the logs of one of the Deployment's Pods (like <code>docker logs</code>)

RECAP

- Build with Docker, load into kind (or push to a registry), deploy with a Deployment and a Service.
- `kubectl port-forward` reaches a Service from your computer for testing.
- `ImagePullBackOff`: the kubelet could not pull; `kubectl describe pod` shows why.
- `:latest` means `imagePullPolicy: Always`: deploy immutable version tags instead.
- Rolling updates and `rollout undo` work because every version has its own tag.

Full lesson with every command and its real output: [139-deploy-an-image-to-kind](#)

Lesson 140 · Kubernetes for Docker users

Your Docker troubleshooting habits carry over to Kubernetes almost one to one: `docker ps` becomes `kubectl get pods`, `docker logs` becomes `kubectl logs`, `docker inspect` becomes `kubectl describe`, a throwaway `docker run --rm` on a network becomes `kubectl run --rm`. This lesson uses them on a running application, then debugs the most common Kubernetes-only failure: a **readiness probe** that never succeeds, which Kubernetes handles in a way Docker never does.

VISUAL

Docker	Kubernetes
<code>docker ps</code>	<code>kubectl get pods (-o wide: node, IP)</code>
<code>docker logs NAME (-f, --tail)</code>	<code>kubectl logs POD deployment/NAME -l app=NAME --prefix</code>
<code>docker exec -it NAME sh</code>	<code>kubectl exec -it POD -- sh</code>
<code>docker inspect NAME</code>	<code>kubectl describe pod POD • kubectl get pod POD -o yaml</code>
<code>docker events</code>	<code>kubectl get events --sort-by=.lastTimestamp</code>
<code>docker run --rm --network net IMG CMD</code>	<code>kubectl run tmp --rm -i --restart=Never --image=IMG -- CMD</code>
<code>docker rm -f NAME</code>	<code>kubectl delete pod POD (the Deployment creates a new one)</code>
HEALTHCHECK → "unhealthy" (still gets traffic)	readinessProbe fails → Pod removed from the Service
	livenessProbe fails → container restarted
rolling update with a broken probe:	
old Pods (ready) —keep serving→ Service	new Pod (0/1 READY) —never receives traffic

COMMANDS

Command	What it does
<code>kubectl get pods -o wide</code>	Pods with their node and IP
<code>kubectl logs -l app=go-api --prefix</code>	logs of all Pods with that label, each line prefixed with its Pod
<code>kubectl run NAME --rm -i --restart=Never --image=IMG -- CMD</code>	a one-off Pod that runs a command and is deleted
<code>kubectl describe pod POD</code>	configuration, state and events of a Pod (start here when something fails)
<code>kubectl rollout undo deployment/NAME</code>	return to the previous version of the Pod template

RECAP

- `get pods`, `logs`, `exec`, `describe`, `get events`, `run --rm`: the Docker troubleshooting toolkit, in Kubernetes.
- Deleting a Pod is not like `docker rm`: its Deployment replaces it.
- A failing readiness probe keeps a Pod out of the Service and stops a rolling update: old Pods keep serving.
- `kubectl rollout undo` returns to the previous working version; then fix the manifest.

Full lesson with every command and its real output: [140-kubernetes-for-docker-users](#)

Module 17 · Troubleshooting

25 problems · about 400 minutes · all levels · run every command from the course folder

Every problem here is reproduced for real on your computer, then worked through the way you would at work:

problem → **symptoms** → **investigation** → **commands** → **output interpretation** → **root cause** → **fix** → **verification** → **prevention**. Start from the symptom you see.

The method

1. What state is it in?	<code>docker ps -a</code>	Up? Exited (CODE)? Created? (unhealthy)?
2. What did it say?	<code>docker logs NAME</code>	the application's own error
3. How is it configured? user, limits	<code>docker inspect NAME</code>	command, env, ports, mounts, networks,
4. What does it see from inside?	<code>docker exec NAME ...</code>	files, processes, DNS, the app on localhost
5. What happened over time?	<code>docker events --since 10m</code>	die, oom, kill, health_status
6. Is it starved? limit	<code>docker stats --no-stream</code>	CPU % against its limit, memory usage /
7. Change one thing, verify, and write down the prevention.		

Command	Answers
<code>docker ps -a --format '{{.Names}}: {{.Status}}'</code>	is it running, how did it end, is it healthy
<code>docker logs NAME 2>&1 \ tail</code>	what the process printed, also after it stopped
<code>docker inspect --format '{{.State.ExitCode}} {{.State.OOMKilled}} {{.State.Error}}' NAME</code>	why it stopped
<code>docker inspect --format '{{json .Config}}' NAME</code>	the command, environment, user and ports it really got
<code>docker inspect --format '{{json .Mounts}}' NAME</code>	which volumes and folders are mounted where
<code>docker exec NAME sh</code>	look around inside a running container
<code>docker events --since 10m --filter container=NAME</code>	the container's history: start, die, oom, health
<code>docker stats --no-stream</code>	live CPU and memory use against the limits

Exit codes: `0` the main process finished · `1` (or another small number) the application failed · `125` Docker could not create or start it · `126` the command is not executable · `127` the command was not found · `137` killed (`SIGKILL` : out of memory or `docker kill`) · `143` stopped with `SIGTERM` .

Problems by symptom

Symptom	Problem
Exited (0) right after <code>docker run -d</code> , clean logs	01 · Container exits immediately
executable file not found in <code>\$PATH</code> , status <code>Created</code> , exit 127	02 · Wrong command
it runs, but the old version	03 · Wrong image
port is already allocated	04 · Port already in use
<code>curl: (7)</code> connection refused / could not connect	05 · Application not reachable
<code>curl: (52) Empty reply</code> , works inside the container	06 · Listening on localhost
<code>curl: (52) Empty reply</code> , image <code>EXPOSE</code> s another port	07 · Wrong container port
Name does not resolve / Try again between networks	08 · Wrong network
a container name does not resolve, its IP works	09 · DNS failure
Connection refused / password authentication failed to the database	10 · Database connection failure
<code>KeyError</code> , <code>undefined</code> , <code>null</code> for configuration, exit 1	11 · Missing environment variable
Permission denied reading a file, non-root user	12 · Permission denied
Permission denied / Read-only file system writing	13 · Cannot write files
data gone after recreating, volume empty	14 · Volume not mounted
database empty after recreating, no volume given	15 · Data disappeared
<code>COPY</code> fails with <code>not found</code> although the file exists	16 · Build failure
a rebuild still contains old content, <code>CACHED</code>	17 · Unexpected cache
hundreds of MB for a small application	18 · Huge image
a token readable with <code>docker history</code>	19 · Dockerfile security issue
<code>uid=0(root)</code> in the container	20 · Running as root
<code>(unhealthy)</code> although the app answers	21 · Healthcheck failure
Exited (137), <code>OOMKilled: true</code> , no logs	22 · Memory limit exceeded
slow, no errors, <code>nr_throttled</code> growing	23 · CPU throttling
no basic auth credentials, <code>401 Unauthorized</code>	24 · Registry authentication failure
pull access denied, manifest unknown, <code>429 Too Many Requests</code>	25 · Image pull failure

How to work through a problem

1. Run the **Symptoms** commands and look at the real output before reading further: what would you check first?
2. Follow the **Investigation**; every step is one question and one command.
3. Apply the **Fix**, run the **Verification**, and read the **Prevention**: that is what stops it from happening again.
4. Run the **Cleanup**. Every problem also starts from scratch, so you can repeat it.

Module 21 · Projects

8 projects · about 9 hours · run every command from the course folder

Each project applies the course to one complete application. It states a goal and a requirements checklist, shows the architecture, builds the solution step by step, **verifies every requirement with a command**, and breaks one thing the way it breaks in real teams. The finished files are in each project's `solution/` folder: try to write your own first, then compare.

#	Project	Skills	Time
01	Static website on Nginx	custom <code>nginx.conf</code> , non-root, <code>HEALTHCHECK</code> , <code>--read-only</code> + <code>--tmpfs</code>	45 min
02	Production Node.js API	<code>npm ci</code> , multi-stage, non-root, exec-form <code>CMD</code> , graceful <code>SIGTERM</code>	1 h
03	Python API with Redis	Gunicorn, Compose, service DNS, <code>depends_on</code> + healthchecks, named volume	1 h
04	Go API, as small as it gets	static binaries, distroless vs <code>scratch</code> , measured sizes, shell vs exec form	45 min
05	Java API on a JRE	JDK → JRE multi-stage, executable jars, container-aware heap	1 h
06	A database container you can back up	init scripts, volumes, <code>pg_dump</code> / <code>pg_restore</code> , restore tests	1 h
07	Full-stack application with Compose	three tiers, internal network, <code>.env</code> configuration, credential changes	1.5 h
08	Four stacks behind one proxy	path routing, four languages, limits, failure isolation, <code>nginx -s reload</code>	2 h

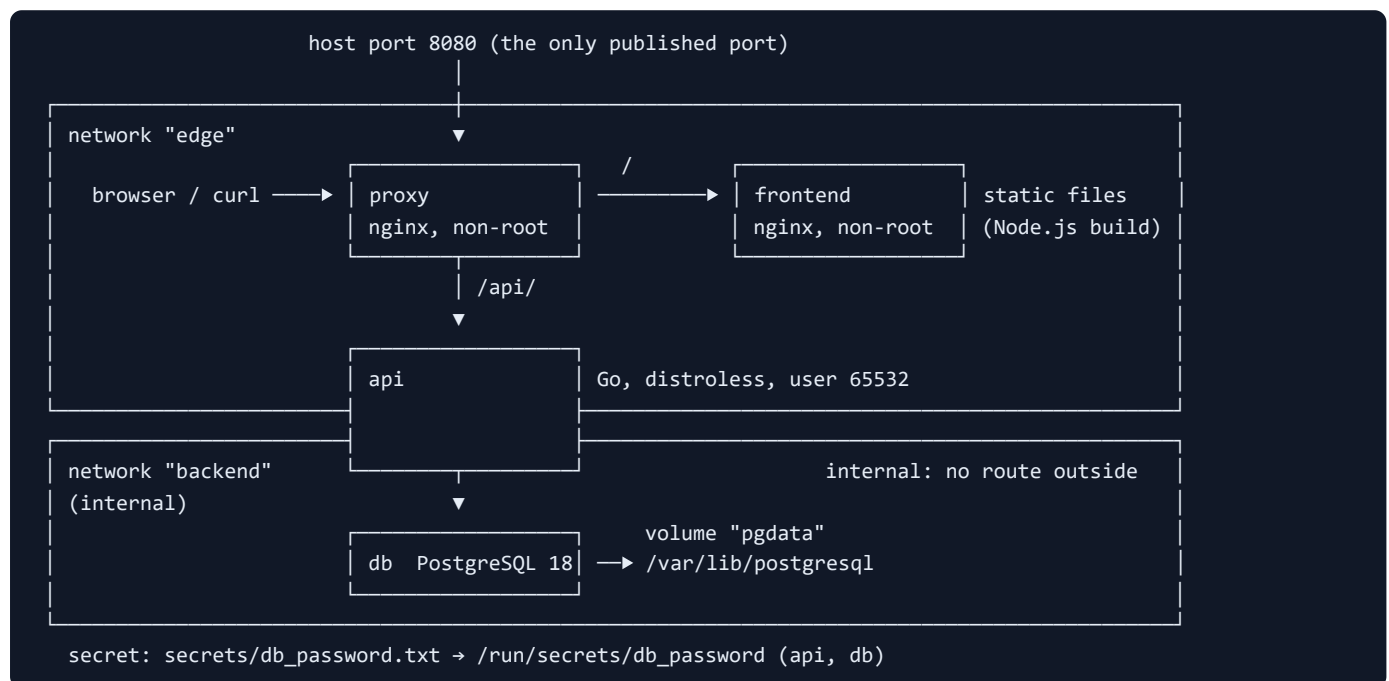
Every project starts from a clean lab (`bash scripts/lab.sh project-NN ...`) and ends with a *Cleanup* section that removes its containers, images, networks and volumes.

Capstone · a production-style container stack

Final project · ⌚ 3 hours · run every command from the course folder

You build, run, harden, verify, break and ship a complete application: a cafe ordering service with a reverse proxy, a frontend, an API and a PostgreSQL database. Every technique of the course is used once, on purpose, and every one is **verified with a command**, not assumed. The finished files are in [app/](#); the walkthrough builds them up and checks them one by one.

Architecture



Requirements and where they are met

Requirement	Where	Verified in step
Multi-stage builds	<code>api/Dockerfile</code> (Go → distroless), <code>frontend/Dockerfile</code> (Node.js → Nginx)	2
Custom networks	<code>edge</code> and an internal <code>backend</code> in <code>compose.yaml</code>	6
Environment configuration	<code>.env.example</code> → <code>.env</code> ; <code>environment:</code> per service	1, 3
Secrets	<code>secrets:</code> from a file, read by the API and PostgreSQL as <code>/run/secrets/db_password</code>	1, 5
Healthchecks	every service; <code>depends_on:</code> <code>condition: service_healthy</code>	4
Volumes	named volume <code>pgdata</code>	7
Resource limits	<code>deploy.resources.limits</code> (CPU, memory, processes)	8
Logging	<code>json-file</code> with rotation (<code>max-size</code> , <code>max-file</code>); JSON logs from the API	8
Security	non-root users, read-only root file systems, <code>cap_drop: [ALL]</code> , <code>no-new-privileges</code> , no shell in the API	5

Requirement	Where	Verified in step
Optimization	distroless API image, layer order for cache, <code>.dockerignore</code>	2
Docker Compose	the whole stack is one <code>compose.yaml</code>	3
Registry	push to a local registry and pull back by digest	10
CI	.github/workflows/capstone.yml builds, tests and pushes to GHCR	11
Documentation	this walkthrough and the comments in every file	

Step 1 · Lab and configuration

```
bash scripts/lab.sh capstone 22-capstone/app
cd ~/docker-practice/capstone
./make-secret.sh
cp .env.example .env
```

Configuration (`.env`) and the secret (`secrets/db_password.txt`) are separate on purpose: `.env` can be shown to anyone, the secret cannot. Check that the Compose file is valid and that the password does not appear in the configuration Compose resolves:

```
docker compose config --quiet && echo "compose file valid"
docker compose config | grep -E "PASSWORD|db_password" | sed 's/^ *//' | sort -u
```

Only the password **file's path** appears, never its content.

Step 2 · Build the images

```
docker compose build --quiet
docker image ls --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep capstone/
```

Compare the API image with the image it was built in:

```
docker image ls --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep -E "capstone/api|^golang:1.26-alpine"
```

```
localhost:5000/capstone/api:1.0.0 21.3MB
golang:1.26-alpine 364MB
```

The Go toolchain, the source code and the module cache stayed in the build stage. The final image is the static binary on distroless: no shell, no package manager, nothing to attack and nothing to patch except the binary.

Step 3 · Start the stack

`--wait` returns only when every service with a healthcheck is **healthy**, not merely running:

```
docker compose up -d --wait 2>&1 | tail -4
```

```
docker compose ps --format 'table {{.Service}}\t{{.Status}}\t{{.Ports}}'
```

SERVICE	STATUS	PORTS
api	Up 16 seconds (healthy)	8080/tcp
db	Up 22 seconds (healthy)	5432/tcp
frontend	Up 22 seconds (healthy)	80/tcp, 8080/tcp
proxy	Up 10 seconds (healthy)	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp

Only the proxy publishes a port. Use the application through it:

```
curl -s http://localhost:8080/api/health
```

```
curl -s -X POST -H "Content-Type: application/json" -d '{"item": "flat white"}'
http://localhost:8080/api/orders
echo
curl -s -X POST -H "Content-Type: application/json" -d '{"item": "cortado"}' http://localhost:8080/api/orders
echo
curl -s http://localhost:8080/api/orders
```

```
{"id":1,"item":"flat white","created":"2026-10-05T17:21:59.471306Z"}

{"id":2,"item":"cortado","created":"2026-10-05T17:21:59.53398Z"}

[{"id":2,"item":"cortado","created":"2026-10-05T17:21:59.53398Z"}, {"id":1,"item":"flat white","created":"2026-10-05T17:21:59.471306Z"}]
```

```
curl -s http://localhost:8080/ | grep "<title>"
```

Open <http://localhost:8080> in a browser to use the page.

Step 4 · Health, not just "running"

```
for s in proxy frontend api db; do
  echo "$s: $(docker inspect --format '{{.State.Health.Status}}' "$(docker compose ps -q $s)")"
done
```

```
proxy: healthy
frontend: healthy
api: healthy
db: healthy
```

The API's healthcheck (`/api -healthcheck`) queries `/api/health` , which pings the database: a running API with a lost database reports **unhealthy** (step 9 proves it).

Step 5 · Security, verified

Users, read-only file systems and capabilities of every container:

```
for s in proxy frontend api db; do
  docker inspect --format "$s: user={{.Config.User}} read-only={{.HostConfig.ReadonlyRootfs}} cap_drop={{.HostConfig.CapDrop}} cap_add={{.HostConfig.CapAdd}}" "$(docker compose ps -q $s)"
done
```

```
proxy: user=nginx read-only=true cap_drop=[ALL] cap_add=[]
frontend: user=nginx read-only=true cap_drop=[ALL] cap_add=[]
api: user=nonroot:nonroot read-only=true cap_drop=[ALL] cap_add=[]
db: user= read-only=true cap_drop=[ALL] cap_add=[CAP_CHOWN CAP_DAC_OVERRIDE CAP_FOWNER CAP_SETGID CAP_SETUID]
```

PostgreSQL's entrypoint starts as root to prepare the data folder, then switches to the `postgres` user: it keeps only the five capabilities that needs. Prove the restrictions hold:

```
docker compose exec proxy sh -c 'echo hacked > /usr/share/nginx/html/index.html' 2>&1 || true
docker compose exec proxy id
```

```
sh: can't create /usr/share/nginx/html/index.html: Read-only file system
uid=101(nginx) gid=101(nginx) groups=101(nginx)
```

```
docker compose exec api sh 2>&1 || true
```

```
OCI runtime exec failed: exec failed: unable to start container process: exec: "sh": executable file not found in $PATH
```

The API image has no shell at all. The secret is a file inside the container, and not in the environment:

```
docker inspect --format '{{range .Config.Env}}{{println .}}{{end}}' "$(docker compose ps -q api)" | grep -qi "password=" \
    && echo "password in the environment!" || echo "no password in the environment"
docker compose exec db sh -c 'test -s /run/secrets/db_password && echo "secret mounted at /run/secrets/db_password"'
```

Step 6 · Networks

```
docker network ls --filter name=capstone --format '{{.Name}} internal={{.Internal}}'
for s in proxy api db; do
    echo "$s: $(docker inspect --format '{{range $n, $_ := .NetworkSettings.Networks}}{{$n}} {{end}}' "$(docker compose ps -q $s)")"
done
```

```
capstone_backend internal=true
capstone_edge internal=false
proxy: capstone_edge
api: capstone_backend capstone_edge
db: capstone_backend
```

The proxy cannot even resolve the database's name: it is not on the `backend` network.

```
docker compose exec proxy nslookup db 2>&1 | tail -2
```

```
** server can't find db: NXDOMAIN
```

Step 7 · Persistence

Recreate every container (`down` without `-v` keeps the volume), and the orders are still there:

```
docker compose down 2>&1 | tail -1
docker compose up -d --wait 2>&1 | tail -1
curl -s http://localhost:8080/api/orders
```

```
docker volume ls --filter name=capstone
```

Step 8 · Limits and logs

```
docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.PIDs}}'
```

NAME	CPU %	MEM USAGE / LIMIT	PIDS
capstone-proxy-1	0.00%	11.11MiB / 64MiB	15
capstone-api-1	0.00%	2.129MiB / 64MiB	7
capstone-db-1	3.22%	27.55MiB / 256MiB	10
capstone-frontend-1	0.00%	11.14MiB / 32MiB	15

Every container has a memory limit (/ 64MiB), so one leaking service cannot take the whole host's memory. Logs rotate, and the API logs structured JSON:

```
docker inspect --format '{{.HostConfig.LogConfig.Type}} {{.HostConfig.LogConfig.Config}}' "$(docker compose ps -q api)"
docker compose logs api --no-log-prefix 2>&1 | tail -2
```

Step 9 · Break it, troubleshoot it, fix it

Stop the database while everything else keeps running:

```
docker compose stop db 2>&1 | tail -1
curl -s http://localhost:8080/api/health
```

The proxy and the API still run, but the API answers `503 database unavailable`, and within about 30 seconds (three failed checks, 10 s apart) Docker marks it **unhealthy**:

```
docker inspect --format '{{.State.Health.Status}}' "$(docker compose ps -q api)"
```

Investigate the way module 17 teaches: state, then logs, then the dependency.

```
docker compose ps -a --format '{{.Service}}: {{.Status}}'
docker inspect --format '{{range .State.Health.Log}}{{.ExitCode}} {{.Output}}{{end}}' "$(docker compose ps -q api)" | tail -2
```

The `db` service is `Exited`. Fix it, and the API recovers by itself (its connection pool reconnects):

```
docker compose start db 2>&1 | tail -1
curl -s http://localhost:8080/api/health
```

Step 10 · Ship the images to a registry

A local registry stands in for Docker Hub, GHCR or ECR (module 11). The image names in `compose.yaml` already start with `$_REGISTRY`, so pushing is one command:

```
docker run -d --name registry -p 5000:5000 registry:3 > /dev/null
sleep 2
docker compose push --quiet 2>&1 | tail -2
echo "pushed"
```

```
curl -s http://localhost:5000/v2/_catalog
```

```
{"repositories":["capstone/api","capstone/frontend","capstone/proxy"]}
```

Deploy by **digest**, not by tag: a digest names exactly one image forever. Remove the local copy, pull it back by digest, and compare:

```
digest=$(docker image inspect --format '{{index .RepoDigests 0}}' localhost:5000/capstone/api:1.0.0)
echo "$digest"
docker image rm -f localhost:5000/capstone/api:1.0.0 > /dev/null
docker pull -q "$digest" > /dev/null
[ "$(docker image inspect --format '{{index .RepoDigests 0}}' "$digest")" = "$digest" ] && echo "same image,
pulled by digest"
```

Step 11 · Continuous integration

[.github/workflows/capstone.yml](#) runs on every push of this course: it builds the three images, starts the stack with `docker compose up --wait`, runs the smoke tests of step 3, and on the `main` branch pushes the images to GitHub Container Registry with the workflow's own `GITHUB_TOKEN` (no stored password). Module 19 explains each step.

Cleanup

```
cd ~/docker-practice/capstone
docker compose down -v --rm local 2>&1 | tail -1
docker rm -f registry > /dev/null
docker image rm -f localhost:5000/capstone/api:1.0.0 localhost:5000/capstone/frontend:1.0.0
localhost:5000/capstone/proxy:1.0.0 > /dev/null 2>&1 || true
docker image ls --format '{{.Repository}}' | grep -q capstone && echo "images left" || echo "capstone images
removed"
cd ~ && rm -rf ~/docker-practice/capstone
```

Then take the [final exam](#).

Command reference

Every Docker command this course runs, grouped by what it works on. For each one: what it does, the flags the course uses, and where it is used. The **first** link is the lesson that teaches it; the others show it in use. Numbers are lessons ([012](#)), [T07](#) a troubleshooting problem, [P03](#) a project, [A10](#) a module assessment.

Every command accepts `--help`: `docker run --help`, `docker compose up --help`.

Contents: [Containers](#) · [Inspecting and debugging](#) · [Images](#) · [Builds](#) · [Networks](#) · [Volumes](#) · [Docker Compose](#) · [Registries](#) · [Security](#) · [Engine and system](#)

Containers

Command	What it does	Flags used in the course	Where
<code>docker run</code>	create and start a container from an image	<code>--name</code> give the container a name, <code>--rm</code> remove the container when it stops, <code>-d</code> run in the background (detached), <code>-p</code> publish HOST:CONTAINER port, <code>-v</code> mount a volume or bind mount, <code>--network</code> connect to a network, <code>-e</code> set an environment variable, <code>--entrypoint</code> replace the image's ENTRYPOINT	001 What is Docker? also 002 , 003 , 005 , A01 , 006 , 007 , 008 , 009 and 137 more
<code>docker rm</code>	remove containers	<code>-f</code> stop and remove, <code>--filter</code> filter, <code>-a</code> all, <code>-q</code> quiet / IDs only, <code>-v</code> also remove anonymous volumes	003 Docker vs virtual machines also 006 , 007 , 008 , 009 , 010 , 011 , 012 , 013 and 111 more
<code>docker container create</code>	create a container without starting it	<code>--name</code>	004 Docker architecture
<code>docker container ls</code>	list containers (same as <code>docker ps</code>)	<code>--filter</code> filter, <code>--format</code> Go template output, <code>-a</code> all, <code>-s</code> writable layer size	004 Docker architecture also 019 , 021 , A03 , 030 , 032 , 053
<code>docker container rm</code>	remove containers	<code>--filter</code> filter, <code>-f</code> force, <code>-q</code> quiet / IDs only	004 Docker architecture also 101
<code>docker container start</code>	start stopped containers	<code>-a</code> all	004 Docker architecture
<code>docker ps</code>	list containers (running only, or all with <code>-a</code>)	<code>--filter</code> filter, <code>--format</code> Go template output, <code>-a</code> include stopped containers, <code>-q</code> IDs only, <code>--no-trunc</code> full IDs and commands, <code>-s</code> writable layer size	006 Your first container also 007 , 008 , 010 , 011 , 012 , 013 , 015 , 016 and 51 more
<code>docker container diff</code>	files added (A), changed (C) or deleted (D) in a container's writable layer	—	007 Images vs containers

Command	What it does	Flags used in the course	Where
<code>docker create</code>	create a container without starting it	<code>--name</code>	007 Images vs containers also 008, 009, 042, 082
<code>docker pause</code>	freeze every process of a container	—	009 Container lifecycle
<code>docker restart</code>	stop and start containers	—	009 Container lifecycle also 057, 093
<code>docker start</code>	start stopped containers	<code>-a</code> all, <code>--filter</code> filter, <code>-q</code> quiet / IDs only	009 Container lifecycle also 011, 013, 053, 130, P02
<code>docker stop</code>	stop running containers gracefully (SIGTERM, then SIGKILL after a timeout)	<code>--filter</code> filter, <code>-q</code> quiet / IDs only, <code>-t</code> seconds before SIGKILL	009 Container lifecycle also 011, 013, 019, 040, 050, 096, 130, P02
<code>docker unpause</code>	resume a paused container	—	009 Container lifecycle
<code>docker rename</code>	rename a container	—	010 Container names
<code>docker container inspect</code>	low-level details of containers as JSON	<code>--format</code> Go template output	021 The danger of the latest tag also 028, 029, 100
<code>docker update</code>	change resource limits or the restart policy of a running container	<code>--cpus</code> CPU limit, <code>--memory</code> memory limit, <code>--memory-swap</code>	095 docker stats also 097, 098, T23
<code>docker kill</code>	send a signal (default SIGKILL) to a container's main process	—	099 Memory pressure and OOM kills

Inspecting and debugging

Command	What it does	Flags used in the course	Where
<code>docker cp</code>	copy files between a container and the host	—	007 Images vs containers also 011, 042, exam
<code>docker inspect</code>	low-level details of any object (container, image, network, volume) as JSON	<code>--format</code> Go template output, <code>-q</code> quiet / IDs only	007 Images vs containers also 009, 010, 011, 014, 016, A02, 040, 046 and 63 more
<code>docker logs</code>	a container's standard output and standard error	<code>-t</code> timestamps, <code>--tail</code> last N lines, <code>-f</code> follow, <code>--since</code> only newer than	011 Running Nginx also 014, 016, A02, 030, 032, 037, 041, 042 and 33 more

Command	What it does	Flags used in the course	Where
<code>docker port</code>	a container's published ports	—	012 Port mapping also A02 , 032 , A06 , 051 , 052 , A07 , T04 , T05 and 2 more
<code>docker exec</code>	run a command in a running container	<code>-e</code> set an environment variable, <code>-d</code> in the background, <code>-i</code> keep standard input open, <code>-it</code> interactive terminal	014 Container logs also 015 , A02 , 021 , 030 , 032 , A04 , 040 , 041 and 50 more
<code>docker top</code>	the processes running in a container	<code>-o</code>	041 Containerizing a Python application also 081 , 094 , 095 , T20 , 130 , 131
<code>docker stats</code>	live CPU, memory, network and I/O use of containers	<code>--format</code> Go template output, <code>--no-stream</code> print once	045 Comparing the stacks also 086 , 095 , A14 , 097 , 098 , 099 , T23 , capstone
<code>docker diff</code>	changes in a container's writable layer	—	053 The container's writable layer also 084 , 132
<code>docker events</code>	the engine's real-time event stream (create, start, die, health, ...)	<code>--until</code> until, <code>--since</code> from this time, <code>--filter</code> filter, <code>--format</code> Go template output	096 docker events also A14 , 099 , T22

Images

Command	What it does	Flags used in the course	Where
<code>docker image ls</code>	list images	<code>--format</code> Go template output, <code>--filter</code> filter (dangling=true, ...), <code>--tree</code> , <code>-q</code> quiet / IDs only, <code>--digests</code>	001 What is Docker? also 002 , 003 , 006 , 017 , 018 , 019 , 020 , 021 and 46 more
<code>docker build</code>	build an image from a Dockerfile and a build context	<code>-t</code> name:tag of the image, <code>-q</code> print only the image ID, <code>-f</code> Dockerfile path, <code>--build-arg</code> set a build argument (ARG), <code>-target</code> build up to this stage, <code>--progress</code> build output style (plain shows every step), <code>--check</code> lint the Dockerfile without building, <code>--no-cache</code> ignore the build cache	002 Why containers? also 017 , 019 , 020 , 021 , A03 , 022 , 023 , 024 and 71 more
<code>docker image rm</code>	remove images	<code>-f</code> force, <code>--format</code> Go template output, <code>-q</code> quiet / IDs only	002 Why containers? also 007 , 017 , 018 , 019 , 020 , 021 , A03 , 022 and 88 more
<code>docker image inspect</code>	an image's configuration and layers as JSON	<code>--format</code> Go template output	004 Docker architecture also 017 , 018 , 021 , A03 , 022 , 024 , 027 , 028 and 43 more
<code>docker image pull</code>	download an image from a registry	<code>-q</code> quiet / IDs only	004 Docker architecture also 018

Command	What it does	Flags used in the course	Where
<code>docker commit</code>	create an image from a container's changes (for investigation, not for production)	—	007 Images vs containers
<code>docker image history</code>	the layers of an image and the instruction that created each one	<code>--format</code> Go template output, <code>--no-trunc</code>	017 Image layers also 031
<code>docker image prune</code>	remove dangling images (all unused with -a)	<code>-f</code> force	019 Listing and removing images also 021
<code>docker image tag</code>	give an image an additional name	—	019 Listing and removing images also 020
<code>docker history</code>	the layers of an image and the instruction that created each one	<code>--format</code> Go template output, <code>--no-trunc</code> full instructions	038 Build layers also 083, 091, T18, T19, 134
<code>docker pull</code>	download an image from a registry	<code>-q</code> quiet	075 What is a registry? also 078, A11, T24, T25, 135, 136, capstone
<code>docker push</code>	upload an image to a registry	<code>-q</code> quiet, <code>--all-tags</code>	075 What is a registry? also 076, 077, 078, 079, A11, T24, 136, exam
<code>docker tag</code>	give an image an additional name (repository:tag)	—	075 What is a registry? also 076, 077, 078, 079, A11, T24, T25, 136 and 3 more
<code>docker save</code>	write images to a tar archive	<code>-o</code>	083 Secrets in images
<code>docker image save</code>	write images to a tar archive	<code>-o</code>	T19 Dockerfile security issue: a secret in the image

Builds

Command	What it does	Flags used in the course	Where
<code>docker buildx build</code>	build with BuildKit: multi-platform, cache import/export, --push	<code>-t</code> , <code>-q</code> quiet / IDs only, <code>--load</code> load the result into the engine, <code>--platform</code> target platforms, <code>--target</code> , <code>--output</code> , <code>--progress</code> , <code>--label</code>	103 BuildKit and buildx also 104, A16
<code>docker buildx ls</code>	list builders and the platforms they support	<code>--format</code> Go template output	103 BuildKit and buildx

Command	What it does	Flags used in the course	Where
<code>docker buildx inspect</code>	details of a builder	<code>--bootstrap</code>	104 Multi-architecture images
<code>docker buildx prune</code>	remove the build cache	<code>-f</code> force	104 Multi-architecture images also 105

Networks

Command	What it does	Flags used in the course	Where
<code>docker network create</code>	create a network	<code>--subnet</code> subnet	044 Containerizing a PHP application also 047 , 048 , 049 , 050 , 051 , 052 , A07 , 062 and 6 more
<code>docker network inspect</code>	a network's configuration and attached containers	<code>--format</code> Go template output	044 Containerizing a PHP application also 046 , 047 , 048 , 049 , 065 , T08 , T10
<code>docker network rm</code>	remove networks	—	044 Containerizing a PHP application also 047 , 048 , 049 , 050 , 051 , 052 , A07 , 062 and 7 more
<code>docker network ls</code>	list networks	<code>--filter</code> filter, <code>--format</code> Go template output, <code>-q</code> quiet / IDs only	046 Networking fundamentals also 048 , 062 , 065 , 074 , P07 , capstone
<code>docker network connect</code>	attach a running container to a network	<code>--alias</code>	047 The default bridge network also 048 , 049 , 050 , A07 , T08 , exam
<code>docker network disconnect</code>	detach a container from a network	—	048 Custom networks also 049 , 050

Volumes

Command	What it does	Flags used in the course	Where
<code>docker volume rm</code>	remove volumes (and their data)	—	053 The container's writable layer also 054 , 055 , 056 , 057 , 058 , A08 , 096 , 100 and 5 more
<code>docker volume ls</code>	list volumes	<code>--filter</code> filter, <code>-q</code> quiet / IDs only, <code>-format</code> Go template output	054 Bind mounts also 055 , 056 , 066 , 074 , T15 , capstone
<code>docker volume create</code>	create a named volume	<code>--label</code>	055 Named volumes also 056 , 096 , 100 , T15 , P06 , exam
<code>docker volume inspect</code>	a volume's driver, mount point and labels	<code>--format</code> Go template output	056 Inspecting volumes
<code>docker volume prune</code>	remove unused volumes (and their data)	<code>-f</code> force, <code>-a</code> also named volumes	056 Inspecting volumes also 058 , T15

Docker Compose

Command	What it does	Flags used in the course	Where
<code>docker compose down</code>	stop and remove the project's containers and networks (volumes with -v)	<code>-v</code> also remove volumes, <code>--rm</code> also remove images, <code>--remove-orphans</code>	062 Why Docker Compose? also 063 , 064 , 065 , 066 , 067 , 068 , 069 , 070 and 11 more
<code>docker compose ps</code>	the project's containers and their state	<code>--format</code> Go template output, <code>-a</code> include stopped containers, <code>-no-trunc</code> , <code>-q</code> quiet / IDs only	062 Why Docker Compose? also 063 , 064 , 068 , 069 , 070 , 072 , 074 , A10 and 6 more
<code>docker compose up</code>	create and start the services of a Compose project	<code>-d</code> in the background, <code>--build</code> build images first, <code>--wait</code> wait until services are running/healthy, <code>--quiet-build</code> hide build output, <code>--force-recreate</code> recreate containers even if unchanged, <code>--scale</code> run N containers of a service, <code>--no-deps</code> do not start linked services, <code>--remove-orphans</code> remove containers of services no longer in the file	062 Why Docker Compose? also 063 , 064 , 065 , 066 , 067 , 068 , 069 , 070 and 11 more
<code>docker compose config</code>	validate the Compose file and print it fully resolved	<code>--services</code> list the service names, <code>--quiet</code> only validate, <code>--volumes</code> , <code>--format</code> Go template output, <code>--profiles</code>	063 Your first Compose file also 064 , 066 , 067 , 070 , 074 , A10 , 134 , P07 and 1 more
<code>docker compose logs</code>	the logs of the project's services	<code>--no-log-prefix</code> without the service-name prefix, <code>-t</code> , <code>--tail</code> , <code>--since</code> , <code>-f</code> follow	063 Your first Compose file also 065 , 068 , 071 , A10 , 134 , P03 , P07 , P08 and 1 more
<code>docker compose exec</code>	run a command in a running service container	<code>-T</code> no terminal (for scripts)	064 Services also 065 , 066 , 067 , 068 , 069 , 070 , 071 , 072 and 6 more
<code>docker compose start</code>	start the project's existing containers	—	064 Services also 072 , 074 , P08 , capstone
<code>docker compose stop</code>	stop the project's containers without removing them	—	064 Services also 072 , 074 , P08 , capstone
<code>docker compose run</code>	run a one-off command in a new service container	<code>--rm</code> , <code>--no-deps</code>	070 Profiles also 072 , 073
<code>docker compose build</code>	build the services' images	<code>--quiet</code> , <code>--build-arg</code>	073 Building images with Compose also P08 , capstone

Command	What it does	Flags used in the course	Where
<code>docker compose images</code>	the images the project's containers use	<code>--format</code> Go template output	073 Building images with Compose
<code>docker compose restart</code>	restart services	—	P03 Python API with Redis
<code>docker compose push</code>	push the services' images	<code>--quiet</code>	capstone a production-style container stack

Registries

Command	What it does	Flags used in the course	Where
<code>docker login</code>	log in to a registry	<code>-u</code> user name, <code>--password-stdin</code> read the password from standard input, <code>--username</code>	076 Docker Hub also 078, 079, 082, T24, T25
<code>docker logout</code>	remove a registry's stored credentials	—	076 Docker Hub also 078, T24

Security

Command	What it does	Flags used in the course	Where
<code>docker scout cves</code>	known vulnerabilities (CVEs) in an image's packages	<code>--only-severity</code> , <code>--exit-code</code>	082 Image security: trusted, minimal, pinned, scanned also 137
<code>docker scout quickview</code>	a summary of an image's vulnerabilities	—	082 Image security: trusted, minimal, pinned, scanned

Engine and system

Command	What it does	Flags used in the course	Where
<code>docker info</code>	engine configuration and state	<code>--format</code> Go template output	003 Docker vs virtual machines also 005, A01, 097, 102
<code>docker context ls</code>	the engines this client can talk to	—	004 Docker architecture
<code>docker context show</code>	the active context	—	004 Docker architecture
<code>docker version</code>	client and server versions	<code>--format</code> Go template output	004 Docker architecture also 005
<code>docker system df</code>	disk space used by images, containers, volumes and build cache	<code>-v</code>	056 Inspecting volumes also 091

Options of `docker compose` itself (before the subcommand): `--env-file` variables file, `--profile` enable a profile, `-f` Compose file, `-p` project name.

Glossary

Every term the course uses, in alphabetical order, with the lesson that explains it.

Term	Meaning	Lesson
ARG	a Dockerfile variable that exists only while the image is built; its value can still be seen in the image history, so never use it for secrets	031
Attack surface	everything an attacker could use: packages, shells, open ports, privileges, capabilities. Smaller images and fewer privileges shrink it	080
Base image	the image a Dockerfile starts from (<code>FROM</code>)	023
Bind mount	a host folder or file mounted into a container (<code>-v "\$(pwd) : /app"</code>); changes are visible on both sides	054
Bridge network	the default network type: a private virtual network on one host, with NAT to the outside	047
Build cache	stored results of earlier build steps; a step is reused while it and every step before it are unchanged	039
Build context	the files sent to the builder with <code>docker build PATH</code> ; <code>COPY</code> can only read from it	036
BuildKit	Docker's build engine: parallel stages, cache mounts, build secrets, multi-platform builds	103
Buildx	the CLI for BuildKit (<code>docker buildx build</code>), used for multi-platform images and cache export	103
Capability	a slice of root's power in Linux (<code>NET_BIND_SERVICE</code> , <code>CHOWN</code> , ...); containers get a reduced set, which you can reduce further with <code>--cap-drop</code>	085
cgroups	the Linux kernel feature that limits and measures a process group's CPU, memory and processes; Docker's resource limits use it	097
CMD	the default command of an image; replaced by the arguments of <code>docker run IMAGE ARGS</code>	027
Compose project	the containers, networks and volumes created from one Compose file, named after its folder or <code>-p</code>	063
Container	a running (or stopped) instance of an image: an isolated process with its own writable layer	007
containerd	the container runtime the Docker engine uses to run containers; Kubernetes uses it directly	004
Context (Docker)	a named engine endpoint the client talks to (<code>docker context ls</code>)	004
Dangling image	an image with no tag (<code><none></code>), usually left over when a tag moved to a newer build	019
Digest	the content hash of an image (<code>sha256:...</code>); unlike a tag it always names exactly the same image	020
Distroless	images with only an application's runtime dependencies: no shell, no package manager	089
Docker daemon (dockerd)	the engine: the server that builds images and manages containers, networks and volumes	004
Docker Desktop	Docker for Windows and macOS: the engine runs in a small Linux virtual machine	005
Dockerfile	the recipe of an image: one instruction per step	022
.dockerignore	files and folders left out of the build context	037
Embedded DNS	the resolver at <code>127.0.0.11</code> in containers on user-defined networks; it resolves container and service names	050
ENTRYPOINT	the fixed executable of an image; <code>CMD</code> becomes its default arguments	028
ENV	an environment variable set in the image, available at build time and at run time	030

Term	Meaning	Lesson
Exit code	the number a container's main process ended with: 0 success, 1 application error, 125 Docker error, 126 not executable, 127 command not found, 137 killed (SIGKILL, often out of memory), 143 terminated (SIGTERM)	009
EXPOSE	documents which port the application listens on; it publishes nothing	032
Exec form / shell form	<code>CMD ["node", "server.js"]</code> runs the program directly (it receives signals); <code>CMD node server.js</code> runs it through <code>/bin/sh -c</code>	027
GHCR	GitHub Container Registry, <code>ghcr.io/OWNER/IMAGE</code>	079
Healthcheck	a command Docker runs periodically inside a container; it turns "running" into <code>healthy</code> or <code>unhealthy</code>	092
Image	a read-only template: a stack of file system layers plus metadata (command, environment, ports, user)	007
Immutable container	a container that is never changed after it starts: a change means a new image and a new container	132
Layer	one read-only file system change in an image, created by a Dockerfile instruction; layers are shared between images	017
Logging driver	where a container's output goes: <code>json-file</code> (default), <code>local</code> , <code>syslog</code> , <code>journald</code> , ...	102
Manifest list / image index	one tag pointing to images for several platforms (amd64, arm64); the engine picks its own	104
Multi-stage build	a Dockerfile with several <code>FROM</code> stages; the final image copies only what it needs from earlier stages	087
Named volume	storage managed by Docker (<code>-v data:/path</code>), independent of any container's life	055
Namespace	the Linux kernel feature that gives a process its own view of processes, network, mounts, host name and users	003
OCI	Open Container Initiative: the open standards for image format and runtimes that Docker, containerd and Kubernetes share	004
OOMKilled	the container was killed by the kernel for exceeding its memory limit (exit code 137)	098
PID 1	the container's main process; it receives the stop signal and the container lives as long as it does	009
Port publishing	<code>-p HOST:CONTAINER</code> forwards a host port to a container port	012
Profile (Compose)	a label that starts a service only when asked (<code>--profile debug</code>)	070
Read-only root file system	<code>--read-only</code> : the container cannot change its own files; writable paths are explicit volumes or tmpfs	084
Registry	a server that stores and distributes images (Docker Hub, GHCR, ECR, a private <code>registry:3</code>)	075
Repository	a set of images with the same name and different tags (<code>nginx</code> , <code>ghcr.io/team/api</code>)	077
Restart policy	what Docker does when a container stops: <code>no</code> , <code>on-failure</code> , <code>unless-stopped</code> , <code>always</code>	009
Rootless / non-root	running the container's process as an unprivileged user (<code>USER</code>), so a compromise does not start as root	081
runc	the low-level runtime that creates a container's namespaces and cgroups, then hands over to the process	004
Secret	sensitive configuration (passwords, keys) given to a container at run time as a file, never baked into an image	061

Term	Meaning	Lesson
Service (Compose)	one component of a Compose application; its name is also its DNS name on the project's network	064
Tag	a movable name of an image version (<code>nginx:1.30-alpine</code>); <code>latest</code> is only the default tag, not "the newest"	020
tmpfs	an in-memory file system mounted into a container, emptied when it stops	084
USER	the Dockerfile instruction that sets the user the container runs as	033
User-defined network	a network you create (<code>docker network create</code>); unlike the default bridge it provides DNS by container name	048
Virtual machine	an emulated computer with its own kernel, run by a hypervisor; heavier and more isolated than a container	003
Volume	storage outside a container's writable layer that survives the container	055
WORKDIR	the working directory for the following Dockerfile instructions and for the container	024
Writable layer	the thin per-container layer on top of the image; deleted with the container	053

Docker interview questions

Questions asked in DevOps, platform and backend interviews, from fundamentals to production. Answer out loud first, then open the model answer; the lesson link has the hands-on version.

Fundamentals

1. What is the difference between a container and a virtual machine?

▼ Answer

A VM emulates hardware and runs a full guest OS with its own kernel; a container is an isolated process sharing the host's kernel (namespaces for isolation, cgroups for limits). Containers start in under a second and weigh megabytes; VMs isolate more strongly. In the cloud, containers usually run inside VMs. → [003](#)

2. What is the difference between an image and a container?

▼ Answer

An image is a read-only template of layers plus metadata; a container is a running instance with its own writable layer on top. Many containers can run from one image; deleting a container deletes its writable layer, not the image. → [007](#)

3. Walk through what happens when you run `docker run nginx`.

▼ Answer

The CLI sends API requests to `dockerd`. If the image is not local, the daemon pulls it from the registry (resolving the tag to a digest, downloading missing layers). It creates the container (writable layer, network endpoint, configuration), then `containerd` and `runc` create the namespaces and cgroups and start the image's ENTRYPOINT/CMD as PID 1. → [004](#)

4. A container exits immediately after `docker run -d`. Why, and how do you find out?

▼ Answer

A container lives as long as its main process. Either the command finished (a shell with nothing to do, a one-off script) or it crashed. `docker ps -a` shows the exit code, `docker logs` the output, `docker inspect` the command and `.State.Error`. Exit code 127 is "command not found", 137 a SIGKILL (often out of memory). → [troubleshooting problem 01](#)

5. What do the exit codes 0, 1, 125, 126, 127, 137 and 143 mean?

▼ Answer

0 success; 1 the application failed; 125 Docker itself failed (bad option); 126 the command is not executable; 127 the command was not found; 137 = 128 + 9, killed with SIGKILL (out of memory, or `docker kill`, or a stop timeout); 143 = 128 + 15, terminated with SIGTERM (a normal `docker stop` the application handled). → [009](#)

Images and Dockerfiles

6. What is the difference between CMD and ENTRYPOINT?

▼ Answer

ENTRYPOINT is the executable; CMD is its default arguments (or the default command when there is no ENTRYPOINT). `docker run IMAGE ARGS` replaces CMD; replacing ENTRYPOINT needs `--entrypoint`. Use the exec form (JSON array) so the process is PID 1 and receives signals. → [029](#)

7. What is the difference between ARG and ENV? Can you use ARG for a password?

▼ Answer

ARG exists only during the build; ENV is stored in the image and present at run time. Neither is safe for secrets: ARG values used in a `RUN` appear in `docker history`, and ENV is visible in `docker inspect`. Use BuildKit build secrets (`--secret`) at build time and runtime secrets (files) at run time. → [031](#), [083](#)

8. How does the build cache work, and how do you order a Dockerfile for it?

▼ Answer

Each instruction's result is cached; a step is reused while it and all previous steps are unchanged (for `COPY`, the copied files' content). Once a step changes, every later step reruns. So: base image, then system packages, then dependency manifests (`package.json`, `requirements.txt`) and their install, and the frequently changing source code last. → [039](#)

9. Why does `.dockerignore` matter?

▼ Answer

It keeps files out of the build context: faster builds (less to send), fewer cache invalidations, and no accidental `COPY . .` of `.git`, `.env`, keys or `node_modules` into the image. → [037](#)

10. Why is the `latest` tag dangerous?

▼ Answer

It is only the default tag, not "the newest", and it moves: the same name means different images on different days and machines, so builds and deployments are not reproducible and rollbacks are unclear. Pin versions (`nginx:1.30-alpine`) and deploy by digest. → [021](#)

11. What is a multi-stage build and why use it?

▼ Answer

Several `FROM` stages in one Dockerfile: compile in a stage with the full toolchain, then `COPY --from=build` only the result into a minimal runtime image. Images become much smaller, with fewer packages to patch and attack, and no compilers or source code in production. → [087](#)

12. How do you make an image smaller?

▼ Answer

Multi-stage builds; a minimal base (alpine, slim, distroless, scratch for static binaries); install only production dependencies; clean package caches in the same `RUN`; `.dockerignore`; and measure with `docker image ls` and `docker history` to find the big layers. → [091](#)

Networking and storage

13. What is the difference between `EXPOSE` and `-p` ?

▼ Answer

`EXPOSE` only documents the port the application listens on. `-p HOST:CONTAINER` actually publishes it on the host. Containers on the same network reach each other on any port without either. → [051](#)

14. Why can't two containers on the default bridge reach each other by name?

▼ Answer

Only user-defined networks have Docker's embedded DNS (127.0.0.11) for container names. Create a network and attach both containers; Compose does this automatically for every project. → [048](#), [050](#)

15. The port is published but the application is not reachable. What do you check?

▼ Answer

`docker ps` for the mapping and that it is running; `docker logs` for the address it listens on: an application bound to `127.0.0.1` inside the container is unreachable from outside it, it must listen on `0.0.0.0`; that the container port in `-p` is the one the application really uses; and the host firewall or a port conflict. → [052](#)

16. Volumes vs bind mounts: when do you use which?

▼ Answer

Named volumes for data Docker should manage (databases, uploads): portable, independent of the host's folder layout, initialized from the image. Bind mounts for files you control on the host: source code in development, a config file. → [057](#)

17. A database container was recreated and the data is gone. What happened?

▼ Answer

The data was in the container's writable layer (no volume, or the volume mounted at the wrong path), which is deleted with the container. Mount a named volume at the database's data directory and verify it with `docker inspect ( .Mounts )`. → [058](#)

Compose

18. Does `depends_on` wait until the database is ready?

▼ Answer

Not by default: it only orders container start. Add a healthcheck to the database and use `depends_on: db: condition: service_healthy`; applications should also retry connections, because dependencies can restart later. → [068](#)

19. What does `docker compose down -v` remove that `down` keeps?

▼ Answer

The project's named volumes, and with them the data. `down` removes containers and networks only. → [074](#)

Security

20. Why should containers not run as root?

▼ Answer

Root in the container is root on the kernel, only limited by namespaces, capabilities and seccomp. A vulnerability in the application then starts with root privileges: it can change any file of the container, use the remaining capabilities, and is one kernel or misconfiguration bug from the host. Add a user and `USER` it. → [081](#)

21. Why is access to `/var/run/docker.sock` equivalent to root on the host?

▼ Answer

Whoever can talk to the daemon can start a privileged container that mounts the host's root file system. Never mount the socket into application containers; the `docker` group on Linux is root-equivalent. → [080](#)

22. How would you harden a container at run time?

▼ Answer

Non-root user; `--read-only` with tmpfs for the paths that must be writable; `--cap-drop ALL` and add back only what is needed; `--security-opt no-new-privileges`; memory, CPU and process limits; no privileged mode, no Docker socket; only the ports it needs, published to the right interface. → [084](#), [085](#)

23. How do secrets leak into images, and how do you prevent it?

▼ Answer

`COPY` of a key or `.env` file (even if deleted in a later layer, it stays in the earlier one); `ARG` / `ENV` values (visible in `docker history` / `docker inspect`). Prevent with `.dockerignore`, BuildKit `--secret` mounts for build time and runtime secret files for run time, and by scanning images. → [083](#)

Operations

24. A container is "Up" but the service does not answer. How can Docker know?

▼ Answer

Running only means the process exists. A HEALTHCHECK runs a real check (an HTTP request, `pg_isready`) and marks the container `healthy` or `unhealthy`; orchestrators and Compose can act on it. → [093](#)

25. A container keeps restarting with exit code 137. What do you check?

▼ Answer

`docker inspect --format '{{.State.OOMKilled}}' : true means the memory limit was exceeded. Compare usage in docker stats with the limit; fix the leak or the configuration (for example a JVM heap larger than the limit), or raise the limit deliberately. → 099`

26. How do you build one image for both amd64 and arm64?

▼ Answer

`docker buildx build --platform linux/amd64,linux/arm64 -t NAME --push .` with a builder that supports both (QEMU emulation or native nodes). The registry stores an image index; each host pulls its own platform. → [104](#)

27. What does a production-ready Dockerfile look like?

▼ Answer

Pinned base image (tag or digest), multi-stage, dependencies installed before the source for caching, production dependencies only, non-root `USER`, `exec-form ENTRYPOINT / CMD`, a `HEALTHCHECK` where the platform uses it, labels (version, source), no secrets, `.dockerignore`, and configuration from the environment.
→ [130](#)

28. How does Docker fit into CI/CD and Kubernetes?

▼ Answer

CI builds the image once from the commit, tests and scans it, and pushes it to a registry with an immutable tag (the commit SHA or a version). Every environment deploys that same image by tag or digest; Kubernetes pulls it and runs it as Pods, with the same concepts: ports, environment, volumes, probes, resource limits, the user. → [135](#), [138](#)